Mapping human lymph node cell types to 10X Visium with Cell2location - study - implementation

This tutorial shows how to use **cell2location** method for spatially resolving fine-grained cell types by integrating 10X Visium data with scRNA-seq reference of cell types. Cell2location is a principled Bayesian model that estimates which combination of cell types in which cell abundance could have given the mRNA counts in the spatial data, while modelling technical effects (platform/technology effect, contaminating RNA, unexplained variance).

Note!
Cell2location is an independent package, but is powered by scvi-tools. If you have questions about cell2location, Visium data or scvi-tools please visit https://discourse.scverse.org/c/ecosytem/cell2location/42, https://discourse.scverse.org/c/general/visium/32 or https://discourse.scverse.org/c/help/scvi-tools/7 correspondingly.

Cell2location

Open in Colab

In this tutorial, we analyse a publicly available Visium dataset of the human lymph node from 10X Genomics, and spatially map a comprehensive atlas of 34 reference cell types derived by integration of scRNA-seq datasets from human secondary lymphoid organs.

- Cell2location provides high sensitivity and resolution by borrowing statistical strength across locations. This is achieved by modelling similarity of location patterns between cell types using a hierarchical factorisation of cell abundance into tissue zones as a prior (see paper methods).

- Using our statistical method based on Negative Binomial regression to robustly combine scRNA-seq reference data across technologies and batches results in improved spatial mapping accuracy. Given cell type annotation for each cell, the corresponding reference cell type signatures $g_{f,g}$, which represent the average mRNA count of each gene g in each cell type f, can be estimated from sc/snRNA-seq data using either 1) NB regression or 2) a hard-coded computation of per-cluster average mRNA counts for individual genes. We generally recommend using NB regression. This notebook shows use a dataset composed on multiple batches and technologies.When the batch effects are small, a faster hard-coded method of computing per cluster averages provides similarly high accuracy. We also recommend the hard-coded method for non-UMI technologies such as Smart-Seq 2.

- Cell2location needs untransformed unnormalised spatial mRNA counts as input.

- You also need to provide cell2location with the expected average cell abundance per location which is used as a prior to guide estimation of absolute cell abundance. This value depends on the tissue and can be estimated by counting nuclei for a few locations in the paired histology image but can be approximate (see paper methods for more guidance).

Workflow diagram

see:
https://cell2location.readthedocs.io/en/latest/notebooks/cell2location_tutorial.html

Contents

- Loading packages
- Loading Visium and single cell data

1. Estimating cell type signatures (NB regression)
2. Cell2location: spatial mapping
3. Visualising cell abundance in spatial coordinates
4. Downstream analysis

- Leiden clustering of cell abundance
- Identifying cellular compartments / tissue zones using non-negative matrix factorisation (NMF)
- Estimate cell-type specific expression of every gene in the spatial data (needed for NCEM)

5. Advanced use

- Working with the posterior distribution and computing arbitrary quantiles

In [ ]:

Slide-seq

Slide-seq is a high-resolution spatial transcriptomics technology developed by researchers at the Broad Institute of MIT and Harvard (Fei Chen and Evan Macosko). It measures genome-wide gene expression within tissue sections while retaining the precise spatial location of cells. 1, 2, 3, 4

How It Works

- DNA-Barcoded Beads ("Pucks"): A dense monolayer of 10-micron DNA-barcoded beads (comparable to the size of individual cells) is packed onto a surface called a puck. 1, 2, 3
- RNA Transfer: Fresh-frozen tissue sections are placed directly onto the puck, allowing cellular RNA to transfer onto the underlying beads. 1, 2
- Sequencing & Mapping: Standard high-throughput RNA sequencing is performed. By matching the RNA sequences back to the known spatial coordinates of the bead

barcodes, researchers map gene expression across the tissue without needing a microscope for primary localization. 1, 2

Key Features & Versions

- Original Slide-seq (2019): Achieved 10-micron spatial resolution and enabled 3D tissue volume reconstruction (such as mapping the mouse hippocampus and cerebellum). 1, 2, 3
- Slide-seqV2 (2020): Improved bead synthesis, library generation, and array indexing to capture roughly 50% of single-cell RNA-seq efficiency—a ~10-fold increase in transcript capture over the original method. 1, 2
- Computational Pipeline: Processed using open-source, cloud-optimized workflows like the Broad Institute WARP Slide-seq Pipeline available on platforms like Terra. Commercialized assays are also known under tools like Curio Seeker. 1

In [ ]:

Loading packages

```
uv add scvi-colab
uv add cell2location
```

In [1]:
```
import os, sys
import pickle
import scanpy as sc
import numpy as np
import pandas as pd
from pathlib import Path

import scipy.sparse as sp
import anndata as ad
import scvi
import h5py

import cell2location
import squidpy as sq

'''
sed -i '1i import anndata' ~/uv/squidpy/.venv/lib/python3.12/site-package
head -5 ~/uv/squidpy/.venv/lib/python3.12/site-packages/cell2location/uti

import traceback
try:
    import cell2location
except Exception:
    traceback.print_exc()
'''

import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib import rcParams
from matplotlib.colors import LinearSegmentedColormap
import matplotlib.patheffects as pe
```

```
from scipy.stats import spearmanr
import scipy.sparse as sps


rcParams['pdf.fonttype']= 42 # enables correct plotting of text for PDFs

import warnings

ROOT_SRC = Path("/home/flavio/uv/squidpy/src")
if str(ROOT_SRC) not in sys.path:
    sys.path.append(str(ROOT_SRC))

from libs.Basic import pdwritecsv, pdreadcsv
from libs.matplotlib_figure_style import *
from libs.cell2location_lib import *          # clip_for_plot, create_gen

sc.logging.print_header()
print(f"squidpy=={sq.__version__}")
```

```
squidpy==0.1.dev34+gb2ed182e8.d20261020
```

In [2]:
```
import importlib

importlib.metadata.version('scanpy'),  importlib.metadata.version('squidp
```

Out[2]:
```
('1.12.4', '0.1.dev34+gb2ed182e8.d20261020')
```

First, let's define where we save the results of our analysis:

In [3]:
```
root0 = Path('/home/flavio/uv/squidpy')
root_results = root0 / 'results'
root_figure = root0 / 'figures'
root_data = root0 / 'data'

os.listdir(root_data)[:3]
```

Out[3]:
```
['visium_hne_liana_bivariate.h5ad',
 'sc_mouse_cortex.h5ad',
 'lymphoid_organ_sc.h5ad']
```

Loading Edited Visium and scRNA-seq reference data

First let's read spatial Visium data from 10X Space Ranger output. Here we use lymph node data generated by 10X and presented in Kleshchevnikov et al (section 4, Fig 4). This dataset can be conveniently downloaded and imported using scanpy. See this tutorial for a more extensive and practical example of data loading (multiple visium samples).

Visium data - adata_vis

In [4]:
```
results_folder =  root_results / 'lymph_nodes_analysis/'

# create paths and names to results folders for reference regression and
ref_run_name = Path(f'{results_folder}/reference_signatures')
run_name = Path(f'{results_folder}/cell2location_map')
```

```
adata_file_sp = run_name / "sp.h5ad"
adata_file_sp.exists(), adata_file_sp
```

Out[4]:
```
(True,
 PosixPath('/home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map/sp.h5ad'))
```

In [5]:
```
adata_vis = sc.read_h5ad(adata_file_sp)
print(f"adata_vis read from {adata_file_sp}")

# Reload the trained spatial model bound to that AnnData
mod_c2l = cell2location.models.Cell2location.load(f"{run_name}", adata_vi
print(f"Model loaded from {run_name}")

# mod_c2l.samples is not saved with the model: rebuild it (needed by plot
adata_vis = ensure_posterior_samples(mod_c2l, adata_vis, quantiles=["q05"
print("posterior samples:", sorted(mod_c2l.samples.keys()))
```

```
adata_vis read from /home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map/sp.h5ad
INFO     File /home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map/model.pt already downloaded
Model loaded from /home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map
posterior samples: ['post_sample_q0001', 'post_sample_q05', 'post_sample_q50', 'post_sample_q95']
```

In [6]:
```
adata_vis
```

Out[6]:
```
AnnData object with n_obs × n_vars = 4035 × 10217
    obs: 'in_tissue', 'array_row', 'array_col', 'sample', '_indices', '_scvi_batch', '_scvi_labels', 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IFN', 'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC', 'DC_CCR7+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macrophages_M1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4+', 'T_CD4+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_CD8+_cytotoxic', 'T_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', 'leiden', 'region_cluster', 'mean_nUMI_factorsfact_0', 'mean_nUMI_factorsfact_1', 'mean_nUMI_factorsfact_2', 'mean_nUMI_factorsfact_3', 'mean_nUMI_factorsfact_4', 'mean_nUMI_factorsfact_5', 'mean_nUMI_factorsfact_6', 'mean_nUMI_factorsfact_7', 'mean_nUMI_factorsfact_8', 'mean_nUMI_factorsfact_9', 'mean_nUMI_factorsfact_10', 'mean_nUMI_factorsfact_11', 'region_name'
    var: 'feature_types', 'genome', 'symbol', 'MT_gene'
    uns: '_scvi_manager_uuid', '_scvi_uuid', 'leiden', 'mod', 'mod_coloc_n_fact11', 'mod_coloc_n_fact12', 'neighbors', 'region_cluster_colors', 'sample_colors', 'spatial', 'umap'
    obsm: 'MT', 'X_umap', 'means_cell_abundance_w_sf', 'q0001_cell_abundance_w_sf', 'q05_cell_abundance_w_sf', 'q50_cell_abundance_w_sf', 'q95_cell_abundance_w_sf', 'spatial', 'stds_cell_abundance_w_sf'
    obsp: 'connectivities', 'distances'
    layers: 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IFN', 'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC', 'DC_CCR7+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macrophages_M1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4+', 'T_CD4+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_CD8+_cytotoxic', 'T_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', None (.X)
```

In [7]:
```
print(adata_vis.var.shape[0])
adata_vis.var.head(3)
```

```
10217
```

Out[7]:

gene_ids	feature_types	genome	symbol	MT_gene
ENSG00000000419	Gene Expression	GRCh38	DPM1	False
ENSG00000000457	Gene Expression	GRCh38	SCYL3	False
ENSG00000000938	Gene Expression	GRCh38	FGR	False

In [8]:
```
adata_vis.uns['spatial'].keys()
```

Out[8]:
```
dict_keys(['V1_Human_Lymph_Node'])
```

In [9]:
```
adata_vis.obs.head(5).T.head(8)
```

Out[9]:

	AAACAAGTATCTCCCA-1	AAACAATCTACTAGCA-1	AAACACCAATAACTGC-1	
in_tissue	1	1	1	
array_row	50	3	59	
array_col	102	43	19	
sample	V1_Human_Lymph_Node	V1_Human_Lymph_Node	V1_Human_Lymph_Node	V
_indices	0	1	2	
_scvi_batch	0	0	0	
_scvi_labels	0	0	0	
B_Cycling	0.688599	0.928032	0.947341	

In [10]:
```
adata_vis.var_names[:5]
```

Out[10]:
```
Index(['ENSG00000000419', 'ENSG00000000457', 'ENSG00000000938',
       'ENSG00000000971', 'ENSG00000001036'],
      dtype='object', name='gene_ids')
```

In [11]:
```
print(adata_vis.var.shape)
adata_vis.var.head(5)
```

```
(10217, 4)
```

Out[11]:

gene_ids	feature_types	genome	symbol	MT_gene
ENSG00000000419	Gene Expression	GRCh38	DPM1	False
ENSG00000000457	Gene Expression	GRCh38	SCYL3	False
ENSG00000000938	Gene Expression	GRCh38	FGR	False
ENSG00000000971	Gene Expression	GRCh38	CFH	False
ENSG00000001036	Gene Expression	GRCh38	FUCA2	False

In [12]:
```
"DPM1" in list(adata_vis.var.symbol)
```

Out[12]:
```
True
```

In [13]:
```
genes = [x for x in adata_vis.var.symbol if x.startswith("CD")]
genes.sort()
np.array(genes)[:20]
```

Out[13]:
```
array(['CD14', 'CD151', 'CD160', 'CD163', 'CD163L1', 'CD164', 'CD177',
       'CD180', 'CD19', 'CD1B', 'CD1C', 'CD1D', 'CD1E', 'CD2', 'CD200',
       'CD207', 'CD209', 'CD22', 'CD226', 'CD24'], dtype='<U10')
```

B-cell geneS

Pan B-cell gene and protein markers used to identify B lymphocytes include CD19, MS4A1 (which encodes CD20), CD79A, CD79B, PAX5, and CD22. 1, 2, 3

Key B-Cell Gene and Surface Markers

- Core Pan B-Cell Markers:
  - CD19: Expressed from the early pro-adata_bcell stage through mature B cells; acts as a coreceptor for B-cell receptor (BCR) signaling.
  - MS4A1 (CD20): Encodes the CD20 calcium channel protein; present from the pre-B stage to mature B cells and a primary target for immunotherapies (e.g., rituximab).
  - CD79A / CD79B: Encodes Igα and Igβ, essential signaling components associated with the surface B-cell receptor complex.
  - PAX5: A master transcription factor critical for B-cell commitment and identity.
  - CD22: A sialic acid-binding lectin expressed on mature and developing B cells. 1, 2, 3, 4, 5, 6, 7
- Stage-Specific and Subset Markers:
  - Naïve B Cells: IGHD (IgD), IGHM (IgM), CD27−, IL4R, FCER2 (CD23).
  - Memory B Cells: CD27+, CD24+, and switched or unswitched immunoglobulins (IgG, IgA, IgE).
  - Plasma Cells / Plasmablasts: Downregulate CD19 and MS4A1/CD20 while upregulating CD38, SDC1 (CD138), PRDM1 (Blimp-1), and XBP1. 1, 2, 3

In [ ]:
```
```

Which genes are unique when comparing the B cell types? post-study

GC DZ, GC LZ, GC prePB, B cycling, preGC, naive, activated, B IFN, B mem, and B plasma

In [14]:
```
fname_sc = 'lymphoid_organ_sc.h5ad'
filename = root_data / fname_sc

print(f"{os.path.getsize(filename)/1e9:.3f} GB")
filename.exists(), filename
```

```
1.662 GB
```

Out[14]: (True, PosixPath('/home/flavio/uv/squidpy/data/lymphoid_organ_sc.h5ad'))

In [15]:
```
adata_ref = ad.read_h5ad(filename)
print(f"Data read from {filename}")

adata_ref
```

```
Data read from /home/flavio/uv/squidpy/data/lymphoid_organ_sc.h5ad
```

Out[15]:
```
AnnData object with n_obs × n_vars = 73260 × 10237
    obs: 'Age', 'BCELL_CLONE', 'BCELL_CLONE_SIZE', 'Donor', 'ID', 'IGH_MU_FREQ', 'ISOTYPE', 'LibraryID', 'Method', 'Population', 'PrelimCellType', 'Sample', 'Sex', 'Study', 'Tissue', 'barcode', 'batch', 'doublet_score', 'index', 'predicted_doublet', 'percent_mito', 'n_counts', 'n_genes', 'S_score', 'G2M_score', 'phase', 'VDJsum', 'cell_cycle_diff', 'PrelimCellType_new', 'leiden', 'leiden_1', 'leiden_2', 'leiden_3', 'leiden_4', 'CellType', 'CellType2', 'Subset', 'Subset_Broad', 'Subset_all', 'new_celltype', 'Subset_int', 'Subset_print'
    var: 'GeneID-2', 'GeneName-2', 'feature_types', 'feature_types-0', 'feature_types-1', 'gene_ids-1', 'gene_ids-4861STDY7135913-0', 'gene_ids-4861STDY7135914-0', 'gene_ids-4861STDY7208412-0', 'gene_ids-4861STDY7208413-0', 'gene_ids-Human_colon_16S7255677-0', 'gene_ids-Human_colon_16S7255678-0', 'gene_ids-Human_colon_16S8000484-0', 'gene_ids-Pan_T7935494-0', 'genome-1', 'n_cells', 'nonz_mean', 'mean_cov_effect_Subset_B_Cycling', 'mean_cov_effect_Subset_B_GC_DZ', 'mean_cov_effect_Subset_B_GC_LZ', 'mean_cov_effect_Subset_B_GC_prePB', 'mean_cov_effect_Subset_B_IFN', 'mean_cov_effect_Subset_B_activated', 'mean_cov_effect_Subset_B_mem', 'mean_cov_effect_Subset_B_naive', 'mean_cov_effect_Subset_B_plasma', 'mean_cov_effect_Subset_B_preGC', 'mean_cov_effect_Subset_DC_CCR7+', 'mean_cov_effect_Subset_DC_cDC1', 'mean_cov_effect_Subset_DC_cDC2', 'mean_cov_effect_Subset_DC_pDC', 'mean_cov_effect_Subset_Endo', 'mean_cov_effect_Subset_FDC', 'mean_cov_effect_Subset_ILC', 'mean_cov_effect_Subset_Macrophages_M1', 'mean_cov_effect_Subset_Macrophages_M2', 'mean_cov_effect_Subset_Mast', 'mean_cov_effect_Subset_Monocytes', 'mean_cov_effect_Subset_NK', 'mean_cov_effect_Subset_NKT', 'mean_cov_effect_Subset_T_CD4+', 'mean_cov_effect_Subset_T_CD4+_TfH', 'mean_cov_effect_Subset_T_CD4+_TfH_GC', 'mean_cov_effect_Subset_T_CD4+_naive', 'mean_cov_effect_Subset_T_CD8+_CD161+', 'mean_cov_effect_Subset_T_CD8+_cytotoxic', 'mean_cov_effect_Subset_T_CD8+_naive', 'mean_cov_effect_Subset_T_TIM3+', 'mean_cov_effect_Subset_T_TfR', 'mean_cov_effect_Subset_T_Treg', 'mean_cov_effect_Subset_VSMC', 'mean_sample_effectSample_4861STDY7135913', 'mean_sample_effectSample_4861STDY7135914', 'mean_sample_effectSample_4861STDY7208412', 'mean_sample_effectSample_4861STDY7208413', 'mean_sample_effectSample_4861STDY7462253', 'mean_sample_effectSample_4861STDY7462254', 'mean_sample_effectSample_4861STDY7462255', 'mean_sample_effectSample_4861STDY7462256', 'mean_sample_effectSample_4861STDY7528597', 'mean_sample_effectSample_4861STDY7528598', 'mean_sample_effectSample_4861STDY7528599', 'mean_sample_effectSample_4861STDY7528600', 'mean_sample_effectSample_BCP002_Total', 'mean_sample_effectSample_BCP003_Total', 'mean_sample_effectSample_BCP004_Total', 'mean_sample_effectSample_BCP005_Total', 'mean_sample_effectSample_BCP006_Total', 'mean_sample_effectSample_BCP008_Total', 'mean_sample_effectSample_BCP009_Total', 'mean_sample_effectSample_Human_colon_16S7255677', 'mean_sample_effectSample_Human_colon_16S7255678', 'mean_sample_effectSample_Human_colon_16S8000484', 'mean_sample_effectSample_Pan_T7935494'
    uns: 'Age_colors', 'Donor_colors', 'LibraryID_colors', 'Method_colors', 'Study_colors', 'Subset_Broad_colors', 'Subset_colors', 'Tissue_colors', 'leiden', 'neighbors', 'pca', 'phase_colors', 'regression_mod', 'umap'
    obsm: 'X_pca', 'X_umap'
    obsp: 'connectivities', 'distances'
    layers: None (.X)
```

In [16]:
```
X=adata_ref.X
print(type(X), getattr(X,'dtype',None))

with h5py.File(filename,'r') as f:
    g=f['X']
```

```
    print(dict(g.attrs) if isinstance(g,h5py.Group) else g.shape)
    for k in f['layers'].keys() if 'layers' in f else []: print('layer',k
```

```
<class 'scipy.sparse._csr.csr_matrix'> float32
{'encoding-type': 'csr_matrix', 'encoding-version': '0.1.0', 'shape': arra
y([73260, 10237])}
```

In [ ]:

In [17]:
```
warnings.filterwarnings("ignore")

btypes=["B_GC_DZ","B_GC_LZ","B_GC_prePB","B_Cycling","B_preGC","B_naive",

adata_bcell = adata_ref[adata_ref.obs.Subset.isin(btypes)].copy()

adata_bcell.obs["Subset"]=pd.Categorical(adata_bcell.obs.Subset.astype(st
print(adata_bcell.shape, adata_bcell.obs.Subset.value_counts().reindex(bt

adata_bcell.layers["counts"]=adata_bcell.X.copy()

sc.pp.normalize_total(adata_bcell,target_sum=1e4)
sc.pp.log1p(adata_bcell)

# drop genes expressed in <1% of adata_bcell cells to speed up
keep=np.asarray((adata_bcell.layers["counts"]>0).mean(0)).ravel()>=0.01
adata_bcell=adata_bcell[:,keep].copy()
print("genes tested:",adata_bcell.n_vars)

sc.tl.rank_genes_groups(adata_bcell,"Subset",method="wilcoxon",pts=True,u

# per-group df_pct and mean (log1p) for specificity vs every other adata_
Xb=(adata_bcell.layers["counts"]>0)
grp=adata_bcell.obs.Subset.values

df_pct=pd.DataFrame({g:np.asarray(Xb[(grp==g)].mean(0)).ravel() for g in
mexp=pd.DataFrame({g:np.asarray(np.expm1(adata_bcell.X[(grp==g)]).mean(0)
df_pct.to_pickle(root_results / "b_pct.pkl"); mexp.to_pickle(root_results

pickle.dump(adata_bcell.uns["rank_genes_groups"],open(root_results / "b_r
adata_bcell.obs[["Subset","Donor","Study","Tissue"]].to_pickle(root_resul

print("done")
```

```
(38309, 10237) {'B_GC_DZ': 2500, 'B_GC_LZ': 3298, 'B_GC_prePB': 74, 'B_Cyc
ling': 4765, 'B_preGC': 404, 'B_naive': 8924, 'B_activated': 3575, 'B_IF
N': 199, 'B_mem': 13476, 'B_plasma': 1094}
genes tested: 8375
done
```

In [18]:
```
btypes
```

Out[18]:
```
['B_GC_DZ',
 'B_GC_LZ',
 'B_GC_prePB',
 'B_Cycling',
 'B_preGC',
 'B_naive',
 'B_activated',
 'B_IFN',
 'B_mem',
 'B_plasma']
```

In [19]:
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[19]:
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [20]:
```
rgg=adata_bcell.uns["rank_genes_groups"]

# CP10k pseudocount
eps=0.05

rows=[]
for btype in btypes:
    other_bcells=[o for o in btypes if o!=btype]
    dfw = c2l.wtab(btype, rgg)

    dic = {"pct_in": df_pct[btype], "max_pct_other": df_pct[other_bcells]
           "next_pct_type": df_pct[other_bcells].idxmax(1),
           "mean_cp10k": mexp[btype],"max_mean_other": mexp[other_bcells]
           "next_mean_type": mexp[other_bcells].idxmax(1)}

    df=pd.DataFrame(dic)
    df["log2fc_vs_next"]=np.log2((df.mean_cp10k+eps)/(df.max_mean_other+e
    df=df.join(dfw)
    df["group"]=btype
    rows.append(df.reset_index().rename(columns={"index":"gene"}))

dft=pd.concat(rows)

strict=(dft.padj<0.05)&(dft.pct_in>=0.30)&(dft.max_pct_other<=0.25)&(dft.
relax =(dft.padj<0.05)&(dft.pct_in>=0.30)&(dft.max_pct_other<=0.50)&(dft.

dft["tier"]=np.where(strict,"unique",np.where(relax,"enriched (shared)","

# donor consistency for candidates
obs=adata_bcell.obs
donors=obs.Donor.astype(str).values
grp=obs.Subset.astype(str).values

cand=dft[dft.tier!=""]
Xn=adata_bcell.X  # log1p

cnt=pd.crosstab(donors,grp)

dc=[c2l.donor_consistency(adata_bcell, r.gene,r.group, btypes, donors, gr
dft.loc[dft.tier!="","donors_top"]=[f"{a}/{b}" for a,b in dc]

dff=dft[dft.tier!=""].sort_values(["group","tier","log2fc_vs_next"],ascen
```

```
dff["group"]=pd.Categorical(dff.group,btypes)
dff=dff.sort_values(["group","tier","log2fc_vs_next"],ascending=[True,Fal

cols=["group","gene","tier","pct_in","max_pct_other","next_pct_type",
      "log2fc_vs_next","next_mean_type","lfc_vs_restB","padj","donors_top

dff[cols].round(3).to_csv(root_results / "bcell_subset_unique_markers.csv

dff_summ=dff.groupby(["group","tier"],observed=True).size().unstack(fill_
print(dff_summ)

for btype in btypes:
    dff_u = dff[(dff.group==btype)&(dff.tier=="unique")].head(12)
    dff_e = dff[(dff.group==btype)&(dff.tier!="unique")].head(6)

    print(f"\n## {btype}\n U:", ", ".join(f"{r.gene}({r.pct_in*100:.0f}/{
    print(" E:", ", ".join(f"{r.gene}({r.pct_in*100:.0f}/{r.max_pct_other

cnt.reindex(columns=btypes)
```

```
tier         enriched (shared)  unique
group
B_GC_DZ                     18       3
B_GC_LZ                     13       0
B_GC_prePB                  47      14
B_Cycling                  107     132
B_preGC                     17       9
B_activated                  5       1
B_IFN                       23      19
B_mem                        2       0
B_plasma                    51      41

## B_GC_DZ
 U: SH3TC1(36/16%,2.2,8/8), USP53(32/17%,1.8,8/8), SLC2A5(38/23%,1.5,8/8)
 E: SUGCT(66/36%,2.0 vs B_Cycling), CDK13(42/33%,1.2 vs B_GC_LZ), MME(50/3
8%,1.2 vs B_Cycling), KMT2A(45/38%,1.2 vs B_Cycling), RNF144B(37/30%,1.0 v
s B_GC_LZ), PEX5(34/29%,1.0 vs B_GC_prePB)

## B_GC_LZ
 U:
 E: LMO2(51/27%,1.5 vs B_GC_DZ), GMDS(56/47%,1.3 vs B_Cycling), PLEK(52/4
0%,1.0 vs B_preGC), LYSMD2(49/39%,0.8 vs B_GC_DZ), DNAJC10(61/48%,0.8 vs B
_GC_DZ), PHF6(51/47%,0.7 vs B_GC_prePB)

## B_GC_prePB
 U: ARHGAP10(31/6%,2.7,1/1), BAIAP2L1(39/7%,2.7,1/1), FRZB(54/17%,2.1,1/
1), BTNL9(35/10%,1.7,1/1), CNR1(31/13%,1.3,1/1), ARMT1(31/22%,1.2,1/1), GP
R137B(45/19%,1.2,1/1), RBKS(45/22%,1.1,1/1), RAB9A(45/24%,1.1,1/1), SMAD5
(35/20%,1.0,1/1), NT5C3B(35/19%,1.0,1/1), REPIN1(41/24%,1.0,1/1)
 E: RASSF6(78/48%,1.9 vs B_plasma), LMO4(64/36%,1.6 vs B_GC_LZ), ALDH2(74/
36%,1.6 vs B_GC_LZ), HIST1H2AC(53/26%,1.4 vs B_activated), HIST1H2BK(31/2
5%,1.4 vs B_plasma), SMAP1(58/37%,1.3 vs B_GC_DZ)

## B_Cycling
 U: RRM2(42/2%,4.1,8/8), HIST1H3B(30/2%,4.0,8/8), CDK1(56/4%,4.0,8/8), ASF
1B(53/3%,4.0,8/8), CCNA2(52/2%,4.0,8/8), HIST1H1B(40/3%,3.9,8/8), UBE2C(6
7/7%,3.9,8/8), TYMS(74/9%,3.8,8/8), SPC25(38/1%,3.8,8/8), TOP2A(60/5%,3.8,
8/8), KIFC1(49/2%,3.8,8/8), CDCA3(52/3%,3.7,8/8)
 E: HMGB2(96/49%,2.5 vs B_GC_DZ), DUT(89/40%,2.5 vs B_GC_DZ), TUBB4B(78/3
4%,2.3 vs B_activated), STMN1(96/49%,2.1 vs B_GC_DZ), KPNA2(58/30%,1.8 vs
B_GC_prePB), CKS2(78/40%,1.7 vs B_GC_LZ)

## B_preGC
 U: LILRA4(39/6%,3.4,5/5), CRIP3(31/7%,2.4,5/5), CCND2(37/13%,2.4,5/5), MI
R155HG(55/25%,2.2,5/5), NOP16(36/23%,1.4,5/5), SLAMF1(34/23%,1.1,5/5), FKB
P4(32/17%,1.1,5/5), PARVB(34/18%,1.1,5/5), MRTO4(32/24%,1.1,5/5)
 E: DNPH1(55/36%,1.3 vs B_IFN), NOLC1(34/28%,1.1 vs B_Cycling), DDX21(62/4
8%,1.1 vs B_activated), TMEM147(37/39%,1.0 vs B_plasma), GM2A(35/26%,1.0 v
s B_activated), WDR43(30/21%,1.0 vs B_GC_LZ)

## B_naive
 U:
 E:

## B_activated
 U: MYADM(37/21%,1.1,7/9)
 E: NFKBIZ(37/25%,1.1 vs B_preGC), NR4A1(64/44%,1.0 vs B_mem), PDE4B(63/4
9%,0.7 vs B_naive), CSRNP1(40/28%,0.7 vs B_preGC), SERTAD1(39/33%,0.6 vs B
_mem)
```

```
## B_IFN
 U: IFIT3(56/2%,5.0,2/2), IFIT1(51/1%,4.8,2/2), IFI44L(73/7%,3.9,2/2), IFI
6(68/18%,3.7,2/2), IFIT2(40/5%,3.6,2/2), SAMD9L(49/14%,2.6,2/2), MX2(60/1
6%,2.6,2/2), GBP1(35/8%,2.4,2/2), EIF2AK2(58/20%,2.3,2/2), XAF1(54/14%,2.
3,2/2), PLSCR1(49/15%,2.2,2/2), STAT1(75/23%,2.2,2/2)
 E: ISG15(77/31%,3.1 vs B_GC_prePB), MX1(82/33%,2.3 vs B_GC_DZ), IRF7(76/4
5%,2.3 vs B_activated), LY6E(87/46%,2.2 vs B_naive), IFI35(52/27%,2.0 vs B
_naive), RNF213(59/27%,1.7 vs B_mem)

## B_mem
 U:
 E: TNFRSF13B(51/30%,1.0 vs B_IFN), CLECL1(40/21%,0.8 vs B_IFN)

## B_plasma
 U: IGHG2(56/16%,5.1,6/6), IGHGP(35/4%,4.8,6/6), DERL3(92/18%,4.5,6/6), FK
BP11(92/17%,3.6,6/6), FNDC3B(39/4%,2.9,6/6), ANKRD28(48/11%,2.7,6/6), PRDX
4(82/24%,2.5,6/6), TXNDC5(41/7%,2.4,6/6), LIME1(35/5%,2.4,5/6), MANEA(36/
6%,2.3,6/6), ERN1(38/12%,2.1,6/6), MEI1(40/6%,2.1,6/6)
 E: IGHG3(56/38%,4.6 vs B_GC_prePB), IGHG4(57/27%,3.7 vs B_GC_prePB), IGLC
3(41/35%,2.9 vs B_naive), SDF2L1(85/30%,2.3 vs B_preGC), KDELR2(79/39%,2.2
vs B_Cycling), FKBP2(88/31%,2.1 vs B_GC_prePB)
```

Out[20]:

col_0	B_GC_DZ	B_GC_LZ	B_GC_prePB	B_Cycling	B_preGC	B_naive	B_activated	B
row_0								
290b	0	0	0	3	0	1416	17	
298c	0	0	0	0	0	180	18	
302c	1	1	0	2	0	92	31	
390c	1	0	0	9	5	71	164	
417c	9	1	0	1	2	48	6	
A16	41	113	2	149	82	2836	1883	
BCP2	140	271	12	365	7	127	68	
BCP3	127	120	4	115	19	514	154	
BCP4	74	74	0	52	7	136	49	
BCP5	1263	1769	34	2583	152	1311	805	
BCP6	194	162	1	609	71	1263	2	
BCP8	324	432	9	484	24	351	170	
BCP9	326	355	12	393	35	579	208	

In [21]:
```
dff.head(3)
```

Out[21]:

	gene	pct_in	max_pct_other	next_pct_type	mean_cp10k	max_mean_other	r
1941	SH3TC1	0.3572	0.160756	B_Cycling	1.701142	0.340692	
2118	USP53	0.3236	0.173347	B_Cycling	1.492049	0.379549	
44	SLC2A5	0.3824	0.229730	B_GC_prePB	1.500510	0.489497	

In [22]:
```
# summary table for the response
cols=["group","gene","pct_in","max_pct_other","log2fc_vs_next","donors_to

dftab=dff[dff.tier=="unique"].groupby("group",observed=True)[cols]
dftab.head(10)
```

Out[22]:

	group	gene	pct_in	max_pct_other	log2fc_vs_next	donors_top
1941	B_GC_DZ	SH3TC1	0.357200	0.160756	2.164193	8/8
2118	B_GC_DZ	USP53	0.323600	0.173347	1.843953	8/8
44	B_GC_DZ	SLC2A5	0.382400	0.229730	1.523056	8/8
2156	B_GC_prePB	ARHGAP10	0.310811	0.062000	2.680541	1/1
3272	B_GC_prePB	BAIAP2L1	0.391892	0.070934	2.667115	1/1
1278	B_GC_prePB	FRZB	0.540541	0.171846	2.143279	1/1
2579	B_GC_prePB	BTNL9	0.351351	0.097938	1.736841	1/1
2908	B_GC_prePB	CNR1	0.310811	0.129799	1.251970	1/1
3028	B_GC_prePB	ARMT1	0.310811	0.224974	1.208161	1/1
831	B_GC_prePB	GPR137B	0.445946	0.187386	1.197037	1/1
937	B_GC_prePB	RBKS	0.445946	0.217404	1.113327	1/1
3468	B_GC_prePB	RAB9A	0.445946	0.241973	1.111815	1/1
2434	B_GC_prePB	SMAD5	0.351351	0.203839	1.040969	1/1
880	B_Cycling	RRM2	0.424344	0.020110	4.051606	8/8
2645	B_Cycling	HIST1H3B	0.303253	0.017200	4.023038	8/8
4908	B_Cycling	CDK1	0.559706	0.037599	4.004521	8/8
7689	B_Cycling	ASF1B	0.525079	0.025126	3.984043	8/8
2123	B_Cycling	CCNA2	0.519203	0.018000	3.968422	8/8
2686	B_Cycling	HIST1H1B	0.401679	0.033200	3.946485	8/8
7408	B_Cycling	UBE2C	0.667156	0.073200	3.878398	8/8
7154	B_Cycling	TYMS	0.735572	0.094400	3.804863	8/8
1237	B_Cycling	SPC25	0.383841	0.012432	3.803988	8/8
6888	B_Cycling	TOP2A	0.603358	0.046400	3.788887	8/8
8031	B_preGC	LILRA4	0.391089	0.059958	3.382827	5/5
2848	B_preGC	CRIP3	0.314356	0.074591	2.401054	5/5
5125	B_preGC	CCND2	0.366337	0.125628	2.377772	5/5
8291	B_preGC	MIR155HG	0.547030	0.245300	2.151793	5/5
2538	B_preGC	NOP16	0.358911	0.228122	1.441513	5/5
610	B_preGC	SLAMF1	0.339109	0.233846	1.135612	5/5
5119	B_preGC	FKBP4	0.319307	0.168671	1.082566	5/5
8253	B_preGC	PARVB	0.341584	0.175944	1.068152	5/5
93	B_preGC	MRTO4	0.316832	0.242183	1.061393	5/5
8022	B_activated	MYADM	0.370909	0.211055	1.140163	7/9
4981	B_IFN	IFIT3	0.562814	0.021594	4.989528	2/2
4982	B_IFN	IFIT1	0.507538	0.013802	4.813535	2/2
347	B_IFN	IFI44L	0.733668	0.070644	3.903902	2/2
149	B_IFN	IFI6	0.678392	0.184644	3.702381	2/2

	group	gene	pct_in	max_pct_other	log2fc_vs_next	donors_top
4980	B_IFN	IFIT2	0.396985	0.050015	3.577789	2/2
3261	B_IFN	SAMD9L	0.487437	0.142596	2.593246	2/2
8331	B_IFN	MX2	0.597990	0.157688	2.551365	2/2
365	B_IFN	GBP1	0.346734	0.081007	2.440760	2/2
959	B_IFN	EIF2AK2	0.582915	0.204407	2.290159	2/2
6717	B_IFN	XAF1	0.537688	0.135105	2.280990	2/2
6017	B_plasma	IGHG2	0.558501	0.161250	5.130054	6/6
6018	B_plasma	IGHGP	0.350091	0.040541	4.796766	6/6
8117	B_plasma	DERL3	0.920475	0.176285	4.548229	6/6
5248	B_plasma	FKBP11	0.917733	0.173267	3.601603	6/6
1828	B_plasma	FNDC3B	0.390311	0.040541	2.883537	6/6
1474	B_plasma	ANKRD28	0.484461	0.113956	2.723938	6/6
3489	B_plasma	PRDX4	0.824497	0.237624	2.453562	6/6
2606	B_plasma	TXNDC5	0.407678	0.074292	2.440852	6/6
7481	B_plasma	LIME1	0.352834	0.053305	2.376275	5/6
2918	B_plasma	MANEA	0.360146	0.058972	2.251941	6/6

In [23]:
```
for btype in ["B_naive", "B_mem", "B_GC_LZ", "B_activated"]:

    dfa=dft[(dft.group==btype) & (dft.padj<0.05) & (dft.pct_in>=0.3)]
    dfa = dfa.sort_values("log2fc_vs_next",ascending=False).head(8)
    print(btype, "| best vs next:", ", ".join(f"{r.gene}({r.pct_in*100:.0

    dfw = c2l.wtab(btype, rgg).head(8)
    print("   top Wilcoxon vs rest-B:", ", ".join(f"{i}({row.lfc_vs_restB

# naive vs mem head to head
sc.tl.rank_genes_groups(adata_bcell,"Subset",groups=["B_naive"],reference
nm=sc.get.rank_genes_groups_df(adata_bcell,"B_naive",key="nm")

print("")
print("naive>mem:", nm.head(10)[["names","logfoldchanges"]].round(2).valu
print("mem>naive:", nm.tail(10)[["names","logfoldchanges"]].round(2).valu
```

```
B_naive | best vs next: TXNIP(73/71%,0.55 vs B_mem), RNASE6(31/29%,0.46 vs
B_activated), CD52(99/100%,0.25 vs B_IFN), TRAF3IP3(41/45%,0.24 vs B_activ
ated), CXCR4(84/90%,0.23 vs B_mem), ANAPC16(65/74%,0.19 vs B_GC_prePB), GA
BPB1-AS1(31/33%,0.15 vs B_GC_DZ), RPL26(100/100%,0.14 vs B_mem)
   top Wilcoxon vs rest-B: IGHD(2.7), RPL18A(0.5), IGHM(2.1), RPS19(0.5),
RPS27(0.6), CD52(1.0), FCER2(2.3), RPL13(0.5)
B_mem | best vs next: TNFRSF13B(51/30%,0.96 vs B_IFN), CLECL1(40/21%,0.84
vs B_IFN), GPR183(75/55%,0.68 vs B_IFN), S100A4(39/28%,0.54 vs B_IFN), RIC
3(33/28%,0.37 vs B_IFN), PLP2(39/43%,0.35 vs B_IFN), C6orf48(52/50%,0.34 v
s B_naive), VIM(71/60%,0.34 vs B_plasma)
   top Wilcoxon vs rest-B: B2M(1.2), GPR183(3.3), RPS14(0.7), HLA-A(1.6),
RPS29(1.0), RPL36(0.6), RPL34(0.6), TNFRSF13B(3.7)
B_GC_LZ | best vs next: LMO2(51/27%,1.50 vs B_GC_DZ), GMDS(56/47%,1.30 vs
B_Cycling), PLEK(52/40%,1.02 vs B_preGC), LYSMD2(49/39%,0.79 vs B_GC_DZ),
DNAJC10(61/48%,0.78 vs B_GC_DZ), PHF6(51/47%,0.74 vs B_GC_prePB), SKAP2(4
7/42%,0.71 vs B_IFN), NIN(43/39%,0.70 vs B_GC_prePB)
   top Wilcoxon vs rest-B: RGS13(3.8), LRMP(2.9), ACTG1(1.9), MS4A1(1.6),
ARPC2(1.5), SERF2(1.4), PRPSAP2(3.3), MARCKSL1(2.3)
B_activated | best vs next: MYADM(37/21%,1.14 vs B_naive), FOS(92/77%,1.12
vs B_IFN), NFKBIZ(37/25%,1.12 vs B_preGC), JUN(86/74%,1.04 vs B_plasma), N
R4A1(64/44%,0.99 vs B_mem), IER2(67/62%,0.90 vs B_IFN), ZFP36(86/76%,0.86
vs B_preGC), CD69(93/80%,0.79 vs B_IFN)
   top Wilcoxon vs rest-B: FOS(2.7), CD69(2.5), DUSP1(2.1), CD83(1.8), JUN
B(2.2), JUN(2.1), ZFP36(1.8), SLC2A3(1.9)

naive>mem: [['TCL1A', 5.460000038146973], ['IGHD', 3.1700000762939453],
['TMSB10', 0.8199999928474426], ['FCER2', 2.450000047683716], ['IGHM', 1.8
300000429153442], ['RPL18A', 0.3700000047683716], ['RPS19', 0.370000004768
3716], ['CD79A', 0.6800000071525574], ['HLA-DRB1', 0.7200000286102295],
['CD72', 1.7100000381469727]]
mem>naive: [['GPR183', -2.4100000858306885], ['CD27', -3.289999961853027
3], ['TNFRSF13B', -3.359999895095825], ['RPS14', -0.33000001311302185],
['VIM', -1.8200000524520874], ['FOSB', -1.7699999809265137], ['H3F3B', -0.
9800000190734863], ['NR4A2', -1.6699999570846558], ['B2M', -0.349999994039
5355], ['PPP1R15A', -1.4900000095367432]]
```

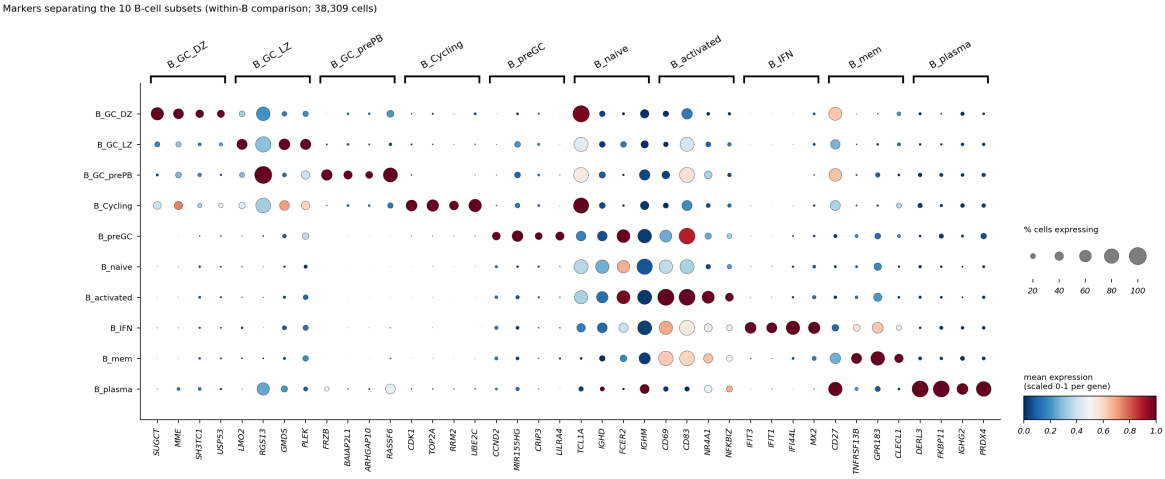
In [ ]:

In [24]:
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[24]:
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [25]:
```
%matplotlib inline

markers={"B_GC_DZ":["SUGCT","MME","SH3TC1","USP53"],
 "B_GC_LZ":["LMO2","RGS13","GMDS","PLEK"],
 "B_GC_prePB":["FRZB","BAIAP2L1","ARHGAP10","RASSF6"],
 "B_Cycling":["CDK1","TOP2A","RRM2","UBE2C"],
 "B_preGC":["CCND2","MIR155HG","CRIP3","LILRA4"],
 "B_naive":["TCL1A","IGHD","FCER2","IGHM"],
 "B_activated":["CD69","CD83","NR4A1","NFKBIZ"],
 "B_IFN":["IFIT3","IFIT1","IFI44L","MX2"],
 "B_mem":["CD27","TNFRSF13B","GPR183","CLECL1"],
 "B_plasma":["DERL3","FKBP11","IGHG2","PRDX4"]}


dp = c2l.dotplot_bcell_markers(adata_bcell, markers, root_figure)
```

```
missing: []
```



Comparing the 10 B-cell subsets against each other only

Not against T or myeloid cells. That way a gene counts as "unique" only if it separates one B subset from all the other B subsets. Per subset, I'll combine:

1. a one-vs-rest Wilcoxon test within the B-cell compartment (log1p CP10k);
2. a stricter specificity filter: the gene must beat the *next-highest* B subset by ≥ 1 log2 fold, be detected in ≥ 30% of the subset's cells, and in ≤ 25% of cells of every other B subset;
3. a donor-consistency check, because cell-level tests overstate significance when cells come from few donors.

The test ran on 38,309 B cells and 8,375 genes. Subset sizes range from 74 (`B_GC_prePB`) to 13,476 (`B_mem`). Now applying the specificity filter and the donor check:

Three subsets (B_naive, B_mem, B_GC_LZ) lack a gene passing the strict filter, so I'll find what best distinguishes them from their nearest neighbors and then build a dot plot of the selected markers.

Naive markers like TCL1A, IGHD, and FCER2 only stand out when naive cells are compared directly against memory cells, since activated, pre-GC, and IFN B subsets also carry them in the full 10-way comparison. I'll generate the dot plot next.

Six of the 10 B-cell subsets have clearly unique genes: plasma, cycling, IFN, pre-plasmablast (prePB), pre-GC and dark zone (DZ). Naive B, memory B and GC light zone (LZ) have **no** gene that is unique against all the other B subsets. Activated B has only one marginal one (*MYADM*). These four are defined by combinations of genes, or by contrast with one neighbouring subset.

Markers separating the 10 B-cell subsets

I compared the B subsets only against each other: 38,309 B cells, 8,375 genes, log1p(CP10k), one-vs-rest Wilcoxon test. A gene counts as **unique** if it meets all of these:

- adjusted p < 0.05;
- detected in ≥ 30% of the subset's cells;
- detected in ≤ 25% of cells in *every* other B subset;
- at least 2× the mean of the next-highest B subset.

"Donors" is the number of donors (each with ≥ 20 cells of that subset) in which the gene is still highest in that subset.

Subset (cells)	Unique genes (top)	% in subset / max % in other B subsets	Donors
B_plasma (1,094)	*DERL3, FKBP11, IGHG2, IGHGP, PRDX4, TXNDC5, FNDC3B*	*DERL3* 92 / 18	6/6
B_Cycling (4,765)	*CDK1, TOP2A, RRM2, UBE2C, CCNA2, TYMS, ASF1B* (132 genes in total)	*CDK1* 56 / 4	8/8
B_IFN (199)	*IFIT3, IFIT1, IFI44L, IFI6, MX2, STAT1, XAF1*	*IFIT3* 56 / 2	2/2
B_GC_prePB (74)	*FRZB, BAIAP2L1, ARHGAP10, BTNL9, CNR1*	*BAIAP2L1* 39 / 7	1/1
B_preGC (404)	*CCND2, MIR155HG, CRIP3, SLAMF1, LILRA4*	*CCND2* 37 / 13	5/5
B_GC_DZ (2,500)	*SH3TC1, USP53, SLC2A5* (unique); *SUGCT, MME* (CD10), shared with cycling	*SH3TC1* 36 / 16	8/8
B_activated (3,575)	*MYADM* only; otherwise immediate-early genes *CD69, CD83, FOS, NR4A1, NFKBIZ*	*MYADM* 37 / 21	7/9
B_GC_LZ (3,298)	none; best are *LMO2* (51 / 27%, 2.8× DZ), *RGS13, GMDS, PLEK* (shared with DZ, prePB, cycling)	—	—
B_mem (13,476)	none; best are *TNFRSF13B* (TACI), *CLECL1, GPR183* (shared with IFN B)	—	—
B_naive (8,924)	none; *IGHD, FCER2, TCL1A, IGHM* are shared with the activated, pre-GC and IFN B cells	—	—

Naive vs memory, head to head: *TCL1A* (log2FC 5.5), *IGHD* (3.2), *FCER2* (2.5) and *CD72* are higher in naive cells. *TNFRSF13B* (3.4), *CD27* (3.3) and *GPR183* (2.4) are higher in memory cells. So the practical rule for naive B is IGHD$^+$ FCER2$^+$ TCL1A$^+$ CD27$^-$.

Caveats:

- **Cycling and IFN describe a cell state, not a lineage.** Their unique genes are the cell-cycle and interferon programs. DZ cells are proliferating too, which is why *SUGCT* and *MME* are shared with `B_Cycling` and not unique to DZ.
- **Some subsets are too small to check across donors.** `B_GC_prePB` has 74 cells, and only one donor has ≥ 20 of them; `B_IFN` has 199 cells, and only two donors qualify. Their markers are consistent only within those few donors.
- ***LILRA4* in pre-GC B is suspicious.** It is a classic plasmacytoid-DC marker, so it may come from contaminating cells or doublets, not genuine B-cell expression.
- **The p-values are optimistic.** The test treats each cell as independent, although cells come from only 13 donors. Rely on the effect-size filters and the donor counts more than on the p-values.

- ***TCL1A* and *CD27* aren't specific to naive or memory cells.** In the dot plot *TCL1A* is highest in GC DZ and cycling cells, and *CD27* is highest in plasma cells. Each is specific only in the naive-vs-memory contrast.
- **The four-genes-per-subset choice was mine.** I picked the dot-plot genes from the unique list where one existed, and otherwise from the best shared or head-to-head genes.

Files are in the output tray:

- `bcell_subset_markers_dotplot.png` — the dot plot above.
- `bcell_subset_unique_markers.csv` — 502 candidate markers in two tiers: "unique", and "enriched (shared)", where the gene is ≥ 1.5× the next subset and detected in ≤ 50% of every other subset. For each gene it gives the % in the subset, the max % in other subsets, the next-highest subset, log2FC vs that subset, adjusted p and the donor count.

In [ ]:

new genes_plot: selected mannually

In [26]:
```
genes_plot = ["CD19", "MS4A1", "CD79A", "CD79B", "PAX5", "CD22", "CD52",
              "CD53", "CDS2", "IGHD", "IGHM", "CD27", "CD28",
              "LEF1", "CCR7", "SELL", "TCF7", "AICDA",
              # extra to obser in cell type signature
              "DERL3", "CDK1", "IFIT3", "IFIT1", "BAIAP2L1", "ARHGAP10",
              "TNFRSF13B", "CLECL1", "GPR183", "IGHD", "FCER2", "TCL1A",
              "MYADM", "IGHG2", "IGHGP", "DERL3"]

genes_plot = np.unique(genes_plot)

print("Genes to plot", len(genes_plot))

genes = [x for x in genes_plot if x in list(adata_vis.var.symbol)]
genes = np.unique(genes)

len(genes), "; ".join(genes)
```

```
Genes to plot 41
```

Out[26]:
```
(40,
 'AICDA; ARHGAP10; BAIAP2L1; CCND2; CCR7; CD19; CD22; CD27; CD28; CD52;
CD53; CD79A; CD79B; CDK1; CDS2; CLECL1; CRIP3; DERL3; FCER2; FRZB; GPR18
3; IFIT1; IFIT3; IGHD; IGHG2; IGHGP; IGHM; LEF1; LILRA4; MS4A1; MYADM; P
AX5; RRM2; SELL; SH3TC1; SLC2A5; TCF7; TCL1A; TNFRSF13B; USP53')
```

Visium spatial plot per gene

- see filtered markers in: result/bcell_subset_unique_markers.csv

In [27]:
```
# clip_for_plot / _values live in libs/cell2location_lib.py
# (warns about unknown features; non-zero percentile cap for very sparse
genes, cols = clip_for_plot(adata_vis, genes, q=99.2, vmin=0)
print(len(genes))
```

```
40
```

In [28]:
```
from IPython.core.display import HTML
display(HTML("<style>.container { width:100% !important; }</style>"))

# pd.reset_option('display.max_colwidth')
# pd.reset_option('display.max_columns')

pd.set_option('display.max_colwidth', 200)
```

In [29]:
```
fname = 'selected_genes_lymphoid_organ.csv'
df_genes = pdreadcsv(fname, root_results, sep=',')
df_genes
```

Out[29]:

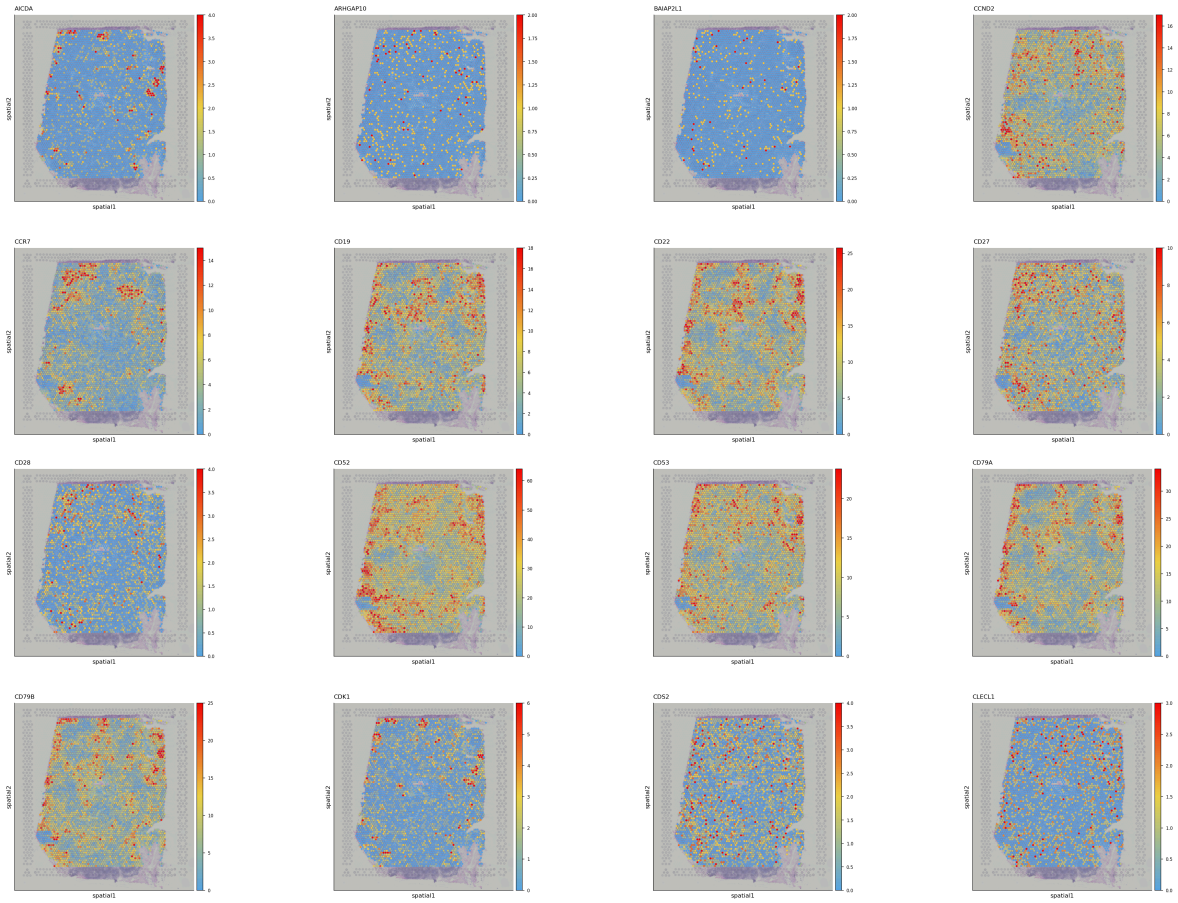
	Symbol	Description	Relation to immune response / secondary lymphoid organs
0	AICDA	Activation-induced cytidine deaminase; deaminates C → U in immunoglobulin loci.	Required for somatic hypermutation and class-switch recombination. Hallmark of germinal-centre (GC) B cells in lymph nodes, tonsil and spleen. Loss causes hyper-IgM syndrome type 2.
1	ARHGAP10	Rho GTPase-activating protein 10 (GRAF2); inactivates RhoA/Cdc42 and regulates actin remodelling.	Indirect role: cytoskeletal control of lymphocyte migration and synapse formation. Appears in B-cell transcriptomic signatures; no defined SLO-specific function.
2	BAIAP2L1	BAI1-associated protein 2-like 1 (IRTKS); I-BAR protein that bends membranes and bundles actin into protrusions.	No established immune function. Recruited by pathogenic E. coli (Tir) for actin pedestals. Appears sporadically in lymphocyte scRNA signatures.
3	CCND2	Cyclin D2; partners CDK4/6 for G1 → S progression.	Main D-type cyclin induced by BCR, CD40 and cytokine signalling in B and T cells. Supports antigen-driven proliferation in follicles; needed for B1 cell development.
4	CCR7	Chemokine receptor for CCL19/CCL21.	Central SLO-homing receptor: drives entry through high endothelial venules (HEV) and placement in the T zone/T–B border. Expressed by naive T and B cells, central-memory T cells and mature migrato...
5	CD19	B-cell co-receptor (with CD21/CD81) that amplifies BCR signalling via PI3K.	Pan-B-cell marker from pro-B to plasmablast (lost on most plasma cells). Lowers the activation threshold to antigen in follicles. CAR-T target.
6	CD22	Siglec-2; inhibitory B-cell co-receptor that binds α2,6-sialic acids and recruits SHP-1 through ITIMs.	Dampens BCR signalling and sets tolerance. High on mature follicular B cells. Drug target (inotuzumab, CAR-T).
7	CD27	TNFRSF7; costimulatory receptor for CD70.	Marks human memory B cells (very high on plasmablasts/plasma cells) and naive/early-memory T cells. Drives GC B-cell and plasma-cell differentiation and T-cell survival.
8	CD28	Main T-cell costimulatory receptor (signal 2) for CD80/CD86.	Required for full naive T-cell priming in the T zone and for Tfh development, and therefore for GC formation. Counter-regulated by CTLA-4. Also on plasma cells.
9	CD52	CAMPATH-1 antigen; small GPI-anchored glycoprotein on lymphocytes and monocytes.	Highly expressed on mature lymphocytes. Soluble CD52 binds Siglec-10 and suppresses T-cell activation. Target of alemtuzumab, which depletes lymphocytes.
10	CD53	Leukocyte-restricted tetraspanin (TSPAN25) that organises membrane microdomains.	Recruits PKCβ to support BCR signalling and B-cell activation. Stabilises L-selectin, so it is needed for efficient lymphocyte homing to lymph nodes.
11	CD79A	Igα; signalling chain of the BCR complex (ITAM-bearing).	Essential for BCR surface expression and signalling at all B-cell stages. Pan-B/plasma-cell marker.
12	CD79B	Igβ; partner chain of CD79A in the BCR.	Needed for BCR signalling. Mutated in ABC-DLBCL (chronic active BCR signalling). Target of polatuzumab vedotin.

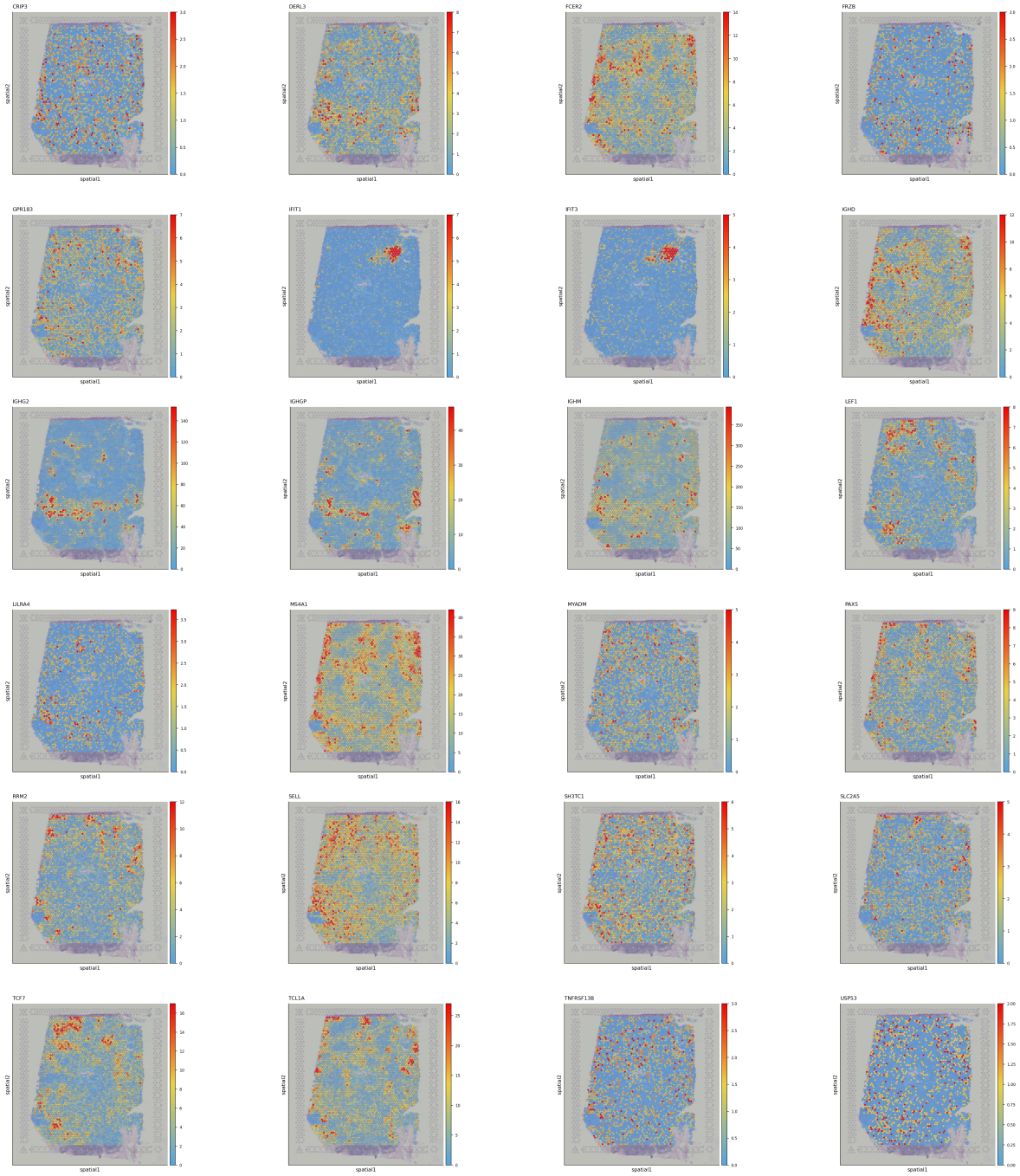
	Symbol	Description	Relation to immune response / secondary lymphoid organs
13	CDK1	Cyclin-dependent kinase 1; drives G2→M transition.	Proliferation marker. High in GC dark-zone centroblasts and in clonally expanding T/B cells.
14	CDS2	CDP-diacylglycerol synthase 2; key step in phosphatidylinositol resynthesis.	Indirect: replenishes PI/PIP2 used by PLCγ downstream of antigen receptors. No specific SLO role described.
15	CLECL1	C-type lectin-like 1 (DCAL-1); type II transmembrane lectin-like protein.	Expressed on B cells and dendritic cells. Acts as a T-cell costimulatory ligand that boosts IL-4 production. Locus linked to autoimmune disease (e.g. MS, T1D) in GWAS.
16	CRIP3	Cysteine-rich protein 3 (thymus LIM protein, TLP); LIM-domain zinc-finger protein.	Expressed in thymus and lymphocytes. Proposed to regulate T-cell proliferation. Poorly characterised in SLO.
17	DERL3	Derlin-3; ER-associated degradation (ERAD) component.	Strongly induced by XBP1/UPR in antibody-secreting cells. Plasmablast/plasma-cell marker (medullary cords, GC-derived ASC).
18	FCER2	CD23; low-affinity IgE receptor (C-type lectin).	Marks naive/follicular and activated B cells; also on FDCs. Regulates IgE synthesis; soluble CD23 binds CD21. Marker of the follicular mantle and of CLL.
19	FRZB	Frizzled-related protein (sFRP3); secreted Wnt antagonist.	Indirect: modulates Wnt/β-catenin signalling, which acts through TCF7/LEF1 in lymphocytes. Found mainly in stromal/mesenchymal cells; no defined SLO role.
20	GPR183	EBI2; receptor for oxysterol 7α,25-dihydroxycholesterol.	Positions activated B cells at the outer/interfollicular follicle and is downregulated for GC entry. Guides DCs and Tfh precursors (splenic bridging channels). Key regulator of SLO microanatomy.
21	IFIT1	IFN-induced protein with tetratricopeptide repeats 1; binds cap-0/5'-ppp viral RNA and blocks its translation.	Classic interferon-stimulated gene (ISG); marks type I IFN-responding cells, e.g. during viral infection or near IFN-producing pDCs.
22	IFIT3	IFIT family member that forms complexes with IFIT1/IFIT2 and links to MAVS–TBK1 signalling.	ISG; amplifies antiviral signalling. Part of the IFN-response signature in lymphoid tissue.
23	IGHD	Immunoglobulin heavy constant δ (IgD).	Co-expressed with IgM on mature naive B cells by alternative splicing. Naive/follicular-mantle B marker, lost after GC entry. IgD class switching occurs in tonsil/upper airway.
24	IGHG2	Immunoglobulin heavy constant γ2 (IgG2).	Class-switched isotype dominating responses to bacterial capsular polysaccharides (often T-independent, linked to the splenic marginal zone). Weak FcγR/complement activation.
25	IGHGP	IgG constant-region pseudogene in the IGH locus (non-functional).	Transcribed in class-switched B and plasma cells (germline/read-through transcription). Mostly a marker of

	Symbol	Description	Relation to immune response / secondary lymphoid organs
			switched cells and of multi-mapping IGHG reads in scRNA.
26	IGHM	Immunoglobulin heavy constant μ (IgM).	First isotype expressed: part of the naive B-cell BCR and secreted as pentameric IgM early in responses. High in naive, marginal-zone and B1 cells.
27	LEF1	Lymphoid enhancer-binding factor 1; HMG-box transcription factor and Wnt/β-catenin effector.	Works with TCF7 in T-cell development and naive T-cell identity. Promotes Tfh differentiation (via BCL6). Expressed in pro-B cells; aberrantly expressed in CLL.
28	LILRA4	ILT7; activating LILR receptor coupled to FcRγ, specific to plasmacytoid DCs.	pDC marker (pDCs sit in the T zone/HEV area). Binding to BST2 restrains IFN-α production.
29	MS4A1	CD20; tetraspan membrane protein that supports BCR-associated Ca^{2+} flux.	Pan-mature-B-cell marker (absent on plasma cells) labelling follicles. Target of rituximab and related antibodies.
30	MYADM	Myeloid-associated differentiation marker; MARVEL-domain membrane protein organising ordered membrane domains.	Regulates RhoA, cell spreading, migration and endothelial permeability. Broadly expressed in leukocytes; no specific SLO role.
31	PAX5	Paired box 5 (BSAP); master transcription factor for B-cell identity.	Activates B-cell genes (CD19, CD79A, BLNK) and represses other lineages and the plasma-cell programme. Repressed by BLIMP1 during plasma-cell differentiation.
32	RRM2	Ribonucleotide reductase regulatory subunit M2; makes dNTPs in S phase.	S-phase/proliferation marker. High in GC dark-zone B cells and expanding lymphocytes. Hydroxyurea target.
33	SELL	L-selectin (CD62L); lectin that mediates tethering/rolling on PNAd at HEVs.	Required for naive/central-memory lymphocyte entry into lymph nodes. Shed by ADAM17 after activation.
34	SH3TC1	SH3 domain and tetratricopeptide repeats 1; poorly characterised adaptor-like protein.	Enriched in B cells in expression atlases. No defined mechanistic role in immunity.
35	SLC2A5	GLUT5; facilitative fructose transporter.	Indirect: fructose uptake can reprogram monocytes/macrophages towards inflammatory responses. Highly expressed in microglia; no specific SLO role.
36	TCF7	TCF-1; HMG-box transcription factor and Wnt effector.	Essential for T-cell commitment and for naive/memory and stem-like (progenitor-exhausted) CD8 T cells. Drives Tfh differentiation and GC help. Also in ILCs.
37	TCL1A	T-cell leukemia/lymphoma 1A; co-activator of AKT.	High in naive B cells and pDCs; downregulated in GC and memory B cells. Oncogene in T-PLL (inv14) and CLL.
38	TNFRSF13B	TACI; receptor for BAFF and APRIL.	Drives T-independent antibody responses, class switching and plasma-cell differentiation. Limits B-cell expansion. On marginal-zone/memory B and plasma cells. Mutations linked to CVID.

	Symbol	Description	Relation to immune response / secondary lymphoid organs
39	USP53	Ubiquitin-specific peptidase 53; deubiquitinase linked to tight junctions.	No established immune role. Mutations cause cholestasis and hearing loss. Appears in some lymphocyte signatures only.

In [30]:
```
%matplotlib inline

nframe=8

colors = ("#57a4e2", "#ebd246", "#f30707")
cmap_byr = LinearSegmentedColormap.from_list("blue_yellow_red", list(colo

for i in range(0, len(genes), nframe):
    # print(i, i+nframe)

    sq.pl.spatial_scatter(
        adata_vis,
        color=cols[i:(i+nframe)],
        title=list(genes[i:(i+nframe)]),            # show original names
        img_res_key="hires",
        img_alpha=0.6,
        size=1.3,
        cmap=cmap_byr,
        ncols=4,
        library_id="V1_Human_Lymph_Node",
    )
```





find mitochondria-encoded (MT) genes

In [31]:
```
adata_vis[:, adata_vis.var['MT_gene'].values]
```

Out[31]:
```
View of AnnData object with n_obs × n_vars = 4035 × 0
    obs: 'in_tissue', 'array_row', 'array_col', 'sample', '_indices', '_
scvi_batch', '_scvi_labels', 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_pr
ePB', 'B_IFN', 'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC',
'DC_CCR7+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macro
phages_M1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4
+', 'T_CD4+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_C
D8+_cytotoxic', 'T_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', 'l
eiden', 'region_cluster', 'mean_nUMI_factorsfact_0', 'mean_nUMI_factorsf
act_1', 'mean_nUMI_factorsfact_2', 'mean_nUMI_factorsfact_3', 'mean_nUMI
_factorsfact_4', 'mean_nUMI_factorsfact_5', 'mean_nUMI_factorsfact_6',
'mean_nUMI_factorsfact_7', 'mean_nUMI_factorsfact_8', 'mean_nUMI_factors
fact_9', 'mean_nUMI_factorsfact_10', 'mean_nUMI_factorsfact_11', 'region
_name', '_plot_AICDA', '_plot_ARHGAP10', '_plot_BAIAP2L1', '_plot_CCND
2', '_plot_CCR7', '_plot_CD19', '_plot_CD22', '_plot_CD27', '_plot_CD2
8', '_plot_CD52', '_plot_CD53', '_plot_CD79A', '_plot_CD79B', '_plot_CDK
1', '_plot_CDS2', '_plot_CLECL1', '_plot_CRIP3', '_plot_DERL3', '_plot_F
CER2', '_plot_FRZB', '_plot_GPR183', '_plot_IFIT1', '_plot_IFIT3', '_plo
t_IGHD', '_plot_IGHG2', '_plot_IGHGP', '_plot_IGHM', '_plot_LEF1', '_plo
t_LILRA4', '_plot_MS4A1', '_plot_MYADM', '_plot_PAX5', '_plot_RRM2', '_p
lot_SELL', '_plot_SH3TC1', '_plot_SLC2A5', '_plot_TCF7', '_plot_TCL1A',
'_plot_TNFRSF13B', '_plot_USP53'
    var: 'feature_types', 'genome', 'symbol', 'MT_gene'
    uns: '_scvi_manager_uuid', '_scvi_uuid', 'leiden', 'mod', 'mod_coloc
_n_fact11', 'mod_coloc_n_fact12', 'neighbors', 'region_cluster_colors',
'sample_colors', 'spatial', 'umap'
    obsm: 'MT', 'X_umap', 'means_cell_abundance_w_sf', 'q0001_cell_abund
ance_w_sf', 'q05_cell_abundance_w_sf', 'q50_cell_abundance_w_sf', 'q95_c
ell_abundance_w_sf', 'spatial', 'stds_cell_abundance_w_sf'
    obsp: 'connectivities', 'distances'
    layers: 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IFN', 'B
_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC', 'DC_CCR7+', 'DC_
cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macrophages_M1', 'Mac
rophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4+', 'T_CD4+_TfH',
'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_CD8+_cytotoxic', 'T
_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', None (.X)
```

In [32]:
```
adata_vis.var[adata_vis.var['symbol'].str.startswith('MT-')]
```

Out[32]:

	feature_types	genome	symbol	MT_gene
gene_ids				

In [33]:
```
adata_vis.obsm['MT']
```

Out[33]:
```
array([[16., 13., 31., ...,  1.,  0., 14.],
       [12.,  9., 30., ...,  2.,  0., 23.],
       [38., 15., 59., ...,  2.,  1., 32.],
       ...,
       [11.,  6., 32., ...,  3.,  0., 13.],
       [ 8.,  8., 23., ...,  0.,  0., 13.],
       [37., 16., 51., ...,  1.,  0., 23.]],
      shape=(4035, 13), dtype=float32)
```

In [ ]:
```

```

edited adata_ref

Published scRNA-seq datasets of lymph nodes have typically lacked an adequate representation of germinal centre-associated immune cell populations due to age of patient donors. We, therefore, include scRNA-seq datasets spanning lymph nodes, spleen and tonsils in our single-cell reference to ensure that we captured the full diversity of immune cell states likely to exist in the spatial transcriptomic dataset.

Here we download this dataset, import into anndata and change variable names to ENSEMBL gene identifiers.

In [34]:
```
adata_ref
```

Out[34]:
```
AnnData object with n_obs × n_vars = 73260 × 10237
    obs: 'Age', 'BCELL_CLONE', 'BCELL_CLONE_SIZE', 'Donor', 'ID', 'IGH_MU_FREQ', 'ISOTYPE', 'LibraryID', 'Method', 'Population', 'PrelimCellType', 'Sample', 'Sex', 'Study', 'Tissue', 'barcode', 'batch', 'doublet_score', 'index', 'predicted_doublet', 'percent_mito', 'n_counts', 'n_genes', 'S_score', 'G2M_score', 'phase', 'VDJsum', 'cell_cycle_diff', 'PrelimCellType_new', 'leiden', 'leiden_1', 'leiden_2', 'leiden_3', 'leiden_4', 'CellType', 'CellType2', 'Subset', 'Subset_Broad', 'Subset_all', 'new_celltype', 'Subset_int', 'Subset_print'
    var: 'GeneID-2', 'GeneName-2', 'feature_types', 'feature_types-0', 'feature_types-1', 'gene_ids-1', 'gene_ids-4861STDY7135913-0', 'gene_ids-4861STDY7135914-0', 'gene_ids-4861STDY7208412-0', 'gene_ids-4861STDY7208413-0', 'gene_ids-Human_colon_16S7255677-0', 'gene_ids-Human_colon_16S7255678-0', 'gene_ids-Human_colon_16S8000484-0', 'gene_ids-Pan_T7935494-0', 'genome-1', 'n_cells', 'nonz_mean', 'mean_cov_effect_Subset_B_Cycling', 'mean_cov_effect_Subset_B_GC_DZ', 'mean_cov_effect_Subset_B_GC_LZ', 'mean_cov_effect_Subset_B_GC_prePB', 'mean_cov_effect_Subset_B_IFN', 'mean_cov_effect_Subset_B_activated', 'mean_cov_effect_Subset_B_mem', 'mean_cov_effect_Subset_B_naive', 'mean_cov_effect_Subset_B_plasma', 'mean_cov_effect_Subset_B_preGC', 'mean_cov_effect_Subset_DC_CCR7+', 'mean_cov_effect_Subset_DC_cDC1', 'mean_cov_effect_Subset_DC_cDC2', 'mean_cov_effect_Subset_DC_pDC', 'mean_cov_effect_Subset_Endo', 'mean_cov_effect_Subset_FDC', 'mean_cov_effect_Subset_ILC', 'mean_cov_effect_Subset_Macrophages_M1', 'mean_cov_effect_Subset_Macrophages_M2', 'mean_cov_effect_Subset_Mast', 'mean_cov_effect_Subset_Monocytes', 'mean_cov_effect_Subset_NK', 'mean_cov_effect_Subset_NKT', 'mean_cov_effect_Subset_T_CD4+', 'mean_cov_effect_Subset_T_CD4+_TfH', 'mean_cov_effect_Subset_T_CD4+_TfH_GC', 'mean_cov_effect_Subset_T_CD4+_naive', 'mean_cov_effect_Subset_T_CD8+_CD161+', 'mean_cov_effect_Subset_T_CD8+_cytotoxic', 'mean_cov_effect_Subset_T_CD8+_naive', 'mean_cov_effect_Subset_T_TIM3+', 'mean_cov_effect_Subset_T_TfR', 'mean_cov_effect_Subset_T_Treg', 'mean_cov_effect_Subset_VSMC', 'mean_sample_effectSample_4861STDY7135913', 'mean_sample_effectSample_4861STDY7135914', 'mean_sample_effectSample_4861STDY7208412', 'mean_sample_effectSample_4861STDY7208413', 'mean_sample_effectSample_4861STDY7462253', 'mean_sample_effectSample_4861STDY7462254', 'mean_sample_effectSample_4861STDY7462255', 'mean_sample_effectSample_4861STDY7462256', 'mean_sample_effectSample_4861STDY7528597', 'mean_sample_effectSample_4861STDY7528598', 'mean_sample_effectSample_4861STDY7528599', 'mean_sample_effectSample_4861STDY7528600', 'mean_sample_effectSample_BCP002_Total', 'mean_sample_effectSample_BCP003_Total', 'mean_sample_effectSample_BCP004_Total', 'mean_sample_effectSample_BCP005_Total', 'mean_sample_effectSample_BCP006_Total', 'mean_sample_effectSample_BCP008_Total', 'mean_sample_effectSample_BCP009_Total', 'mean_sample_effectSample_Human_colon_16S7255677', 'mean_sample_effectSample_Human_colon_16S7255678', 'mean_sample_effectSample_Human_colon_16S8000484', 'mean_sample_effectSample_Pan_T7935494'
    uns: 'Age_colors', 'Donor_colors', 'LibraryID_colors', 'Method_colors', 'Study_colors', 'Subset_Broad_colors', 'Subset_colors', 'Tissue_colors', 'leiden', 'neighbors', 'pca', 'phase_colors', 'regression_mod', 'umap'
    obsm: 'X_pca', 'X_umap'
    obsp: 'connectivities', 'distances'
    layers: None (.X)
```

In [35]:
```
print(adata_ref.var.shape)
list(adata_ref.var.columns)[:10]
```

```
(10237, 74)
```

Out[35]:
```
['GeneID-2',
 'GeneName-2',
 'feature_types',
 'feature_types-0',
 'feature_types-1',
 'gene_ids-1',
 'gene_ids-4861STDY7135913-0',
 'gene_ids-4861STDY7135914-0',
 'gene_ids-4861STDY7208412-0',
 'gene_ids-4861STDY7208413-0']
```

In [36]:
```
adata_ref.var.head(3)
```

Out[36]:

	GeneID-2	GeneName-2	feature_types	feature_types-0	feature_types-1
NOC2L	ENSG00000188976	16551	nan	Gene Expression	Gene Expression
HES4	ENSG00000188290	11070	nan	Gene Expression	Gene Expression
ISG15	ENSG00000187608	12375	nan	Gene Expression	Gene Expression

3 rows × 74 columns

In [37]:
```
if 'symbol' not in adata_ref.var.columns:
    adata_ref.var['symbol']= adata_ref.var.index
    # rename 'GeneID-2' as necessary for your data
    adata_ref.var.set_index('GeneID-2', drop=True, inplace=True)

# delete unnecessary raw slot (to be removed in a future version of the t
del adata_ref.raw

adata_ref.var.head(5).T.head(6)
```

Out[37]:

GeneID-2	ENSG00000188976	ENSG00000188290	ENSG00000187608	ENSG000(
GeneName-2	16551	11070	12375	
feature_types	nan	nan	nan	
feature_types-0	Gene Expression	Gene Expression	Gene Expression	Gene E
feature_types-1	Gene Expression	Gene Expression	Gene Expression	Gene E
gene_ids-1	ENSG00000188976	ENSG00000188290	ENSG00000187608	ENSG000(
gene_ids-4861STDY7135913-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	ENSG000(

In [ ]:
```
```

lymphoid_organ_sc.h5ad - reference

`lymphoid_organ_sc.h5ad` is the single-cell reference for the cell2location paper's **lymph node / tonsil analysis**.

It holds **73,260 cells × 10,237 genes**, 1.66 GB on disk, and contains both raw counts and a fitted reference model. I opened it read-only in backed mode, so the file was not changed.

Expression matrix (`X`)

- **Raw UMI counts.** The matrix is sparse (CSR, float32) and every value is a whole number. In the first 2,000 cells the counts run from 1 to 1,276, with about 1,080 detected genes per cell.
- **Genes are already filtered.**
  - Gene symbols are the index and Ensembl IDs are in `var['GeneID-2']`.
  - The `n_counts` and `n_genes` columns were computed on the full gene set before filtering, so they are slightly higher than the row sums of `X` (e.g. 3,148 vs 3,122).
- **Cell QC:**
  - median 3,483 UMIs and 1,657 genes per cell;
  - minimum 581 UMIs and 698 genes.
  - Mitochondrial fraction (`percent_mito`) is 0–15%, median 4%.
  - Every cell has `predicted_doublet = False`, so doublets were already removed.

Studies and tissues (`obs`)

Study	Tissue	3′ (3GEX)	5′ (5GEX)
King_2020	Tonsil	2,002	27,703
James_2020	Mesenteric LN	10,250	4,184
Park_2020	Spleen / iLN / mLN / tLN	15,506	13,615

- **23 samples from 13 donors.** Donor A16 alone contributes 29,121 cells.
- **Ages** run from 0–4 years to 65–70 years.
- **Sex:** 48,545 female and 24,715 male cells.
- **Extra columns:** B-cell receptor fields (`BCELL_CLONE` , `ISOTYPE` , `IGH_MU_FREQ` , `VDJsum`), cell-cycle scores and phase, five Leiden clusterings, and several generations of annotation columns.

Cell-type annotations

- **`Subset` (34 types)** is the label set the regression model was trained on. The largest groups are B_mem (13,476), B_naive (8,924), T_CD4+_naive (6,012), B_Cycling (4,765) and T_CD4+_TfH (4,690). It also covers germinal-centre B cells (DZ, LZ, prePB, preGC), T-cell types (TfH, Tfr, Treg, CD8 types), NK/NKT/ILC, myeloid cells (DCs, macrophages, monocytes, mast cells) and stromal cells (Endo, FDC, VSMC). Rare types include ILC/Mast (18), VSMC (40) and DC_CCR7+ (42).

- `Subset_Broad` **(38 types)** is finer, despite its name. For example, it splits the endothelium into ACKR1+, CXCL12+ and lymphatic cells.
- **Likely label swap:** `Subset` has ILC = 617 and Mast = 18, while `Subset_Broad` has Mast = 617 and ILC = 18. I only compared the counts and did not cross-tabulate cell by cell. Check marker genes (e.g. TPSAB1/KIT for mast cells) before relying on either label for these two types.

Fitted reference model

This file already contains the trained cell2location reference model, so you don't need to retrain it.

- `uns['regression_mod']` is a `RegressionNBV4Torch` model with 57 factors: 34 cell types plus 23 sample effects. Its posterior means are:
  - `gene_factors` (57 × 10,237)
  - `sample_scaling` (23 × 1)
  - `tech_scaling` (2 × 10,237, one row per chemistry, 3′ and 5′)
  - `gene_E` (1 × 10,237)
- **In `var`:** 34 `mean_cov_effect_Subset_*` columns hold each cell type's expression signature. These are the columns to pass to cell2location as the reference. There are also 23 `mean_sample_effectSample_*` columns, plus `n_cells` and `nonz_mean`.

Embeddings and metadata

- **Embeddings:** `obsm['X_pca']` (100 components), `obsm['X_umap']`, and a neighbour graph built with 69 neighbours (`obsp` connectivities and distances).
- **Colour palettes** are stored in `uns` for the main categories.
- **Unusable columns:** `GeneName-2` holds numbers instead of gene names, and the `feature_types` / `gene_ids-*` columns are mostly NaN. Use the var index for gene names instead.

To extract the signatures for spatial mapping:

```
sig = adata.var[[c for c in adata.var if
c.startswith('mean_cov_effect_Subset_')]]
sig.columns = sig.columns.str.replace('mean_cov_effect_Subset_',
'')
```

This was a description only, so I didn't save any files.

In [38]:
```
obs=adata_ref.obs

for c in ['Study','Tissue','Method','Donor','Sample','Age','Sex','Subset_
    vc=obs[c].value_counts(dropna=False)
    print(f"\n## {c} ({vc.size} levels)")
    print(vc.head(40).to_string())

print()
print(obs[['n_counts','n_genes','percent_mito','doublet_score']].describe
```

```
## Study (3 levels)
Study
King_2020     29705
Park_2020     29121
James_2020    14434

## Tissue (5 levels)
Tissue
Tonsil    29705
mLN       20480
Spleen     8174
iLN        7455
tLN        7446

## Method (2 levels)
Method
5GEX    45502
3GEX    27758

## Donor (13 levels)
Donor
A16     29121
BCP5    13323
290b     6067
BCP6     4531
BCP9     3867
BCP8     2914
390c     2876
298c     2358
BCP3     2083
BCP2     2002
302c     1825
417c     1308
BCP4      985

## Sample (23 levels)
Sample
BCP005_Total              13323
BCP006_Total               4531
4861STDY7462253            4286
4861STDY7462255            4283
4861STDY7528597            3888
BCP009_Total               3867
4861STDY7462256            3852
4861STDY7528600            3594
4861STDY7528599            3172
4861STDY7135914            3126
4861STDY7462254            3085
4861STDY7528598            2961
4861STDY7135913            2941
BCP008_Total               2914
Pan_T7935494               2876
BCP003_Total               2083
BCP002_Total               2002
Human_colon_16S8000484     1308
4861STDY7208413            1228
4861STDY7208412            1130
Human_colon_16S7255678     1125
BCP004_Total                985
Human_colon_16S7255677      700
```

```
## Age (7 levels)
Age
20-25y    29121
0-4y      16237
5-9y      13468
60-64y     6067
65-70y     4184
55-60y     2358
40-44y     1825

## Sex (2 levels)
Sex
female    48545
male      24715

## Subset_Broad (38 levels)
Subset_Broad
B_mem               11880
B_naive              9059
T_CD4+_naive         6012
T_CD4+_TfH           4690
B_Cycling            4603
B_activated          3979
T_CD8+_cytotoxic     3890
T_CD4+_TfH_GC        3653
T_CD4+               3059
T_Treg               2958
B_GC_DZ              2500
T_CD8+_CD161+        2294
T_CD8+_naive         2253
B_GC_IZ              2058
B_mem_FCRL4+         1660
B_GC_LZ              1240
B_plasma             1094
T_Tfr                1065
NK                    940
NKT                   896
Mast                  617
NK CX3CR1+            432
T_TIM3+               357
Endo_ACKR1+           314
Monocytes             306
DC_pDC                226
Endo_CXCL12+          180
DC_cDC2               173
T_Cycling             162
Endo_Lymphatic        128
Macrophages_M1        121
Macrophages_M2        110
DC_cDC1               101
FDC                    76
B_GC_prePB             74
DC_CCR7+               42
VSMC                   40
ILC                    18

## Subset (34 levels)
Subset
B_mem               13476
```

```
B_naive              8924
T_CD4+_naive         6012
B_Cycling            4765
T_CD4+_TfH           4690
T_CD8+_cytotoxic     3890
T_CD4+_TfH_GC        3653
B_activated          3575
B_GC_LZ              3298
T_CD4+               3059
T_Treg               2958
B_GC_DZ              2500
T_CD8+_CD161+        2294
T_CD8+_naive         2253
NK                   1372
B_plasma             1094
T_TfR                1065
NKT                   896
Endo                  622
ILC                   617
B_preGC               404
T_TIM3+               357
Monocytes             306
DC_pDC                226
B_IFN                 199
DC_cDC2               173
Macrophages_M1        121
Macrophages_M2        110
DC_cDC1               101
FDC                    76
B_GC_prePB             74
DC_CCR7+               42
VSMC                   40
Mast                   18

## phase (3 levels)
phase
G1     53232
S      16761
G2M     3267

## predicted_doublet (1 levels)
predicted_doublet
False    73260

       n_counts   n_genes  percent_mito  doublet_score
count  73260.00  73260.00      73260.00       73260.00
mean    4105.59   2029.96          0.05           0.09
std     2620.86    936.30          0.03           0.08
min      581.00    698.00          0.00           0.00
25%     2546.00   1330.00          0.03           0.03
50%     3483.00   1657.00          0.04           0.07
75%     4817.00   2499.00          0.06           0.13
max    19999.00   7932.00          0.15           0.59
```

In [39]:
```
# X values sample
sub=adata_ref.X[:2000]
d=sub.data

print("\n")
```

```
print("X sample: min",d.min(),"max",d.max(),"integer?",np.allclose(d,np.r
print("rowsum vs n_counts", np.asarray(sub.sum(1)).ravel()[:5], obs.n_cou
print(adata_ref.var.index[:5].tolist())
print(adata_ref.uns['regression_mod'].keys() if hasattr(adata_ref.uns['re
print({k:(adata_ref.uns[k] if k!='leiden' else adata_ref.uns[k]) for k in

cols=['GeneName-2','n_cells','nonz_mean']
adata_ref.var[cols].head(3)
```

```
X sample: min 1.0 max 1276.0 integer? True nnz/cell 1080.674
rowsum vs n_counts [3122. 3231. 3741. 2616. 2854.] [3148. 3266. 3782. 265
4. 2898.]
['ENSG00000188976', 'ENSG00000188290', 'ENSG00000187608', 'ENSG0000018689
1', 'ENSG00000186827']
dict_keys(['fact_names', 'mod_name', 'obs_names', 'post_sample_means', 'va
r_names'])
{'neighbors': {'params': {'method': 'umap', 'n_neighbors': 69}}}
```

Out[39]:

GeneID-2	GeneName-2	n_cells	nonz_mean
ENSG00000188976	16551	11327	1.119714
ENSG00000188290	11070	2553	1.784567
ENSG00000187608	12375	19155	1.651318

In [ ]:

Split by Age-group and Tissues

First I'll check how tissue and age overlap in the scRNA reference, and look in the squidpy project for the `genes` list and the `adata_vis` object.

Note: tissue and age are confounded. Tonsil is the only babies/young tissue, and spleen, iLN and tLN are all adult. Let me find the notebook that defines `genes` and `adata_vis`.

This Visium slide has no tissue/age labels, so I'll instead build the B-cell gene panel on the scRNA reference, coloring UMAPs by each gene split by tissue × age group with the same 99.2-percentile clipping. First, I'll load the figure-style skill.

Since CD20 isn't a valid gene symbol (the gene is MS4A1), I'll plot MS4A1 labelled as "MS4A1 (CD20)" instead. Next I'll build the grid with genes in rows and tissue × age groups in columns, using a shared colour scale per gene.

I found two issues to fix before saving: colliding column headers, and a blank CD14 column caused by its extreme sparsity flattening the 99.2-percentile clip to 0. I'll fix the headers and adjust sparse genes like CD14 to use the 99.2th percentile of non-zero values instead.

Yes, but only on the scRNA reference, not on the Visium slide. `V1_Human_Lymph_Node` is a single lymph node section from one donor, so its spots carry no tissue or age labels. Tissue and age exist only in

`lymphoid_organ_sc.h5ad`. So I drew the same gene panel and the same 99.2-percentile clipping on the reference UMAP, with one column per tissue × age group.

B-cell gene panel on the scRNA reference UMAP, split by tissue and age group

Age groups. Your cut-offs map onto the reference's age bins like this:

Group	Age bins	Cells
babies	0–4y	16,237
young	5–9y	13,468
adult	20–25y	29,121
old adult	40–44y	1,825
elder	55–60y, 60–64y, 65–70y	12,609

Tissue and age are almost entirely confounded. Only 8 of the 25 tissue × age combinations contain cells:

- Tonsil exists only as babies and young.
- Spleen, iLN and tLN are adult only, and all come from the single donor A16.
- Only mLN spans three age groups: adult, old adult (one donor, 1,825 cells) and elder (four donors).

So a difference between columns is just as likely to come from tissue, study (King, Park or James) or donor as from age.

How the plot differs from your Visium version:

- **CD20 is replaced by MS4A1.** `CD20` is not a gene symbol, so your `clip_for_plot` drops it without a warning, and it is probably missing from your Visium plot too. I plotted `MS4A1`, labelled "MS4A1 (CD20)".
- **CD14 needed a different cap.** More than 99.2% of cells have zero *CD14*, so the 99.2-percentile cap was 0 and the row came out blank. For that gene I clipped at the 99.2th percentile of the non-zero values instead. The same problem can hit any sparse gene in your Visium helper.
- **Expression is normalised.** The reference holds raw counts from mixed 3′ and 5′ chemistry, so I coloured by log1p(counts per 10k), computed per cell. Each gene has one colour scale shared across all columns, so columns can be compared directly.
- **The colour map is sequential.** I used a light-grey-to-red ramp instead of `RdBu_r`, because expression has no meaningful midpoint for a diverging map to centre on.

If you want the split on the slide itself, one option is to build separate cell2location reference signatures per group and map each onto the Visium spots. Given the confounding, though, the result would mostly reflect tissue and study, not age.

Files are in the output tray:

- `bcell_genes_umap_by_tissue_age.png` — the figure above.

- `bcell_genes_by_tissue_age.csv` — per gene and group: cell count, % of cells expressing, and mean log1p(CP10k).

Tissue types

In [40]:
```
np.unique(adata_ref.obs["Tissue"])
```

Out[40]:
```
array(['Spleen', 'Tonsil', 'iLN', 'mLN', 'tLN'], dtype=object)
```

mLN only: naive / memory / GC markers by age group (adult, old adult, elder)

- genes are looked up by symbol

In [41]:
```
ml = adata_ref[adata_ref.obs["Tissue"] == "mLN"].copy()
ml.obs["age_group"] = ml.obs["Age"].map(age_group)          # age_group()

sc.pp.normalize_total(ml, target_sum=1e4)
sc.pp.log1p(ml)

mln_genes, mln_ids = resolve_gene_ids(ml, ["LEF1", "CCR7", "SELL", "TCF7"
Xm = ml[:, mln_ids].X
Xm = Xm.toarray() if sp.issparse(Xm) else np.asarray(Xm)

print(">> found genes", mln_genes, '\n')

df_mln = pd.DataFrame(Xm, columns=mln_genes, index=ml.obs_names)
df_mln["age_group"] = ml.obs["age_group"].values
mln_order = ["adult", "old adult", "elder"]

print(df_mln.groupby("age_group")[mln_genes].mean().reindex(mln_order).ro
print("\n")
print((df_mln[mln_genes] > 0).groupby(df_mln["age_group"]).mean().reindex

del ml, Xm
```

```
>> found genes ['LEF1', 'CCR7', 'SELL', 'TCF7', 'CD27', 'CD28', 'AICDA']

            LEF1   CCR7   SELL   TCF7   CD27   CD28  AICDA
age_group
adult      0.441  0.763  0.963  0.537  0.826  0.282  0.002
old adult  0.184  0.622  0.482  0.509  0.556  0.185  0.002
elder      0.413  0.813  0.816  0.629  0.634  0.253  0.001


            LEF1   CCR7   SELL   TCF7   CD27   CD28  AICDA
age_group
adult      0.259  0.426  0.543  0.334  0.518  0.190  0.001
old adult  0.113  0.376  0.317  0.336  0.369  0.115  0.002
elder      0.254  0.454  0.442  0.371  0.388  0.167  0.001
```

In [42]:
```
print(">>> df_mln is a subset from X")
print(df_mln.shape)
df_mln.head(3)
```

```
>>> df_mln is a subset from X
(20480, 8)
```

Out[42]:

	LEF1	CCR7	SELL	TCF7	CD27	CD28	A
AAACCTGGTAGAGCTG-1-4861STDY7135913	0.000000	0.000000	0.000000	2.002311	1.435816	1.435816	
AAACCTGGTGTCAATC-1-4861STDY7135913	1.409771	1.972696	1.972696	0.000000	0.000000	1.972696	
AAACCTGTCAGAGGTG-1-4861STDY7135913	0.000000	0.000000	0.000000	1.301031	0.000000	0.000000	

In [ ]:

`CD20` is not a gene symbol in the reference (the gene is `MS4A1`), so the panel uses `MS4A1`, labelled "MS4A1 (CD20)".

Grid below: genes in rows, tissue × age groups in columns; colour = log1p(CP10k) clipped at the 99.2th percentile per gene, one scale per gene row (shared across columns).

Caveat: tissue and age are confounded (Tonsil = babies/young only; Spleen/iLN/tLN = adult, one donor; mLN = adult / old adult / elder), so column differences are not pure age effects.

My selected genes - to observe them inside the lymphonode

In [43]:
```
found, ids = resolve_gene_ids(adata_ref, genes_plot)   # works with symbo
print(len(found), 'of', len(genes_plot), 'genes found')
";".join(found)
```

```
41 of 41 genes found
```

Out[43]:
```
'AICDA;ARHGAP10;BAIAP2L1;CCND2;CCR7;CD19;CD22;CD27;CD28;CD52;CD53;CD79A;
CD79B;CDK1;CDS2;CLECL1;CRIP3;DERL3;FCER2;FRZB;GPR183;HIST1H3B;IFIT1;IFIT
3;IGHD;IGHG2;IGHGP;IGHM;LEF1;LILRA4;MS4A1;MYADM;PAX5;RRM2;SELL;SH3TC1;SL
C2A5;TCF7;TCL1A;TNFRSF13B;USP53'
```

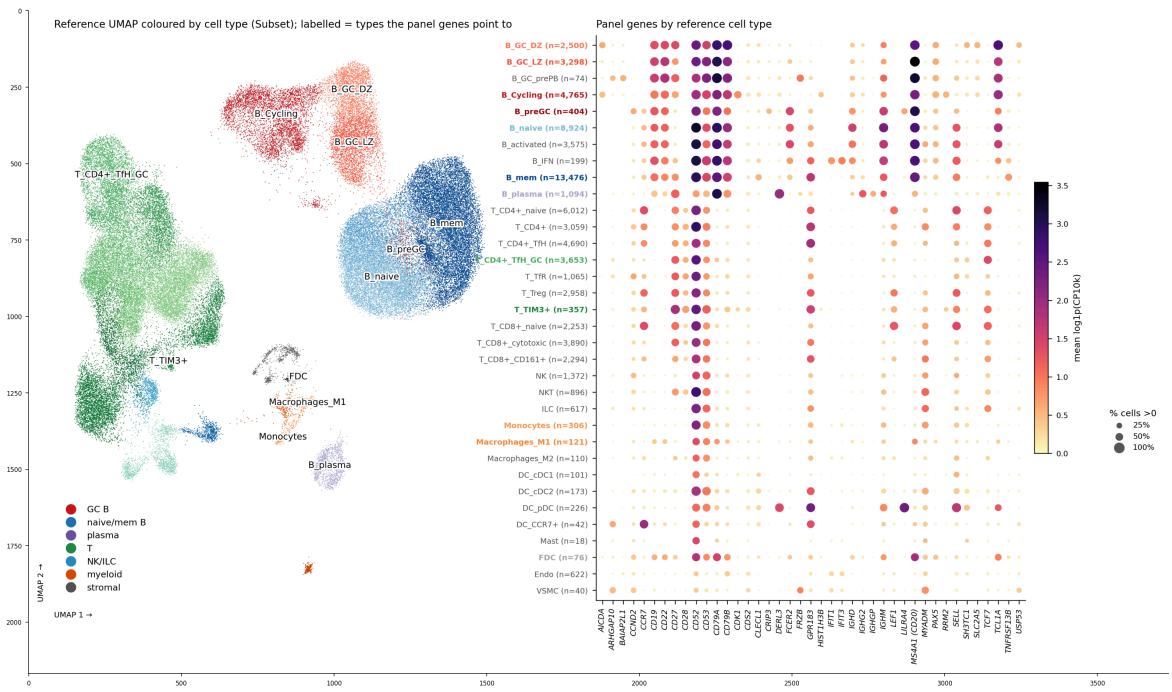
In [ ]:

Below one calculate the UMAP's regions

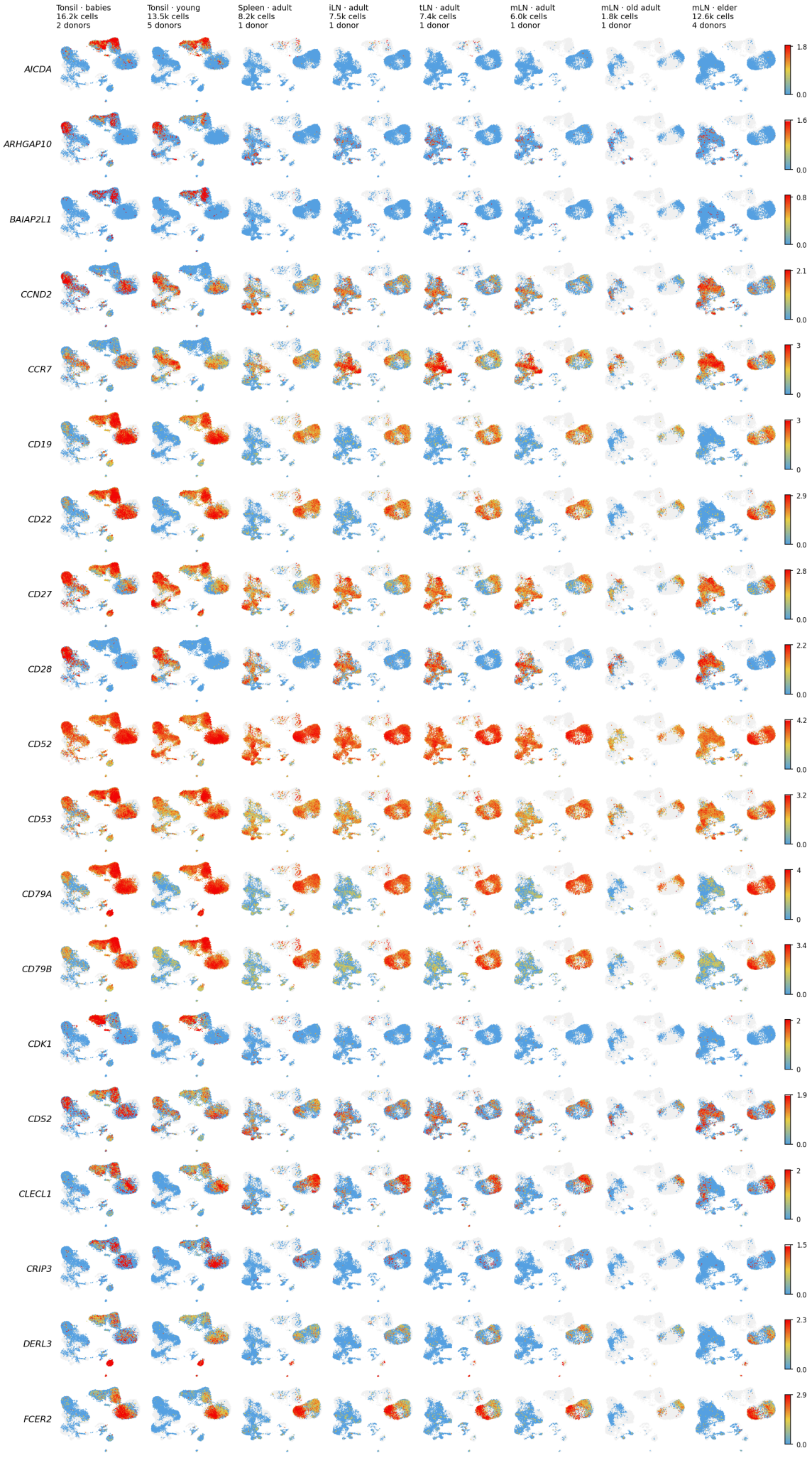
- GC DZ: dark zone (DZ) of a germinal center (GC)
- GC LZ: germinal center (GC) light zone (LZ)
- preGC: Pre-germinal center (preGC) B cells
  - refer to B-cell subsets and precursor states that exist before they enter the germinal center (GC) reaction to undergo somatic hypermutation and class-switch recombination
- FDCs: Follicular dendritic cells
- B-cell cycling
  - refers to the process by which B lymphocytes transition from a resting state into active proliferation, DNA replication, and division during immune responses or development

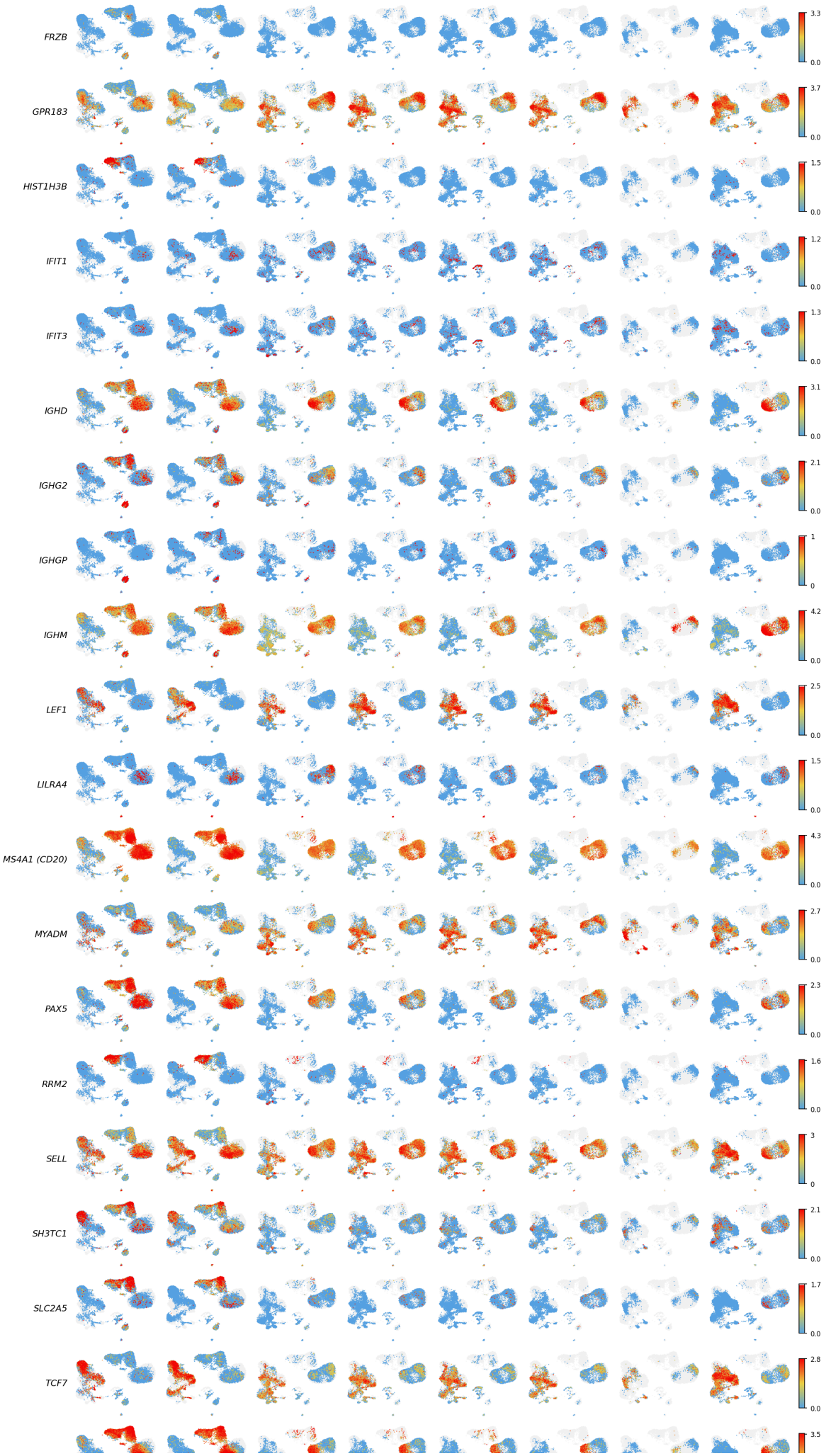
- CD4+ Tfh (T follicular helper) cells are a specialized subset of CD4+ helper T cells
  - that reside primarily in lymphoid tissues and orchestrate germinal center (GC) reactions.
- TIM3+ refers to cells that express TIM-3 (T-cell immunoglobulin and mucin-domain containing-3)
  - an immune checkpoint protein found on various immune and tumor cells
  - Receptor Type: An inhibitory immune checkpoint receptor that acts as a negative regulator of the immune system.
  - Cell Expression: Found on exhausted CD8+ T cells, CD4+ T cells, regulatory T cells (Tregs), natural killer (NK) cells, M2 macrophages, and certain tumor cells.
  - Ligands: Interacts with molecules like galectin-9 (GAL-9), CEACAM1, HMGB1, and phosphatidylserine (PtdSer).

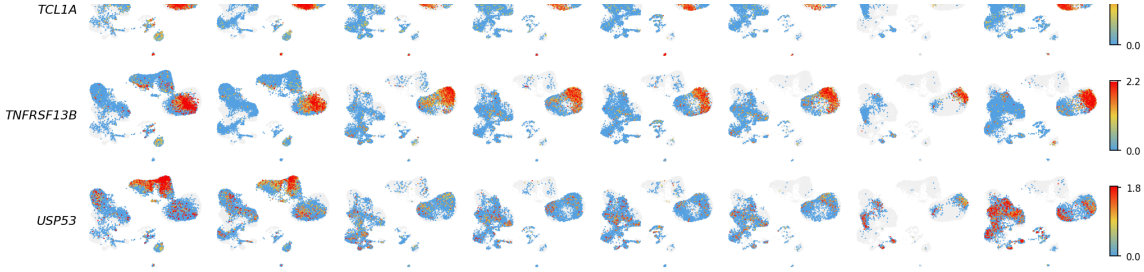
In [44]:
```
filefig = root_figure / "celltype_umap_dotplot.png"

if filefig.exists():
    plt.figure(figsize=(20,12))
    img = plt.imread(filefig)
    plt.imshow(img)
```



In [45]:
```
apply_figure_style(sizes=(8, 7, 6))

fig = plot_panel_umap_by_tissue_age(
    adata_ref, genes_plot,
    out_png = root_figure / 'bcell_genes_umap_by_tissue_age.png',
    labels={'MS4A1': 'MS4A1 (CD20)'},
    q=99.2, cmap_num=2
)
```


Tonsil · babies
16.2k cells
2 donors
Tonsil · young
13.5k cells
5 donors
Spleen · adult
8.2k cells
1 donor
iLN · adult
7.5k cells
1 donor
tLN · adult
7.4k cells
1 donor
mLN · adult
6.0k cells
1 donor
mLN · old adult
1.8k cells
1 donor
mLN · elder
12.6k cells
4 donors
AICDA
ARHGAP10
BAIAP2L1
CCND2
CCR7
CD19
CD22
CD27
CD28
CD52
CD53
CD79A
CD79B
CDK1
CDS2
CLECL1
CRIP3
DERL3
FCER2


FRZB
3.3
0.0
GPR183
3.7
0.0
HIST1H3B
1.5
0.0
IFIT1
1.2
0.0
IFIT3
1.3
0.0
IGHD
3.1
0.0
IGHG2
2.1
0.0
IGHGP
1
0
IGHM
4.2
0.0
LEF1
2.5
0.0
LILRA4
1.5
0.0
MS4A1 (CD20)
4.3
0.0
MYADM
2.7
0.0
PAX5
2.3
0.0
RRM2
1.6
0.0
SELL
3
0
SH3TC1
2.1
0.0
SLC2A5
1.7
0.0
TCF7
2.8
0.0
3.5



colour = log1p(CP10k) clipped at the 99.2th percentile per gene (non-zero values for very sparse genes), one scale per row. Grey = all other cells.

In [ ]:

AID, the enzyme encoded by *AICDA*

is the single best marker of germinal-centre B cells, so your observation matches biology. "Only in tonsils", though, needs checking against how this reference was assembled: tissue here is confounded with study, donor age and 10x chemistry. I'll quantify it before critiquing:

- *AICDA* detection per tissue within each B subset, especially within GC B cells.
- How many GC B cells each tissue contributes at all.
- Whether the signal holds per donor and per chemistry (3′ vs 5′).
- Whether the "naive/memory" *AICDA*$^+$ cells are single-UMI noise or GC-like cells under a non-GC label.

The data partly contradicts "only in tonsils". One more check, on the *AICDA*$^+$ cells labelled memory or naive: do they carry a germinal-centre program, and are they likely doublets?

Your observation that *AICDA* is mostly in the dark zone (DZ) and cycling B cells is correct and fits the biology. "Only in tonsils" isn't: within germinal-centre (GC) B cells, the spleen and lymph nodes express *AICDA* at about the same rate. Tonsils dominate only because they contain almost all of the GC B cells in this reference.

What AICDA does

AICDA encodes **AID (activation-induced cytidine deaminase)**, an APOBEC-family enzyme. It converts cytosine to uracil in single-stranded DNA at actively transcribed immunoglobulin loci. That one reaction starts two processes:

- **Somatic hypermutation (SHM):** point mutations in the antibody variable regions. Proliferating DZ cells acquire the mutations, and light-zone (LZ) cells with higher affinity are selected. This drives affinity maturation.
- **Class-switch recombination (CSR):** double-strand breaks in switch regions change the antibody class from IgM/IgD to IgG, IgA or IgE. CSR can also happen outside germinal centres, in activated B cells at the T–B border.

Other points:

- **Regulation:** AID is induced by CD40L together with IL-4 or IL-21, and by TLR signals. It is tightly controlled after translation, through nuclear export and degradation.
- **Disease:** loss of AID causes hyper-IgM syndrome type 2 (no class switching, no hypermutation). Off-target AID activity causes oncogenic mutations and translocations such as *MYC–IGH* in Burkitt lymphoma and diffuse large B-cell lymphoma.

Critique

1. "Only in tonsils" is mostly a cell-composition effect. Half of tonsil B cells are GC B cells (DZ/LZ/prePB/Cycling), against 0.5–4% in the other tissues:

Tissue	GC B cells	Share of B cells
Tonsil	10,304	50%
tLN	107	4.4%
Spleen	122	2.9%
iLN	69	2.0%
mLN	35	0.5%

That is why tonsils contribute 96% of all *AICDA*⁺ cells. Within GC B cells, per donor (≥ 20 GC cells each), detection is similar everywhere:

- tonsil donors: 21–47%;
- spleen: 42–46%;
- iLN: 27–30%;
- tLN: 19–27%.

So *AICDA* marks GC B cells in every tissue. Pediatric tonsils simply have many active germinal centres, consistent with constant antigen exposure in childhood.

2. Tissue is confounded with donor, age and study.

- All tonsils come from King 2020, from children.
- Spleen, iLN and tLN all come from one adult organ donor (A16, Park 2020).
- mLN comes from James 2020. It has only 35 GC B cells and no donor with ≥ 20, so it can't be evaluated.

Chemistry is not the explanation: within tonsil GC B cells, 3′ and 5′ data give 29.2% vs 29.3%.

3. Detection is limited by sparse capture. Between 52% and 79% of *AICDA*⁺ cells have a single UMI. So "41% of DZ cells" understates true expression, and a cell without *AICDA* reads may still express it. Ranking DZ above LZ (41% vs 9% in tonsil) is supported, and fits AID acting mainly during DZ proliferation. The exact percentages are partly set by how much RNA was captured.

4. "A little in pre-GC" is not supported. I found 0 of 404 `B_preGC` cells with any *AICDA* read.

5. The *AICDA*⁺ naive and memory cells are probably not true naive or memory cells.

- **"Naive":** 23 of 8,924 cells (0.26%). In tonsil, these 19 cells carry a full GC program, unlike the *AICDA*⁻ naive cells:

Gene	*AICDA*⁺ "naive"	*AICDA*⁻ naive
RGS13	68%	2%
BCL6	63%	9%
GCSAM	68%	6%
MKI67	58%	0%

They look like mislabelled GC or cycling cells.

- **"Memory":** 96 of 13,476 cells (0.7%), 78 of them in tonsil. They show a partial GC program:

Gene	*AICDA*⁺ memory	*AICDA*⁻ memory
RGS13	24%	9%
LMO2	29%	8%
MKI67	19%	1%

Plausible explanations: cells that recently left the GC (memory precursors), mislabelled LZ cells, or ambient RNA in tonsil suspensions, which are rich in GC transcripts.

- Their doublet scores are not elevated, but doublets were already filtered from this reference, so that check carries little weight.

6. mRNA is not activity. AID is regulated mostly after translation, so *AICDA* transcripts don't show that hypermutation or class switching is happening. This reference includes BCR data that could test it directly: `IGH_MU_FREQ` (IGH mutation frequency) and `ISOTYPE`. I haven't run that check.

7. These are cell-level percentages from few donors. For mLN and the A16 tissues, the GC B-cell counts are too small for firm conclusions.

I saved two tables in the output tray:

- `aicda_by_subset_tissue.csv` — % *AICDA*⁺ and cell counts for each B subset × tissue
- `aicda_gc_by_donor.csv` — % *AICDA*⁺ within GC B cells per donor, study and chemistry

In [46]:
```
adata_ref.var.head(6).T.head(12)
```

Out[46]:

GeneID-2	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
GeneName-2	16551	11070	12375	
feature_types	nan	nan	nan	
feature_types-0	Gene Expression	Gene Expression	Gene Expression	
feature_types-1	Gene Expression	Gene Expression	Gene Expression	
gene_ids-1	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-4861STDY7135913-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-4861STDY7135914-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-4861STDY7208412-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-4861STDY7208413-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-Human_colon_16S7255677-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-Human_colon_16S7255678-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-Human_colon_16S8000484-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E

In [47]:
```
# var re-indexing (symbol column + Ensembl index) is done once in cell 34
# Repeating it here would overwrite var['symbol'] with Ensembl IDs and ra
assert "symbol" in adata_ref.var.columns and str(adata_ref.var_names[0]).
adata_ref.var[["symbol"]].head(3)
```

Out[47]:

GeneID-2	symbol
ENSG00000188976	NOC2L
ENSG00000188290	HES4
ENSG00000187608	ISG15

In [48]:
```
cols = ['symbol', 'GeneName-2', 'feature_types', 'feature_types-0', 'feat
       'gene_ids-1', 'gene_ids-4861STDY7135913-0',
       'genome-1', 'n_cells', 'nonz_mean', 'mean_cov_effect_Subset_B_Cycl
       'mean_cov_effect_Subset_B_GC_DZ',
       'mean_sample_effectSample_BCP009_Total',
       'mean_sample_effectSample_Human_colon_16S7255677',
       'mean_sample_effectSample_Human_colon_16S7255678',
       'mean_sample_effectSample_Human_colon_16S8000484',
       'mean_sample_effectSample_Pan_T7935494',]

adata_ref.var[cols].head(3).T
```

Out[48]:

	GeneID-2	ENSG00000188976	ENSG0000018
symbol	NOC2L	H
GeneName-2	16551	1
feature_types	nan	
feature_types-0	Gene Expression	Gene Expres
feature_types-1	Gene Expression	Gene Expres
gene_ids-1	ENSG00000188976	ENSG0000018
gene_ids-4861STDY7135913-0	ENSG00000188976	ENSG0000018
genome-1	GRCh38	GR
n_cells	11327	
nonz_mean	1.119714	1.784
mean_cov_effect_Subset_B_Cycling	0.370568	0.00
mean_cov_effect_Subset_B_GC_DZ	0.203758	0.00
mean_sample_effectSample_BCP009_Total	0.002198	0.00
mean_sample_effectSample_Human_colon_16S7255677	0.005277	0.23
mean_sample_effectSample_Human_colon_16S7255678	0.05186	0.41
mean_sample_effectSample_Human_colon_16S8000484	0.07157	0.0
mean_sample_effectSample_Pan_T7935494	0.00536	0.00

Note!
Before we estimate the reference cell type signature we recommend to perform very permissive genes selection. We prefer this to standard highly-variable-gene selection because our procedure keeps markers of rare genes while removing most of the uninformative genes.

The default parameters **cell_count_cutoff=5, cell_percentage_cutoff2=0.03, nonz_mean_cutoff=1.12** are a good starting point, however, you can increase the cut-off to exclude more genes.

To preserve marker genes of rare cell types we recommend low **cell_count_cutoff=5**, however, **cell_percentage_cutoff2** and **nonz_mean_cutoff** can be increased to select between 8k-16k genes.

In this 2D histogram, orange rectangle highlights genes excluded based on the combination of number of cells expressing that gene (Y-axis) and average RNA count for cells where the gene was detected (X-axis).

In this case, the downloaded dataset was already filtered using this method, hence no density under the orange rectangle (to be changed in the future version of the tutorial).

In [ ]:

cell2location: "Markers of rare genes" is a slip in the original; it should read "markers of rare **cell types**".

The problem it solves

The reference regression model estimates an expression signature for every cell type. You want to:

- **remove** genes with almost no signal, which add noise and computation.
- **keep** genes that identify any cell type, including rare ones.

Standard highly-variable-gene (HVG) selection doesn't serve the second goal well:

- **It keeps a fixed top-N** (e.g. 2,000 genes) ranked by dispersion across *all* cells. A gene expressed only in a population making up 0.5% of cells adds little to the global variance, so it can fall below the cutoff.
- **It is tuned for a different job.** HVG selection picks genes for clustering and embedding. Cell2location wants as many informative genes as possible, because more genes give better deconvolution of spots.
- **It runs on normalised, log-transformed data.** Cell2location models raw counts.

What `filter_genes` does

```
from cell2location.utils.filtering import filter_genes
selected = filter_genes(adata_ref, cell_count_cutoff=5,
                        cell_percentage_cutoff2=0.03,
nonz_mean_cutoff=1.12)
```

It works on raw counts. For each gene it computes two numbers:

- `n_cells` : how many cells have a count above 0.
- `nonz_mean` : the mean count *in those expressing cells only*.

A gene is kept if **either** rule applies:

1. **Broadly detected:** it is expressed in more than 3% of all cells (`cell_percentage_cutoff2`). This keeps genes expressed in common cell types.
2. **Rare but strongly expressed:** it is expressed in more than 5 cells (`cell_count_cutoff`) **and** its mean in the expressing cells is above 1.12 (`nonz_mean_cutoff`). This rule is how rare-cell-type markers survive. A gene found in only 200 cells, but at several UMIs per cell, looks like a real marker. A gene found in 200 cells at exactly 1 UMI each looks like ambient RNA or dropout noise.

Genes that meet neither rule are discarded. These are genes detected in 5 or fewer cells, or detected in under 3% of cells at only about 1 count each.

The value 1.12 is chosen just above 1. A gene passes rule 2 only if at least some of its expressing cells have 2 or more UMIs, which separates real expression from scattered single-molecule background.

Reading the diagnostic plot

The function draws a 2D histogram of genes: log10(`n_cells`) on the x-axis and log10(`nonz_mean`) on the y-axis.

- A vertical line marks the 3% threshold. Everything to its right is kept.
- To the left of that line, only genes above the horizontal 1.12 line are kept. These sit in the top-left region: few cells, but high counts.
- The large dense cloud at the bottom-left (low detection, about 1 count) is what gets removed.

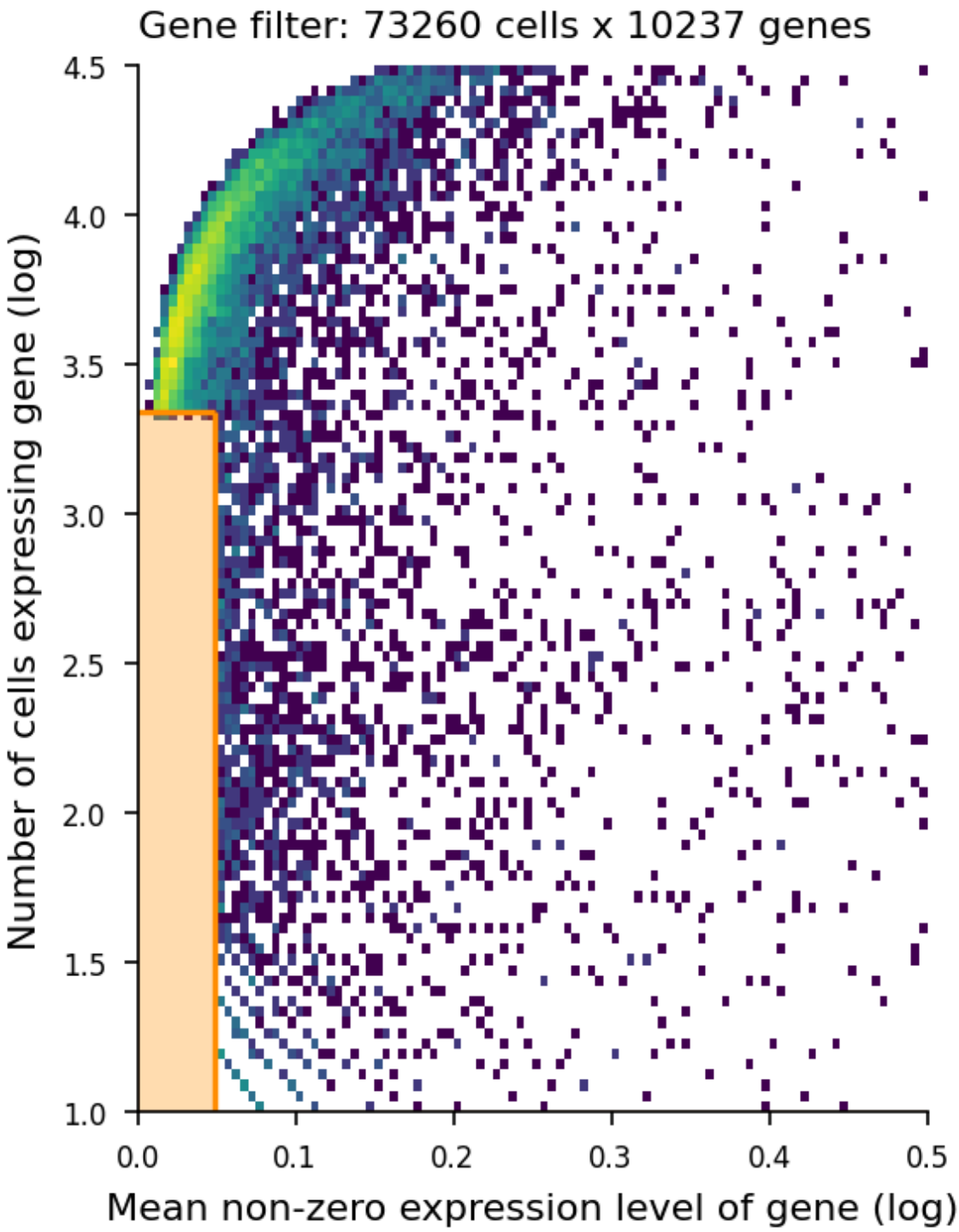
In your lymphoid-organ reference

With 73,260 cells, 3% is about 2,198 cells, so rule 1 keeps genes detected in more than ~2,200 cells. Several genes from your table rely on rule 2 instead. For example, LILRA4 (pDCs), DERL3 (plasma cells) and AICDA (germinal-centre B cells) are expressed in small populations but strongly. HVG selection might drop them; this filter keeps them. I haven't run the filter on your data, so I can't confirm which of your genes actually pass.

The thresholds can be adjusted: raising `nonz_mean_cutoff` or `cell_percentage_cutoff2` gives stricter selection. I can run the filter on your reference and report which of the 40 genes in `selected_genes_lymphoid_organ.csv` survive.

In [ ]:

In [49]:
```
from cell2location.utils.filtering import filter_genes

selected = filter_genes(adata_ref, cell_count_cutoff=5,
                        cell_percentage_cutoff2=0.03, nonz_mean_cutoff=1.
                        figsize=(3,4))
# filter the object
adata_ref = adata_ref[:, selected].copy()

len(selected), len(adata_ref), selected[:10]
```



Out[49]:
```
(10237,
 73260,
 Index(['ENSG00000188976', 'ENSG00000188290', 'ENSG00000187608',
        'ENSG00000186891', 'ENSG00000186827', 'ENSG00000078808',
        'ENSG00000160087', 'ENSG00000169972', 'ENSG00000224051',
        'ENSG00000162576'],
       dtype='object', name='GeneID-2'))
```

In [ ]:

Estimation of reference cell type signatures (NB regression)

The signatures are estimated from scRNA-seq data, accounting for batch effect, using a Negative binomial regression model.

Preparing anndata.

First, prepare anndata object for the regression model:

In [50]:
```
adata_ref.obs.head(3)
```

Out[50]:

	Age	BCELL_CLONE	BCELL_CLONE_SIZE	Donor	
AAACCTGGTAGAGCTG-1-4861STDY7135913	60-64y	nan	NaN	290b	4861STDY7135
AAACCTGGTGTCAATC-1-4861STDY7135913	60-64y	nan	NaN	290b	4861STDY7135
AAACCTGTCAGAGGTG-1-4861STDY7135913	60-64y	nan	NaN	290b	4861STDY7135

3 rows × 43 columns

In [51]:
```
adata_ref.obsm
```

Out[51]:
```
AxisArrays with keys: 'X_pca', 'X_umap'
```

In [52]:
```
adata_ref.var.head(3)
```

Out[52]:

GeneID-2	GeneName-2	feature_types	feature_types-0	feature_types-1	g
ENSG00000188976	16551	nan	Gene Expression	Gene Expression	ENSG000
ENSG00000188290	11070	nan	Gene Expression	Gene Expression	ENSG000
ENSG00000187608	12375	nan	Gene Expression	Gene Expression	ENSG000

3 rows × 74 columns

In [53]:
```
adata_ref.var.shape
```

Out[53]:
```
(10237, 74)
```

In [54]:
```
adata_ref
```

Out[54]:
```
AnnData object with n_obs × n_vars = 73260 × 10237
    obs: 'Age', 'BCELL_CLONE', 'BCELL_CLONE_SIZE', 'Donor', 'ID', 'IGH_MU_FREQ', 'ISOTYPE', 'LibraryID', 'Method', 'Population', 'PrelimCellType', 'Sample', 'Sex', 'Study', 'Tissue', 'barcode', 'batch', 'doublet_score', 'index', 'predicted_doublet', 'percent_mito', 'n_counts', 'n_genes', 'S_score', 'G2M_score', 'phase', 'VDJsum', 'cell_cycle_diff', 'PrelimCellType_new', 'leiden', 'leiden_1', 'leiden_2', 'leiden_3', 'leiden_4', 'CellType', 'CellType2', 'Subset', 'Subset_Broad', 'Subset_all', 'new_celltype', 'Subset_int', 'Subset_print', 'age_group'
    var: 'GeneName-2', 'feature_types', 'feature_types-0', 'feature_types-1', 'gene_ids-1', 'gene_ids-4861STDY7135913-0', 'gene_ids-4861STDY7135914-0', 'gene_ids-4861STDY7208412-0', 'gene_ids-4861STDY7208413-0', 'gene_ids-Human_colon_16S7255677-0', 'gene_ids-Human_colon_16S7255678-0', 'gene_ids-Human_colon_16S8000484-0', 'gene_ids-Pan_T7935494-0', 'genome-1', 'n_cells', 'nonz_mean', 'mean_cov_effect_Subset_B_Cycling', 'mean_cov_effect_Subset_B_GC_DZ', 'mean_cov_effect_Subset_B_GC_LZ', 'mean_cov_effect_Subset_B_GC_prePB', 'mean_cov_effect_Subset_B_IFN', 'mean_cov_effect_Subset_B_activated', 'mean_cov_effect_Subset_B_mem', 'mean_cov_effect_Subset_B_naive', 'mean_cov_effect_Subset_B_plasma', 'mean_cov_effect_Subset_B_preGC', 'mean_cov_effect_Subset_DC_CCR7+', 'mean_cov_effect_Subset_DC_cDC1', 'mean_cov_effect_Subset_DC_cDC2', 'mean_cov_effect_Subset_DC_pDC', 'mean_cov_effect_Subset_Endo', 'mean_cov_effect_Subset_FDC', 'mean_cov_effect_Subset_ILC', 'mean_cov_effect_Subset_Macrophages_M1', 'mean_cov_effect_Subset_Macrophages_M2', 'mean_cov_effect_Subset_Mast', 'mean_cov_effect_Subset_Monocytes', 'mean_cov_effect_Subset_NK', 'mean_cov_effect_Subset_NKT', 'mean_cov_effect_Subset_T_CD4+', 'mean_cov_effect_Subset_T_CD4+_TfH', 'mean_cov_effect_Subset_T_CD4+_TfH_GC', 'mean_cov_effect_Subset_T_CD4+_naive', 'mean_cov_effect_Subset_T_CD8+_CD161+', 'mean_cov_effect_Subset_T_CD8+_cytotoxic', 'mean_cov_effect_Subset_T_CD8+_naive', 'mean_cov_effect_Subset_T_TIM3+', 'mean_cov_effect_Subset_T_TfR', 'mean_cov_effect_Subset_T_Treg', 'mean_cov_effect_Subset_VSMC', 'mean_sample_effectSample_4861STDY7135913', 'mean_sample_effectSample_4861STDY7135914', 'mean_sample_effectSample_4861STDY7208412', 'mean_sample_effectSample_4861STDY7208413', 'mean_sample_effectSample_4861STDY7462253', 'mean_sample_effectSample_4861STDY7462254', 'mean_sample_effectSample_4861STDY7462255', 'mean_sample_effectSample_4861STDY7462256', 'mean_sample_effectSample_4861STDY7528597', 'mean_sample_effectSample_4861STDY7528598', 'mean_sample_effectSample_4861STDY7528599', 'mean_sample_effectSample_4861STDY7528600', 'mean_sample_effectSample_BCP002_Total', 'mean_sample_effectSample_BCP003_Total', 'mean_sample_effectSample_BCP004_Total', 'mean_sample_effectSample_BCP005_Total', 'mean_sample_effectSample_BCP006_Total', 'mean_sample_effectSample_BCP008_Total', 'mean_sample_effectSample_BCP009_Total', 'mean_sample_effectSample_Human_colon_16S7255677', 'mean_sample_effectSample_Human_colon_16S7255678', 'mean_sample_effectSample_Human_colon_16S8000484', 'mean_sample_effectSample_Pan_T7935494', 'symbol'
    uns: 'Age_colors', 'Donor_colors', 'LibraryID_colors', 'Method_colors', 'Study_colors', 'Subset_Broad_colors', 'Subset_colors', 'Tissue_colors', 'leiden', 'neighbors', 'pca', 'phase_colors', 'regression_mod', 'umap'
    obsm: 'X_pca', 'X_umap'
    obsp: 'connectivities', 'distances'
    layers: None (.X)
```

prepare anndata for the regression model

In [55]:
```
cell2location.models.RegressionModel.setup_anndata(adata=adata_ref,
                        # 10X reaction / sample / batch
                        batch_key='Sample',
                        # cell type, covariate used for constructing sign
```

```
                            labels_key='Subset',
                            # multiplicative technical effects (platform, 3'
                            categorical_covariate_keys=['Method']
                           )
```

In [56]:
```
# create the regression model
from cell2location.models import RegressionModel
mod = RegressionModel(adata_ref)

# view anndata_setup as a sanity check
mod.view_anndata_setup()
```

```
Anndata setup with scvi-tools version 1.5.1.

Setup via `RegressionModel.setup_anndata` with arguments:
{
    'layer': None,
    'batch_key': 'Sample',
    'labels_key': 'Subset',
    'categorical_covariate_keys': ['Method'],
    'continuous_covariate_keys': None
}
```

Summary Statistics

Summary Stat Key	Value
n_batch	23
n_cells	73260
n_extra_categorical_covs	1
n_extra_continuous_covs	0
n_labels	34
n_vars	10237

Data Registry

Registry Key	scvi-tools Location
X	adata.X
batch	adata.obs['_scvi_batch']
extra_categorical_covs	adata.obsm['_scvi_extra_categorical_covs
ind_x	adata.obs['_indices']
labels	adata.obs['_scvi_labels']

batch State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['Sample']	4861STDY7135913	0
	4861STDY7135914	1
	4861STDY7208412	2
	4861STDY7208413	3
	4861STDY7462253	4
	4861STDY7462254	5
	4861STDY7462255	6
	4861STDY7462256	7
	4861STDY7528597	8
	4861STDY7528598	9
	4861STDY7528599	10
	4861STDY7528600	11
	BCP002_Total	12
	BCP003_Total	13
	BCP004_Total	14
	BCP005_Total	15
	BCP006_Total	16
	BCP008_Total	17
	BCP009_Total	18
	Human_colon_16S7255677	19
	Human_colon_16S7255678	20
	Human_colon_16S8000484	21
	Pan_T7935494	22

labels State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['Subset']	B_Cycling	0
	B_GC_DZ	1
	B_GC_LZ	2
	B_GC_prePB	3
	B_IFN	4
	B_activated	5
	B_mem	6
	B_naive	7
	B_plasma	8
	B_preGC	9
	DC_CCR7+	10
	DC_cDC1	11
	DC_cDC2	12
	DC_pDC	13
	Endo	14
	FDC	15
	ILC	16
	Macrophages_M1	17
	Macrophages_M2	18
	Mast	19
	Monocytes	20
	NK	21
	NKT	22
	T_CD4+	23
	T_CD4+_TfH	24
	T_CD4+_TfH_GC	25
	T_CD4+_naive	26
	T_CD8+_CD161+	27
	T_CD8+_cytotoxic	28
	T_CD8+_naive	29
	T_TIM3+	30
	T_TfR	31
	T_Treg	32
	VSMC	33

extra_categorical_covs State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['Method']	3GEX	0
	5GEX	1

Training model.

Now we train the model to estimate the reference cell type signatures.

Note that to achieve convergence on your data (=to get stabilization of the loss) you may need to increase `max_epochs=250` (See below).

Also note that here we are using `batch_size=2500` which is much larger than scvi-tools default and perform training on all cells in the data (`train_size=1`) - both parameters are defaults.

In [57]:
```
adata_ref_filename = Path(f"{ref_run_name}/sc.h5ad")
adata_ref_filename.exists(), adata_ref_filename
```

Out[57]:
```
(True,
 PosixPath('/home/flavio/uv/squidpy/results/lymph_nodes_analysis/referen
ce_signatures/sc.h5ad'))
```

In [58]:
```
if not adata_ref_filename.exists():
    mod.train(max_epochs=250, accelerator='gpu')

    # In this section, we export the estimated cell abundance (summary of
    adata_ref = mod.export_posterior(
        adata_ref, sample_kwargs={'num_samples': 1000, 'batch_size': 2500
    )

    # Save model
    mod.save(f"{ref_run_name}", overwrite=True, save_anndata=True)
    print(f"Model saved with anndata to {ref_run_name}")

    # Save anndata object with results
    adata_ref.write(adata_ref_filename)
    print(f"Model saved to {adata_ref_filename}")
else:
    # Load AnnData with posterior results (already registered via setup_a
    # adata_ref = sc.read_h5ad(adata_ref_filename)

    # Reload the trained model bound to that AnnData
    mod = cell2location.models.RegressionModel.load(f"{ref_run_name}", ad
    adata_ref = mod.adata

    print(f"Model loaded from {ref_run_name}")
```

```
INFO     File /home/flavio/uv/squidpy/results/lymph_nodes_analysis/referen
ce_signatures/model.pt already downloaded
Model loaded from /home/flavio/uv/squidpy/results/lymph_nodes_analysis/ref
erence_signatures
```

In [59]:
```
adata_ref.var.shape
```

Out[59]:
```
(10237, 74)
```

In [ ]:
```

```

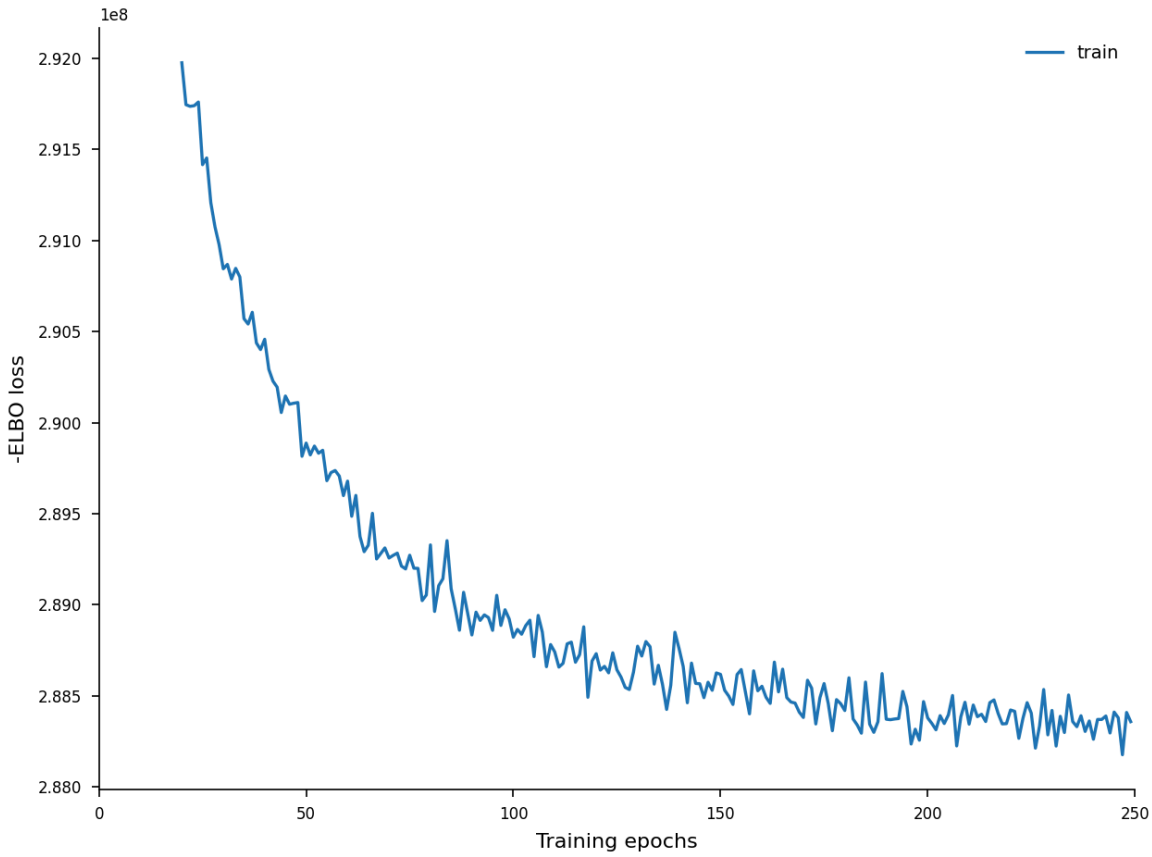
In [60]:
```
import torch

saved = torch.load(f"{ref_run_name}/model.pt", map_location="cpu", weight
model_genes = pd.Index(saved["var_names"])
adata_genes = adata_ref.var_names

print(len(model_genes), len(adata_genes))
print("missing from adata:", model_genes.difference(adata_genes)[:20].tol
print("extra in adata:", adata_genes.difference(model_genes)[:20].tolist(
```

```
10237 10237
missing from adata: []
extra in adata: []
```

Determine if the model needs more training.

Here, we plot ELBO loss history during training, removing first 20 epochs from the plot. This plot should have a decreasing trend and level off by the end of training. If it is still decreasing, increase `max_epochs`.

In [61]:
```
mod.plot_history(20)
```



You can compute the 5%, 50% and 95% quantiles of the posterior distribution directly rather than using 1000 samples from the distribution (or any other quantiles). This speeds up application on large datasets and requires less memory - however, posterior df_mean and standard deviation cannot be computed this way.

In [62]:
```
adata_ref = mod.export_posterior(
    adata_ref, use_quantiles=True,
    # choose quantiles
    add_to_varm=["q05","q50", "q95", "q0001"],
    sample_kwargs={'batch_size': 2500, 'accelerator': 'gpu'}
)
```

In [63]:
```
# picks 'means', else 'q50', else 'q05' (export_posterior(use_quantiles=T
inf_aver, summary = get_reference_signatures(adata_ref)
print(f"Using {summary} signatures")

inf_aver.head(6).T.head(8)
```

```
Using q50 signatures
```

Out[63]:

GeneID-2	ENSG00000188976	ENSG00000188290	ENSG00000187608	ENSG000001868
B_Cycling	0.433266	0.001882	0.384278	0.0198
B_GC_DZ	0.244424	0.000460	0.213292	0.0005
B_GC_LZ	0.311102	0.000511	0.274602	0.0558
B_GC_prePB	0.336622	0.037657	0.491036	0.0582
B_IFN	0.150344	0.037758	3.888645	0.0064
B_activated	0.174388	0.011069	0.132449	0.0038
B_mem	0.183805	0.033827	0.256200	0.0034
B_naive	0.154610	0.006882	0.131011	0.0002

In [ ]:

In [64]:
```
factor_names = adata_ref.uns['mod']['factor_names']
inf_aver = adata_ref.varm['q50_per_cluster_mu_fg'][
    [f'q50_per_cluster_mu_fg_{i}' for i in factor_names]
].copy()
inf_aver.columns = factor_names

inf_aver
```

Out[64]:

GeneID-2	B_Cycling	B_GC_DZ	B_GC_LZ	B_GC_prePB	B_IFN	B_activated
ENSG00000188976	0.433266	0.244424	0.311102	0.336622	0.150344	0.174388
ENSG00000188290	0.001882	0.000460	0.000511	0.037657	0.037758	0.011069
ENSG00000187608	0.384278	0.213292	0.274602	0.491036	3.888645	0.132449
ENSG00000186891	0.019857	0.000512	0.055829	0.058270	0.006450	0.003825
ENSG00000186827	0.007621	0.000338	0.005983	0.020730	0.008931	0.000947
...	...	...	...	...	...	...
IGL_VDJsum	9.899196	7.434495	3.698826	10.365810	4.099563	3.026543
TCRA_VDJsum	0.024391	0.002943	0.009637	0.039222	0.002000	0.002072
TCRB_VDJsum	0.045600	0.001751	0.006663	0.004124	0.004914	0.003644
TCRD_VDJsum	0.000379	0.001717	0.002060	0.060702	0.007969	0.000303
TCRG_VDJsum	0.014638	0.008726	0.013115	0.026978	0.015475	0.006124

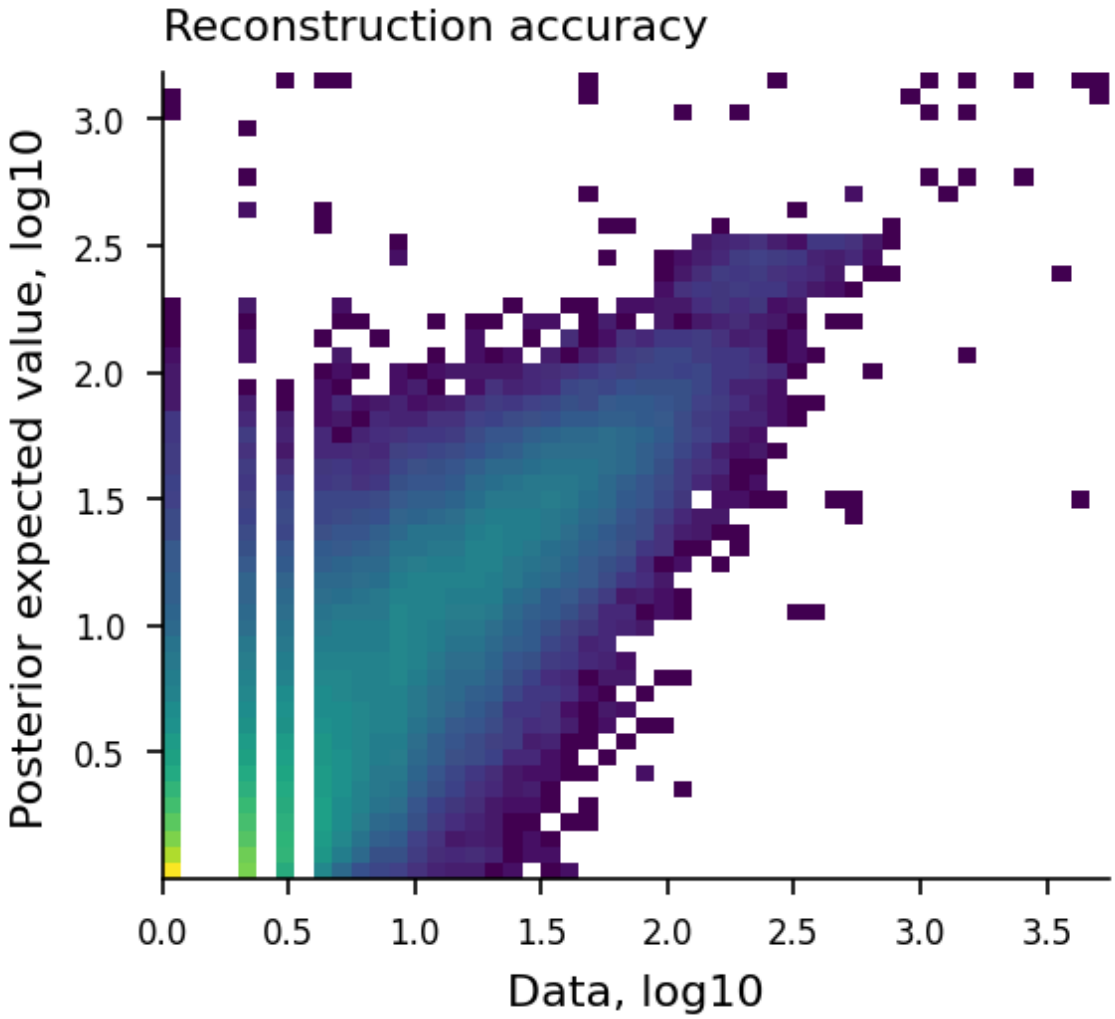
10237 rows × 34 columns

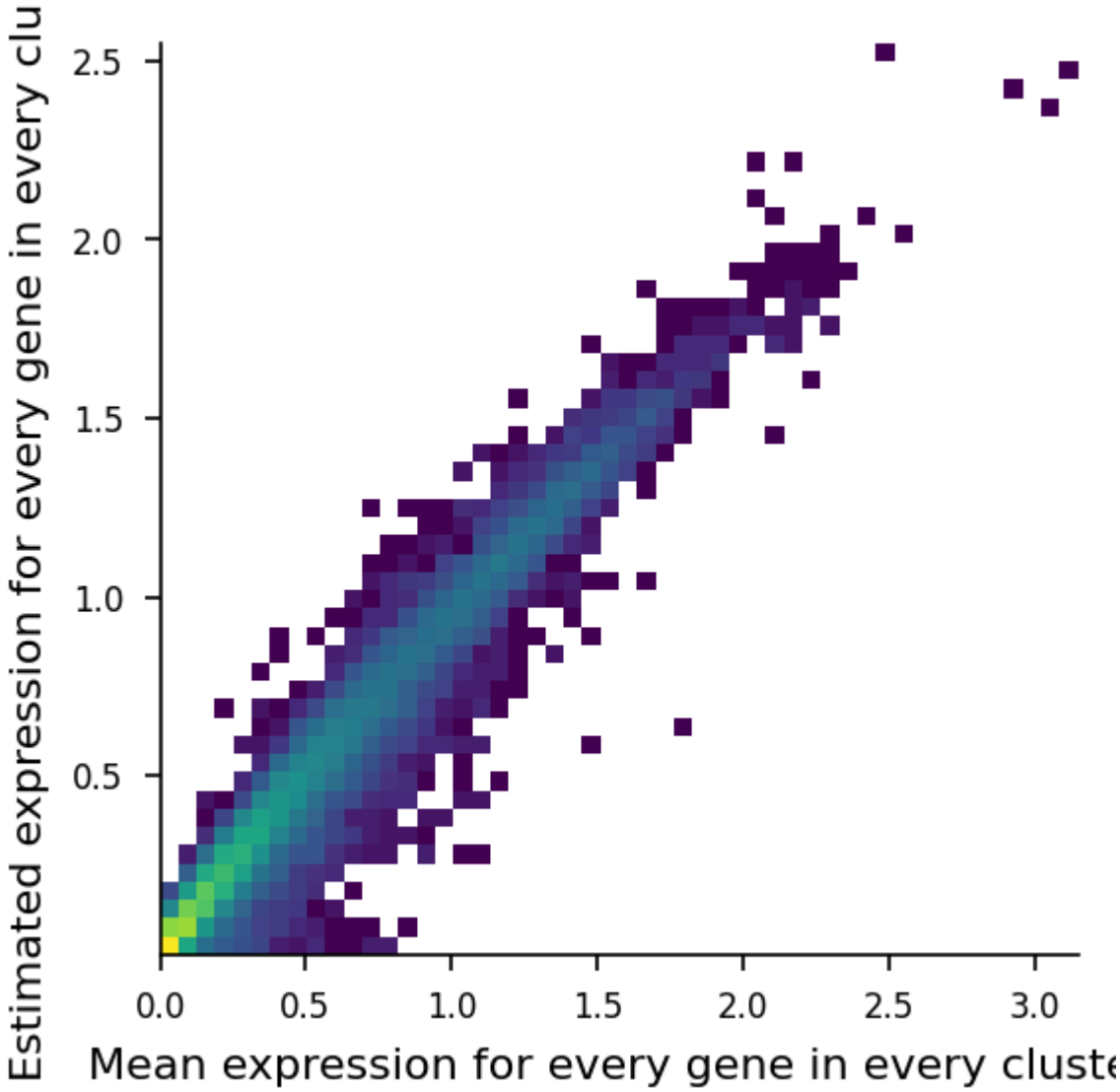
Examine QC plots.

1. Reconstruction accuracy to assess if there are any issues with inference. This 2D histogram plot should have most observations along a noisy diagonal.

2. The estimated expression signatures are distinct from mean expression in each cluster because of batch effects. For scRNA-seq datasets which do not suffer from batch effect (this dataset does), cluster average expression can be used instead of estimating signatures with a model. When this plot is very different from a diagonal plot (e.g. very low values on Y-axis, density everywhere) it indicates problems with signature estimation.

In [65]:
```
with plt.rc_context({"figure.figsize": (3,3)}):
    mod.plot_QC(summary_name="q50");    # median signatures
```





Extracting reference cell types signatures as a pd.DataFrame.
All parameters of the a Negative Binomial regression model are exported into reference anndata object, however for spatial mapping we just need the estimated expression of every gene in every cell type. Here we extract that from standard output:

In [66]:
```
[x for x in adata_ref.var.columns if 'cluster' in x]
```

Out[66]:
```
[]
```

In [67]:
```
# export estimated expression in each cluster
# (the original hard-coded 'means_per_cluster_mu_fg' -> KeyError after a
inf_aver, summary = get_reference_signatures(adata_ref, prefer=("means",
print(f"Using {summary} signatures")
inf_aver.iloc[0:5, 0:5]
```

```
Using q50 signatures
```

Out[67]:

GeneID-2	B_Cycling	B_GC_DZ	B_GC_LZ	B_GC_prePB	B_IFN
ENSG00000188976	0.433266	0.244424	0.311102	0.336622	0.150344
ENSG00000188290	0.001882	0.000460	0.000511	0.037657	0.037758
ENSG00000187608	0.384278	0.213292	0.274602	0.491036	3.888645
ENSG00000186891	0.019857	0.000512	0.055829	0.058270	0.006450
ENSG00000186827	0.007621	0.000338	0.005983	0.020730	0.008931

In [ ]:

Cell2location: spatial mapping

Find shared genes and prepare anndata. Subset both anndata and reference signatures:

In [68]:
```
# find shared genes and subset both anndata and reference signatures
intersect = np.intersect1d(adata_vis.var_names, inf_aver.index)
adata_vis = adata_vis[:, intersect].copy()
inf_aver = inf_aver.loc[intersect, :].copy()

len(intersect), len(adata_vis.var_names), len(inf_aver.index)
```

Out[68]: (10217, 10217, 10217)

prepare anndata for cell2location model

In [69]:
```
cell2location.models.Cell2location.setup_anndata(adata=adata_vis, batch_k
```

Important
To use cell2location spatial mapping model, you need to specify 2 user-provided hyperparameters (**N_cells_per_location** and **detection_alpha**)

For detailed guidance on setting these hyperparameters and their impact see the flow diagram and the note.

Choosing hyperparameter `N_cells_per_location`!

It is useful to adapt the expected cell abundance `N_cells_per_location` to every tissue. This value can be estimated from paired histology images and as described in the note above. Change the value presented in this tutorial (`N_cells_per_location=30`) to the value observed in your your tissue.

Choosing hyperparameter `detection_alpha`!

To improve accuracy & sensitivity on datasets with large technical variability in RNA detection sensitivity within the slide/batch - you need to relax regularisation of per-

location normalisation (use `detection_alpha=20`). High technical variability in RNA detection sensitivity is present in your sample when you observe the spatial distribution of total RNA count per location that doesn't match expected cell numbers based on histological examination.

We initially opted for high regularisation (`detection_alpha=200`) as a default because the mouse brain & human lymph node datasets used in our paper have low technical effects and using high regularisation strenght improves consistency between total estimated cell abundance per location and the nuclei count quantified from histology (Fig S8F in cell2location paper). However, in many collaborations, we see that Visium experiments on human tissues suffer from technical effects. This motivates the new default value of `detection_alpha=20` and the recommendation of testing both settings on your data (`detection_alpha=20` and `detection_alpha=200`).

In [70]:
```
# create and train the model
mod = cell2location.models.Cell2location(
    adata_vis, cell_state_df=inf_aver,
    # the expected average cell abundance: tissue-dependent
    # hyper-prior which can be estimated from paired histology:
    N_cells_per_location=30,
    # hyperparameter controlling normalisation of
    # within-experiment variation in RNA detection:
    detection_alpha=20
)
mod.view_anndata_setup()
```

```
Anndata setup with scvi-tools version 1.5.1.

Setup via `Cell2location.setup_anndata` with arguments:
{
    'layer': None,
    'batch_key': 'sample',
    'labels_key': None,
    'categorical_covariate_keys': None,
    'continuous_covariate_keys': None
}
```

Summary Statistics

Summary Stat Key	Value
n_batch	1
n_cells	4035
n_extra_categorical_covs	0
n_extra_continuous_covs	0
n_labels	1
n_vars	10217

Data Registry

Registry Key	scvi-tools Location
X	adata.X
batch	adata.obs['_scvi_batch']
ind_x	adata.obs['_indices']
labels	adata.obs['_scvi_labels']

batch State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['sample']	V1_Human_Lymph_Node	0

labels State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['_scvi_labels']	0	0

Training cell2location:

In [71]:
```
adata_vis_filename_run = run_name / "sp.h5ad"
adata_vis_filename_run.exists(), adata_vis_filename_run
```

Out[71]:
```
(True,
 PosixPath('/home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map/sp.h5ad'))
```

In [72]:
```
if not adata_vis_filename_run.exists():
    mod.train(max_epochs=15000,
            # train using full data (batch_size=None)
            batch_size=None,
            # use all data points in training because
            # we need to estimate cell abundance at all locations
            train_size=1,
            accelerator='gpu',
            )

    # In this section, we export the estimated cell abundance (summary of
    adata_vis = mod.export_posterior(
        adata_vis, use_quantiles=True,
        add_to_obsm=["q05", "q50", "q95", "q0001"],
        sample_kwargs={'batch_size': mod.adata.n_obs, 'accelerator': 'gpu
        # sample_kwargs={'num_samples': 1000, 'batch_size': mod.adata.n_o
    )

    # Compute expected expression per cell type
    expected_dict = mod.module.model.compute_expected_per_cell_type(
        mod.samples["post_sample_q05"], mod.adata_manager
    )

    # Add to anndata layers
    for i, n in enumerate(mod.factor_names_):
        adata_vis.layers[n] = expected_dict['mu'][i]

    # Save model
    mod.save(f"{run_name}", overwrite=True)
    print(f"Model saved to {run_name}")

    # Save anndata object with results
    adata_vis.write(adata_vis_filename_run)
    print(f"adata_vis saved to {adata_vis_filename_run}")
else:
    # Load AnnData with posterior abundances in .obsm (already registered
```

```
    adata_vis = sc.read_h5ad(adata_vis_filename_run)
    print(f"adata_vis read from {adata_vis_filename_run}")

    # Reload the trained spatial model bound to that AnnData
    mod = cell2location.models.Cell2location.load(f"{run_name}", adata_vi
    print(f"Model loaded from {run_name}")

    # mod.samples is not saved with the model: rebuild it (needed by plot
    adata_vis = ensure_posterior_samples(mod, adata_vis, quantiles=["q05"
    print("posterior samples:", sorted(mod.samples.keys()))

# Common to both branches: copy 5% quantile abundances into .obs for plot
factor_names = adata_vis.uns['mod']['factor_names']
adata_vis.obs[factor_names] = adata_vis.obsm['q05_cell_abundance_w_sf']
```

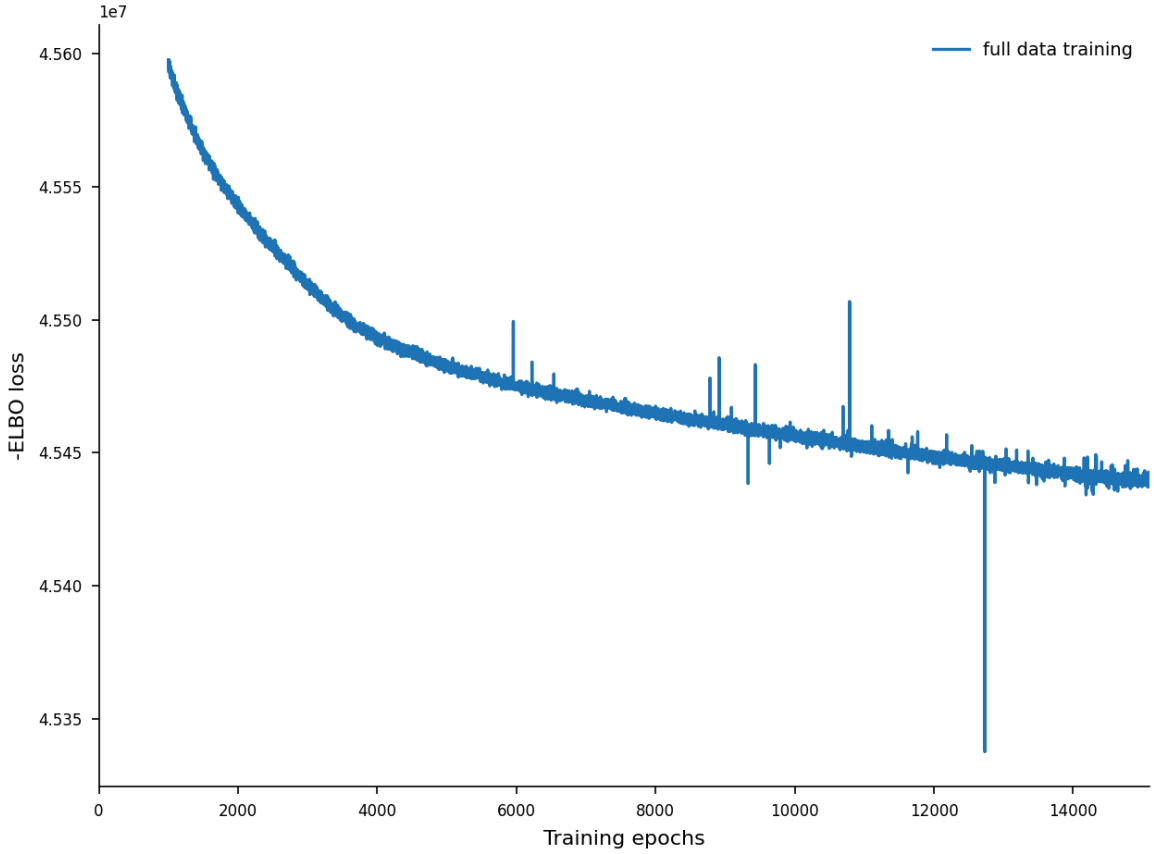
```
adata_vis read from /home/flavio/uv/squidpy/results/lymph_nodes_analysis/c
ell2location_map/sp.h5ad
INFO     File /home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2lo
cation_map/model.pt already downloaded
Model loaded from /home/flavio/uv/squidpy/results/lymph_nodes_analysis/cel
l2location_map
posterior samples: ['post_sample_q0001', 'post_sample_q05', 'post_sample_q
50', 'post_sample_q95']
```

In [73]:
```
mod.adata.n_obs
```

Out[73]: 4035

plot ELBO loss history during training, removing first 100 epochs from the plot

In [74]:
```
mod.plot_history(1000)
plt.legend(labels=['full data training']);
```



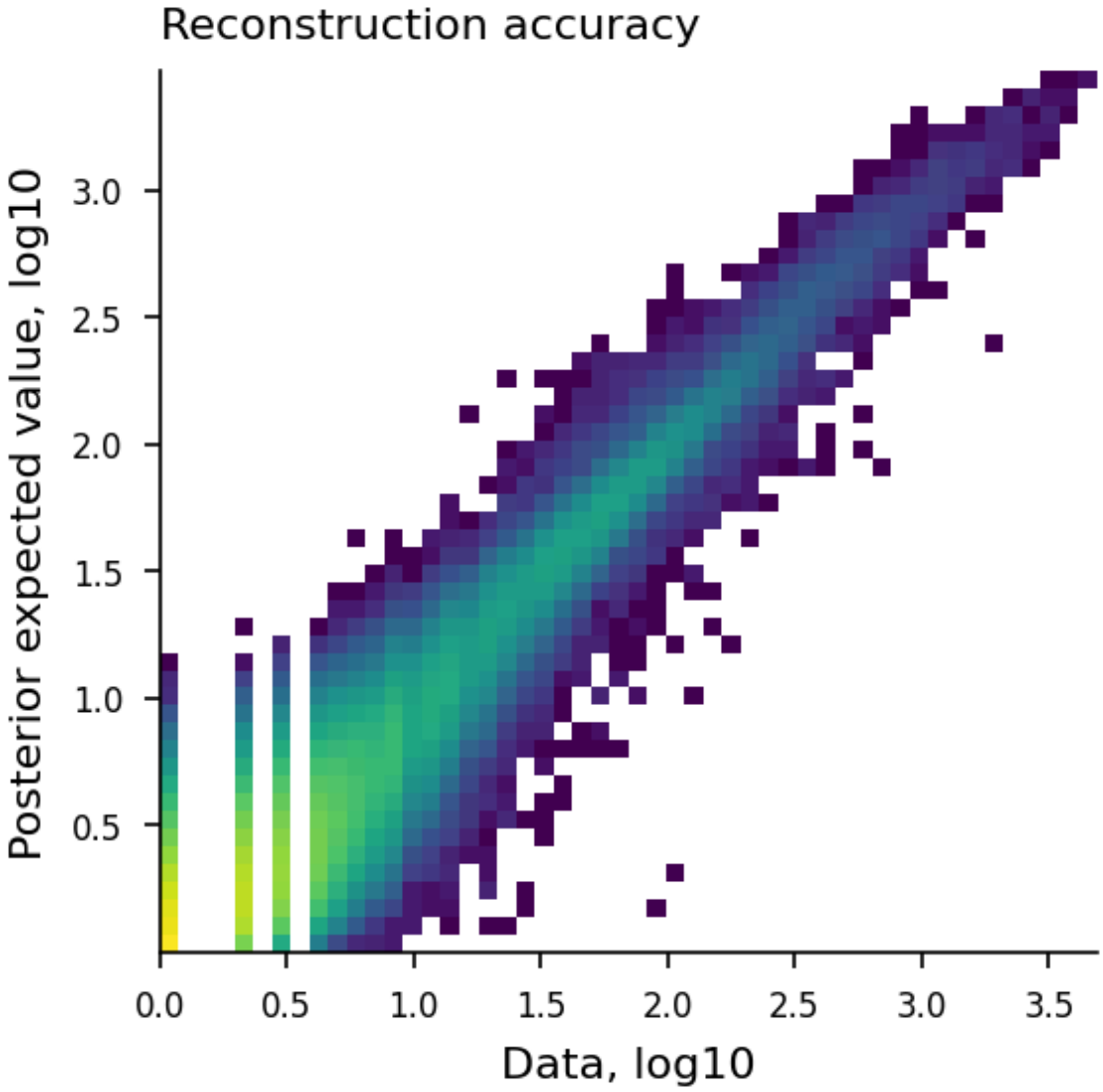
Exporting estimated posterior distributions of cell abundance and saving results:

The model and output h5ad can be loaded later like this:

```
adata_ref_filename = f"{run_name}/sp.h5ad"
adata_vis = sc.read_h5ad(adata_ref_filename)
mod = cell2location.models.Cell2location.load(f"{run_name}",
adata_vis)
```

Assessing mapping quality. Examine reconstruction accuracy to assess if there are any issues with mapping. The plot should be roughly diagonal, strong deviations will signal problems that need to be investigated.

In [75]:
```
with plt.rc_context({"figure.figsize": (3,3)}):
    mod.plot_QC(summary_name="q50");    # median signatures
```



When intergrating multiple spatial batches and when working with datasets that have substantial variation of detected RNA within slides (that cannot be explained by high cellular density in the histology), it is important to assess whether cell2location normalised those effects. You expect to see similar total cell abundance across batches but distinct RNA detection sensitivity (both estimated by cell2location). You expect total cell abundance to mirror high cellular density in the histology.

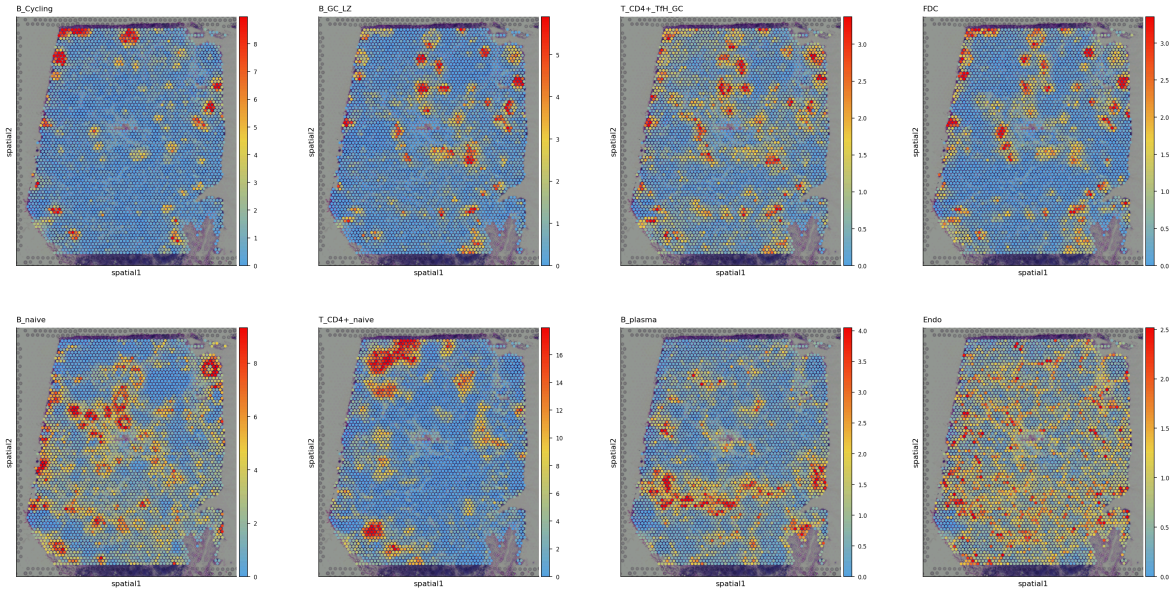
```
fig = mod.plot_spatial_QC_across_batches()
```

Visualising cell abundance in spatial coordinates

Note
We use 5% quantile of the posterior distribution, representing the value of cell abundance that the model has high confidence in (aka 'at least this amount is present').

In [76]:
```
# add 5% quantile, representing confident cell abundance, 'at least this
# to adata.obs with nice names for plottingp
adata_vis.obs[adata_vis.uns['mod']['factor_names']]= adata_vis.obsm['q05_
```

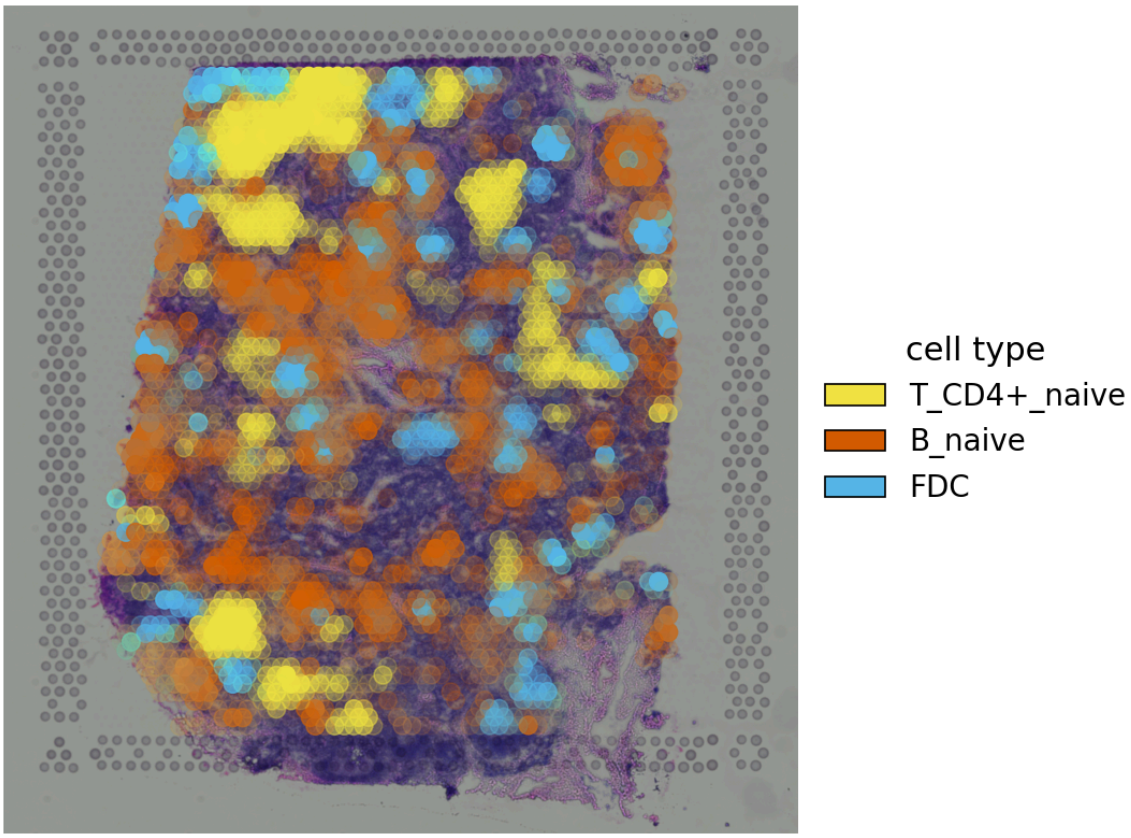
```
# select one slide
from cell2location.utils import select_slide
slide = select_slide(adata_vis, 'V1_Human_Lymph_Node')


# plot in spatial coordinates
with mpl.rc_context({'axes.facecolor':  'black',
                     'figure.figsize': [4.5, 5]}):

    sc.pl.spatial(slide, cmap=cmap_byr,
                  # show first 8 cell types
                  color=['B_Cycling', 'B_GC_LZ', 'T_CD4+_TfH_GC', 'FDC',
                         'B_naive', 'T_CD4+_naive', 'B_plasma', 'Endo'],
                  ncols=4, size=1.3,
                  img_key='hires',
                  # limit color scale at 99.2% quantile of cell abundance
                  vmin=0, vmax='p99.2'
                 )
```



In [85]:
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[85]:
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [86]:
```
%matplotlib inline

# Now we use cell2location plotter that allows showing multiple cell type
C2L_COLORS = ["#F0E442", "#D55E00", "#56B4E9", "#009E73", "#5A14A5", "#C8

clust_labels = ['T_CD4+_naive', 'B_naive', 'FDC']

c2l.plot_slide_loc_cell_types(slide, clust_labels, C2L_COLORS, root_figur
```

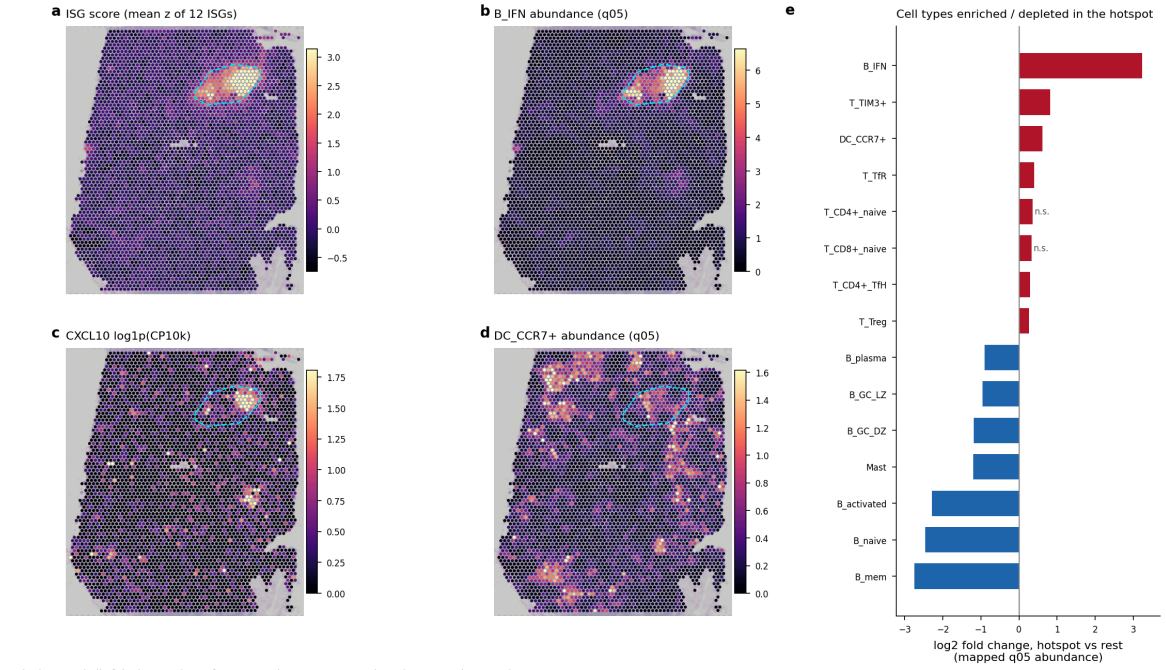


B-INF - IFIT1 and IFIT3 signatures

In [87]:
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[87]:
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [90]:
```
%matplotlib inline

ISG_GENES = ["IFIT3","IFIT1","IFIT2","IFI44L","ISG15","MX1","MX2","IFI6",

hot_res = c2l.run_isg_hotspot(adata_vis, ISG_GENES, root_results,root_fig
```



Dashed cyan = hull of the hotspot (120 of 4,035 spots). V1_Human_Lymph_Node, one section, one donor.

In [91]:
```
hot_res
```

Out[91]:

	cell_type	mean_hot	mean_rest	log2fc	p	padj
4	B_IFN	5.276842	0.518910	3.227008	0.000000e+00	0.000000e+00
30	T_TIM3+	2.070963	1.154092	0.816774	2.328564e-32	1.583424e-31
10	DC_CCR7+	0.512085	0.315345	0.621530	4.105472e-14	1.073739e-13
31	T_TfR	0.811089	0.600535	0.404534	5.629911e-11	1.196356e-10
26	T_CD4+_naive	3.568315	2.779992	0.354520	4.002597e-01	4.536277e-01
29	T_CD8+_naive	1.350425	1.067173	0.326012	1.622711e-01	2.043413e-01
24	T_CD4+_TfH	0.505526	0.402999	0.294349	5.118303e-06	1.023661e-05
32	T_Treg	0.782690	0.640921	0.269259	3.855518e-05	6.554381e-05
27	T_CD8+_CD161+	0.390787	0.354267	0.124776	1.961364e-02	2.667455e-02
21	NK	0.465257	0.463129	0.005971	8.248143e-01	8.498087e-01
28	T_CD8+_cytotoxic	0.446826	0.447773	-0.002749	9.757248e-01	9.757248e-01
23	T_CD4+	0.436560	0.468931	-0.092925	6.983534e-01	7.420005e-01
9	B_preGC	1.196184	1.304072	-0.119787	2.488750e-01	2.917845e-01
20	Monocytes	0.267828	0.302695	-0.150174	5.048176e-02	6.601461e-02
12	DC_cDC2	0.352470	0.406158	-0.180651	1.629397e-04	2.638071e-04
0	B_Cycling	0.936146	1.077111	-0.192756	9.433107e-03	1.336357e-02
22	NKT	0.214296	0.268254	-0.268027	3.217516e-05	5.757660e-05
17	Macrophages_M1	0.679277	0.852898	-0.308097	2.029939e-04	3.137178e-04
15	FDC	0.465031	0.597329	-0.329840	4.408064e-01	4.834651e-01
18	Macrophages_M2	0.382435	0.512648	-0.379749	2.203047e-01	2.675128e-01
13	DC_pDC	0.311972	0.426411	-0.396328	1.001901e-05	1.892479e-05
33	VSMC	0.496196	0.682775	-0.423952	1.185536e-11	2.687214e-11
11	DC_cDC1	0.429387	0.618506	-0.479749	8.574667e-03	1.267560e-02
14	Endo	0.545371	0.821788	-0.550189	1.260369e-13	3.060896e-13
16	ILC	0.205553	0.339408	-0.607661	2.440524e-14	6.914819e-14
3	B_GC_prePB	0.107930	0.228511	-0.818451	1.327993e-15	4.515176e-15
25	T_CD4+_TfH_GC	0.394772	0.749917	-0.846783	5.813729e-19	2.196298e-18
8	B_plasma	0.464272	0.910540	-0.901315	1.903696e-14	5.884152e-14
2	B_GC_LZ	0.416351	0.852812	-0.953010	2.388326e-21	1.015039e-20
1	B_GC_DZ	0.151245	0.407469	-1.184721	1.301721e-30	7.376419e-30
19	Mast	0.111524	0.321979	-1.203474	2.488390e-25	1.208647e-24
5	B_activated	0.246180	1.393959	-2.285482	0.000000e+00	0.000000e+00
7	B_naive	0.410620	2.485461	-2.460600	0.000000e+00	0.000000e+00
6	B_mem	0.326103	2.473387	-2.746163	0.000000e+00	0.000000e+00

Downstream analysis

Identifying discrete tissue regions by Leiden clustering<a class="anchor"

id="Identifying-discrete-tissue-regions-by-Leiden-clustering">

We identify tissue regions that differ in their cell composition by clustering locations using cell abundance estimated by cell2location.

We find tissue regions by clustering Visium spots using estimated cell abundance each cell type. We constuct a K-nearest neigbour (KNN) graph representing similarity of locations in estimated cell abundance and then apply Leiden clustering. The number of KNN neighbours should be adapted to size of dataset and the size of anatomically defined regions (e.i. hippocampus regions are rather small compared to size of the brain so could be masked by large `n_neighbors`). This can be done for a range KNN neighbours and Leiden clustering resolutions until a clustering matching the anatomical structure of the tissue is obtained.

The clustering is done jointly across all Visium sections / batches, hence the region identities are directly comparable. When there are strong technical effects between multiple batches (not the case here) `sc.external.pp.bbknn` can be in principle used to account for those effects during the KNN construction.

The resulting clusters are saved in `adata_vis.obs['region_cluster']`.

In [94]:
```
# compute KNN using the cell2location output stored in adata.obsm
sc.pp.neighbors(adata_vis, use_rep='q05_cell_abundance_w_sf', n_neighbors

# Cluster spots into regions using scanpy
sc.tl.leiden(adata_vis, resolution=1.1)

# add region as categorical variable
adata_vis.obs["region_cluster"]= adata_vis.obs["leiden"].astype("category

adata_vis
```

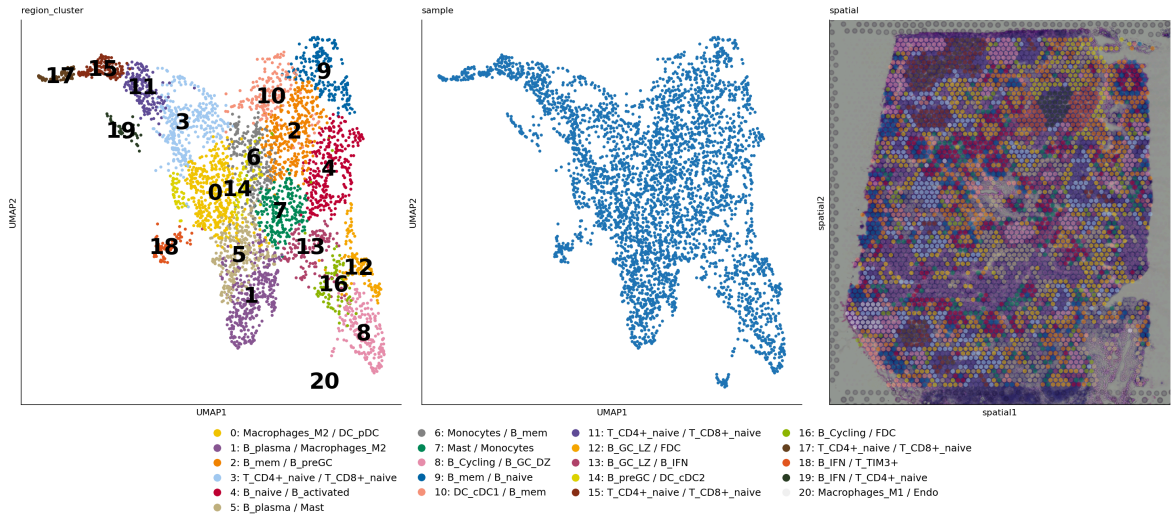
Out[94]:
```
AnnData object with n_obs × n_vars = 4035 × 10217
    obs: 'in_tissue', 'array_row', 'array_col', 'sample', '_indices', '_
scvi_batch', '_scvi_labels', 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_pr
ePB', 'B_IFN', 'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC',
'DC_CCR7+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macro
phages_M1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4
+', 'T_CD4+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_C
D8+_cytotoxic', 'T_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', 'l
eiden', 'region_cluster', 'mean_nUMI_factorsfact_0', 'mean_nUMI_factorsf
act_1', 'mean_nUMI_factorsfact_2', 'mean_nUMI_factorsfact_3', 'mean_nUMI
_factorsfact_4', 'mean_nUMI_factorsfact_5', 'mean_nUMI_factorsfact_6',
'mean_nUMI_factorsfact_7', 'mean_nUMI_factorsfact_8', 'mean_nUMI_factors
fact_9', 'mean_nUMI_factorsfact_10', 'mean_nUMI_factorsfact_11', 'region
_name'
    var: 'feature_types', 'genome', 'symbol', 'MT_gene'
    uns: '_scvi_manager_uuid', '_scvi_uuid', 'leiden', 'mod', 'mod_coloc
_n_fact11', 'mod_coloc_n_fact12', 'neighbors', 'region_cluster_colors',
'sample_colors', 'spatial', 'umap'
    obsm: 'MT', 'X_umap', 'means_cell_abundance_w_sf', 'q0001_cell_abund
ance_w_sf', 'q05_cell_abundance_w_sf', 'q50_cell_abundance_w_sf', 'q95_c
ell_abundance_w_sf', 'spatial', 'stds_cell_abundance_w_sf'
    obsp: 'connectivities', 'distances'
    layers: 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IFN', 'B
_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC', 'DC_CCR7+', 'DC_
cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macrophages_M1', 'Mac
rophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4+', 'T_CD4+_TfH',
'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_CD8+_cytotoxic', 'T
_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', None (.X)
```

We can use the location composition similarity graph to build a joint integrated UMAP representation of all section/Visium batches.

In [95]:
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[95]:
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [98]:
```
%matplotlib inline

c2l.umap_spatial_3plot(adata_vis, root_figure);
```



In [ ]:

Grouped the panel genes into five cell-type markers:

1. pan-B (CD19/MS4A1/CD79A/B/PAX5/CD22)
2. follicular mantle B (IGHD/IGHM)
3. pan-lymphocyte (CD52/CD53)
4. TIM3+ T, GC TfH, Treg, GC DZ B (CD27)
5. monocytes/macrophages (CD14)

CDS2 doesn't mark any clear cell type.

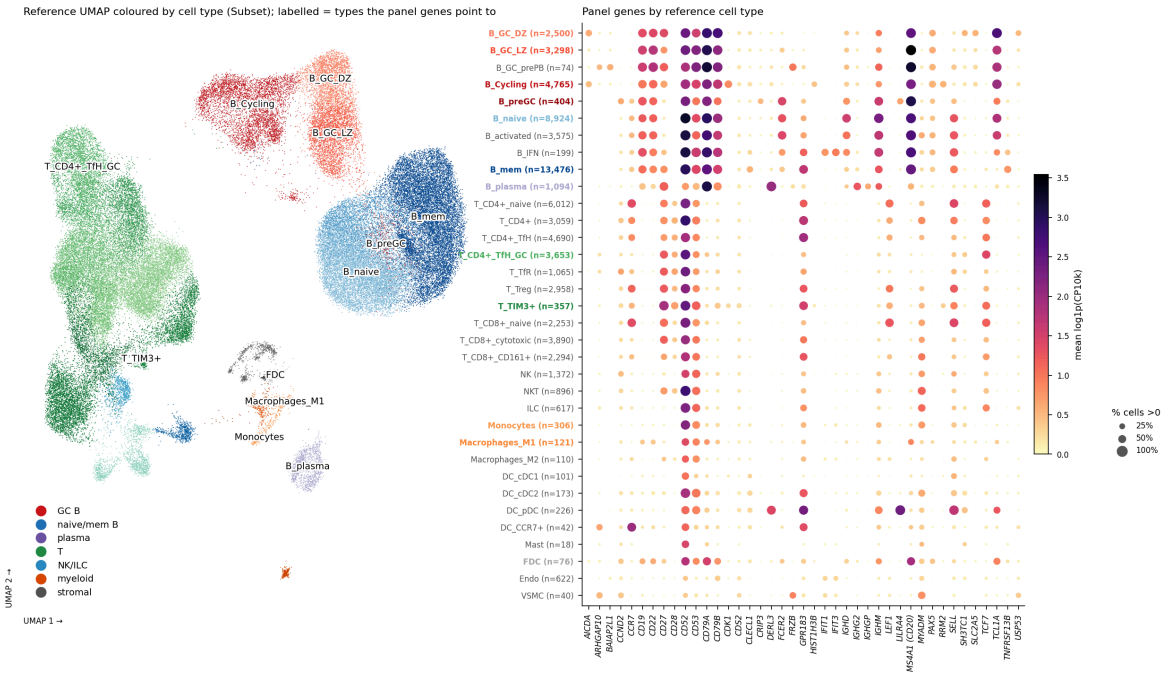
In [102…
```
ct = adata_ref.obs["Subset"].astype(str).values
np.unique(ct)
```

Out[102…
```
array(['B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IFN',
       'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC',
       'DC_CCR7+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC',
       'Macrophages_M1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK',
       'NKT', 'T_CD4+', 'T_CD4+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive',
       'T_CD8+_CD161+', 'T_CD8+_cytotoxic', 'T_CD8+_naive', 'T_TIM3+',
       'T_TfR', 'T_Treg', 'VSMC'], dtype=object)
```

In [106…
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[106…
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [108…
```
%matplotlib inline

# row order for the dot plot: cell types grouped by lineage
lineage = {
    "GC B":        ["B_GC_DZ","B_GC_LZ","B_GC_prePB","B_Cycling","B_preGC
    "naive/mem B": ["B_naive","B_activated","B_IFN","B_mem"],
    "plasma":      ["B_plasma"],
    "T":           ["T_CD4+_naive","T_CD4+","T_CD4+_TfH","T_CD4+_TfH_GC",
                    "T_CD8+_naive","T_CD8+_cytotoxic","T_CD8+_CD161+"],
    "NK/ILC":      ["NK","NKT","ILC"],
    "myeloid":     ["Monocytes","Macrophages_M1","Macrophages_M2","DC_cDC
    "stromal":     ["FDC","Endo","VSMC"],
}

focus=["B_GC_LZ","B_GC_DZ","B_Cycling","B_preGC","B_naive","B_mem","B_pla


# Figure 1: UMAP by cell type + dot plot
dfg = c2l.umap_cell_type_plot(adata_vis, adata_ref, genes_plot, lineage,
```



In [109…
```
print(dfg.shape)
dfg.head(4)
```

```
(328, 6)
```

Out[109…

	tissue	age_group	gene	n_cells	pct_expr	mean_log1p_cp10k
0	Tonsil	babies	AICDA	16237	11.5	0.163
1	Tonsil	babies	ARHGAP10	16237	4.4	0.064
2	Tonsil	babies	BAIAP2L1	16237	1.9	0.025
3	Tonsil	babies	CCND2	16237	7.6	0.133

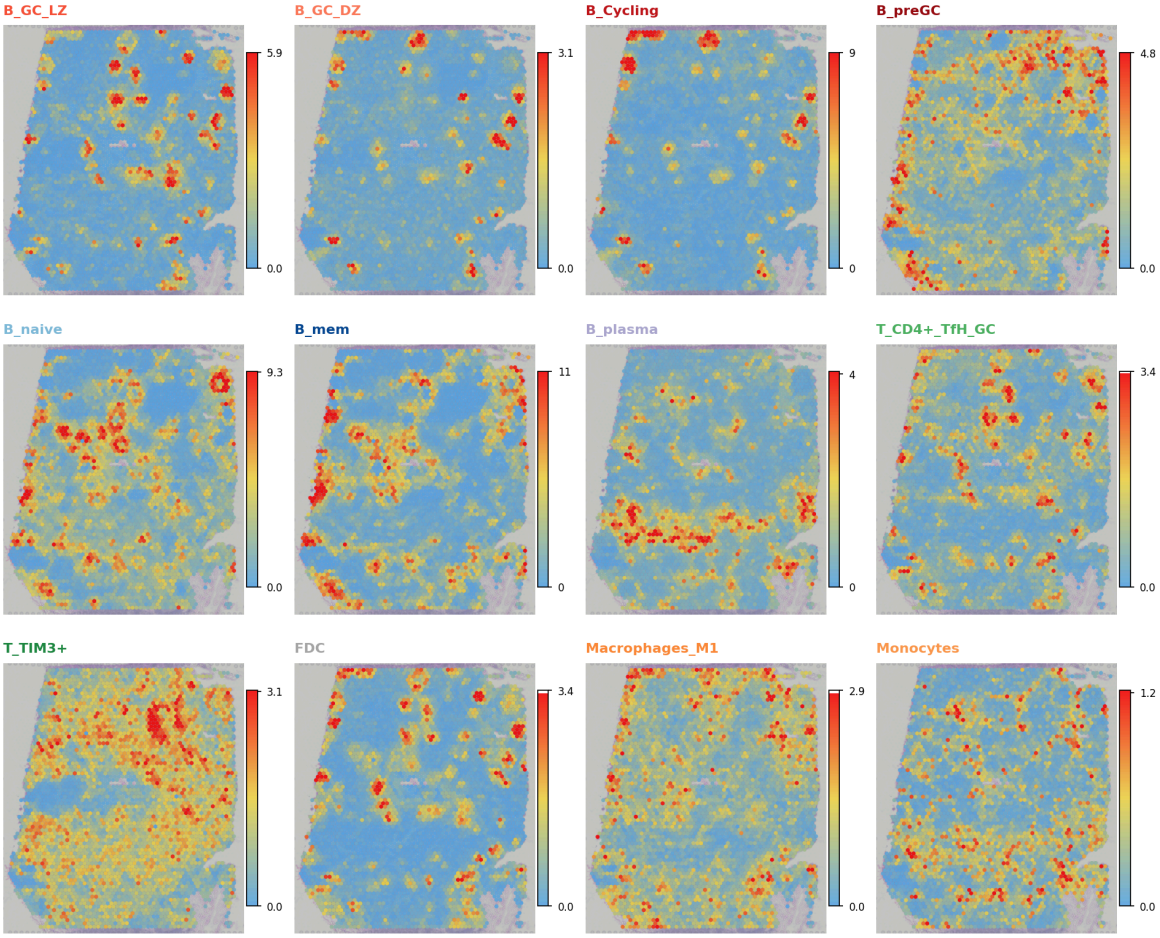
In [ ]:
```
```

Figure 2: spatial maps

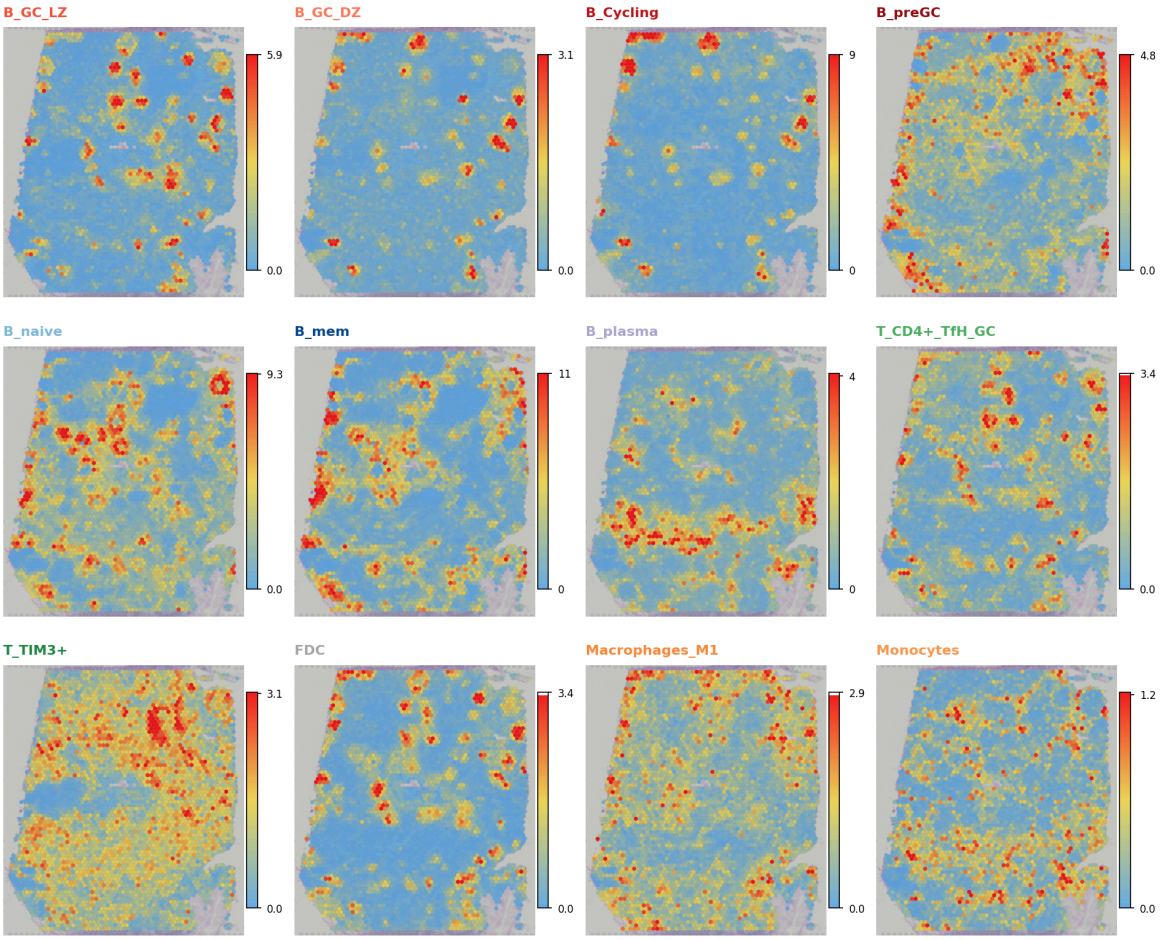
In [110…
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[110…
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [112…
```
%matplotlib inline

c2l.spatial_plot_bcell_types(adata_vis, focus, lineage, root_figure)
```



cell2location q05 abundance (cells per spot, 'at least this many present'), clipped at the 99.2th percentile per cell type; V1_Human_Lymph_Node, 4,035 spots.



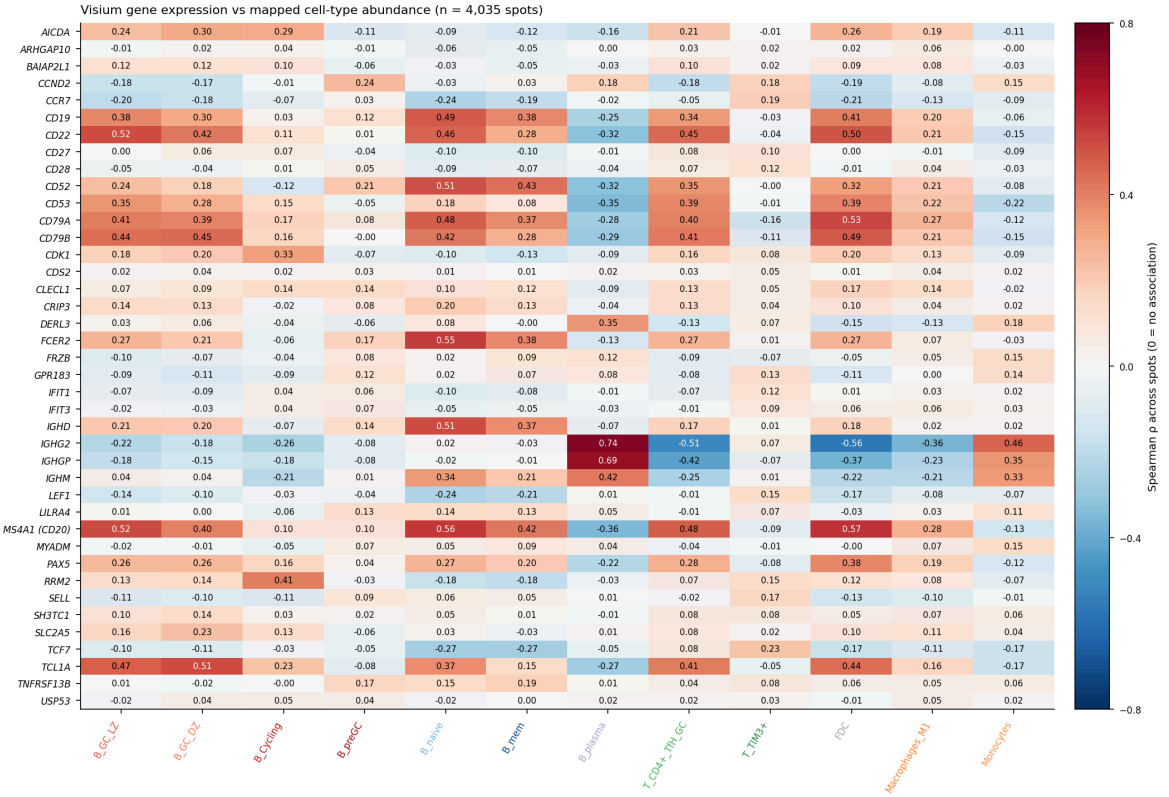
cell2location q05 abundance (cells per spot, 'at least this many present'), clipped at the 99.2th percentile per cell type; V1_Human_Lymph_Node, 4,035 spots.

Figure 3: gene (Visium) vs cell-type abundance correlation

In [113…
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[113…
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [116…
```
%matplotlib inline

df_rho = c2l.spatial_plot_cell_abundance(adata_vis, adata_ref, genes_plot
```



In [117…
```
df_rho.head(6)
```

Out[117…

	B_GC_LZ	B_GC_DZ	B_Cycling	B_preGC	B_naive	B_mem	B_plasma	T_CI
AICDA	0.241	0.304	0.289	-0.107	-0.091	-0.118	-0.163	
ARHGAP10	-0.009	0.025	0.036	-0.014	-0.064	-0.055	0.000	
BAIAP2L1	0.119	0.118	0.095	-0.057	-0.029	-0.054	-0.034	
CCND2	-0.179	-0.171	-0.009	0.245	-0.034	0.031	0.183	
CCR7	-0.199	-0.179	-0.071	0.028	-0.238	-0.191	-0.016	
CD19	0.382	0.298	0.029	0.125	0.486	0.378	-0.252	

In [ ]:
```
```

Identifying cellular compartments / tissue zones using matrix factorisation (NMF)

Here, we use the cell2location mapping results to identify the spatial co-occurrence of cell types in order to better understand the tissue organisation and predict cellular interactions. We performed non-negative matrix factorization (NMF) of the cell type abundance estimates from cell2location (paper section 4, Fig 4D). Similar to the established benefits of applying NMF to conventional scRNA-seq, the additive NMF decomposition yielded a grouping of spatial cell type abundance profiles into components that capture co-localised cell types (Supplemenary Methods section 4.2, p. 60). This NMF-based decomposition naturally accounts for the fact that multiple cell types and microenvironments can co-exist at the same Visium locations (see paper Fig S20, p. 34), while sharing information across tissue areas (e.g. individual germinal centres).

Tip

In practice, it is better to train NMF for a range of factors $R = 5, . . , 30$ and select R as a balance between capturing fine-grained and splitting known well-established tissue zones.

If you want to find a few most disctinct cellular compartments, use a small number of factors.
If you want to find very strong co-location signal and assume that most cell types don't co-locate, use a lot of factors (> 30 - used here).

Below we show how to perform this analysis. To aid this analysis, we wrapped the analysis shown the notebook on advanced downstream analysis into a pipeline that automates training of the NMF model with varying number of factors:

In [118…

```
from cell2location import run_colocation
import pickle

fname = "res_dict_coloc.pkl"
filename = root_results / fname

if filename.exists():
    with open(filename, "rb") as fh:
      res_dict = pickle.load(fh)
    print(f"Results loaded from {filename}")
else:
    res_dict, adata_vis = run_colocation(
        adata_vis,
        model_name='CoLocatedGroupsSklearnNMF',
        train_args={
          'n_fact': np.arange(11, 13), # IMPORTANT: use a wider range of
          'sample_name_col': 'sample', # columns in adata_vis.obs that id
          'n_restarts': 3 # number of training restarts
        },
        # the hyperparameters of NMF can be also adjusted:
        model_kwargs={'alpha': 0.01, 'init': 'random', "nmf_kwd_args": {"
        export_args={'path': f'{run_name}/CoLocatedComb/'}
    )

    with open(filename, "wb") as fh:
```

```
        pickle.dump(res_dict, fh)
    print(f"Results saved to {filename}")
```

```
Results loaded from /home/flavio/uv/squidpy/results/res_dict_coloc.pkl
```

In [ ]:

For every factor number, the model produces the following list of folder outputs:

`cell_type_fractions_heatmap/` : a dot plot of the estimated NMF weights of cell types (rows) across NMF components (columns)

`cell_type_fractions_mean/` : the data used for dot plot

`factor_markers/` : tables listing top 10 cell types most speficic to each NMF factor

`models/` : saved NMF models

`predictive_accuracy/` : 2D histogram plot showing how well NMF explains cell2location output

`spatial/` : NMF weights across locatinos in spatial coordinates

`location_factors_mean/` : the data used for the plot in spatial coordiantes

`stability_plots/` : stability of NMF weights between training restarts

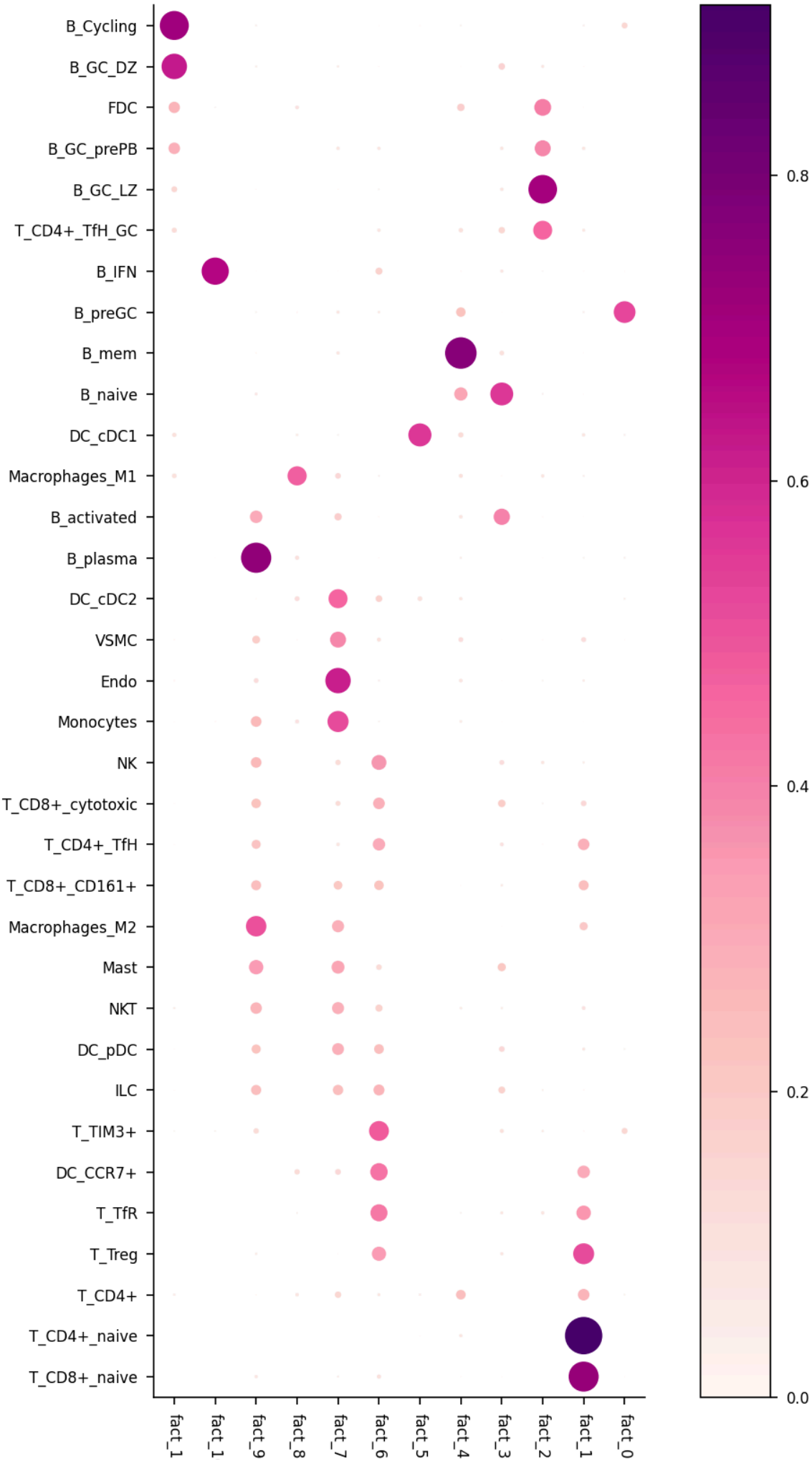
Key output that you want to examine are the files in `cell_type_fractions_heatmap/` which show a dot plot of the estimated NMF weights of cell types (rows) across NMF components (columns) which correspond to cellular compartments. Shown are relative weights, normalized across components for every cell type.

Tip

The NMF model output such as factor loadings are stored in `adata.uns[f"mod_coloc_n_fact{n_fact}"]` in a similar output format as main cell2location results in `adata.uns['mod']`.

In [119…

```
# Here we plot the NMF weights (Same as saved to `cell_type_fractions_hea
res_dict['n_fact12']['mod'].plot_cell_type_loadings()
```


B_Cycling
B_GC_DZ
FDC
B_GC_prePB
B_GC_LZ
T_CD4+_TfH_GC
B_IFN
B_preGC
B_mem
B_naive
DC_cDC1
Macrophages_M1
B_activated
B_plasma
DC_cDC2
VSMC
Endo
Monocytes
NK
T_CD8+_cytotoxic
T_CD4+_TfH
T_CD8+_CD161+
Macrophages_M2
Mast
NKT
DC_pDC
ILC
T_TIM3+
DC_CCR7+
T_TfR
T_Treg
T_CD4+
T_CD4+_naive
T_CD8+_naive
fact_1
fact_1
fact_9
fact_8
fact_7
fact_6
fact_5
fact_4
fact_3
fact_2
fact_1
fact_0
0.8
0.6
0.4
0.2
0.0

Estimate cell-type specific expression of every gene in the spatial data (needed for NCEM)

The cell-type specific expression of every gene at every spatial location in the spatial data enables learning cell communication with NCEM model using Visium data (https://github.com/theislab/ncem).

To derive this, we adapt the approach of estimating conditional expected expression proposed by RCTD (Cable et al) method.

With cell2location, we can look at the posterior distribution rather than just point estimates of cell type specific expression (see `mod.samples.keys()` and next section on using full distribution).

Note that this analysis requires substantial amount of RAM memory and thefore doesn't work on free Google Colab (12 GB limit).

In [ ]:
```
mod
```

In [120…
```
print(list(mod.samples.keys()))
```

```
['post_sample_q05', 'post_sample_q50', 'post_sample_q95', 'post_sample_q00
01']
```

In [121…
```
adata_vis.layers
```

Out[121…
```
Layers with keys: 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IF
N', 'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC', 'DC_CCR7
+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macrophages_M
1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4+', 'T_CD4
+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_CD8+_cytoto
xic', 'T_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', None
```

In [122…
```
'''
# Compute expected expression per cell type
expected_dict = mod.module.model.compute_expected_per_cell_type(
    mod.samples["post_sample_q05"], mod.adata_manager
)

# Add to anndata layers
for i, n in enumerate(mod.factor_names_):
    adata_vis.layers[n]= expected_dict['mu'][i]
'''

# Save anndata object with results (adata_vis_filename_run, not adata_ref
adata_vis.write(adata_vis_filename_run)
adata_vis_filename_run
```

Out[122…
```
PosixPath('/home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2loc
ation_map/sp.h5ad')
```

Plotting cell-type specific expression of genes in spatial coordinates.

Below we plot the cell-type specific expression of genes (rows, second to last columns) compared to total expression of those genes (first column).

Here we highlight *CD3D*, pan T-cell marker expressed by 2 subtypes of T cells in distinct locations but not expressed by co-located B cells, that instead express *CR2* gene.

In [123…
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[123…
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [125…
```
# list cell types and genes for plotting
ctypes = ['T_CD4+_TfH_GC', 'T_CD4+_naive', 'B_GC_LZ']
genes = ['CD3D', 'CR2']

with mpl.rc_context({'axes.facecolor':  'black'}):
    # select one slide
    slide = select_slide(adata_vis, 'V1_Human_Lymph_Node')

    # from tutorial_utils import plot_genes_per_cell_type
    c2l.plot_genes_per_cell_type(slide, genes, ctypes)
```

In [126…
```
slide = select_slide(adata_vis, 'V1_Human_Lymph_Node')
```

In [127…
```
print(slide.var.shape)
slide.var.head(3)
```

```
(10217, 4)
```

Out[127…

gene_ids	feature_types	genome	symbol	MT_gene
ENSG00000000419	Gene Expression	GRCh38	DPM1	False
ENSG00000000457	Gene Expression	GRCh38	SCYL3	False
ENSG00000000938	Gene Expression	GRCh38	FGR	False

In [128…
```
slide.layers
```

Out[128…
```
Layers with keys: 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IF
N', 'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC', 'DC_CCR7
+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macrophages_M
1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4+', 'T_CD4
+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_CD8+_cytoto
xic', 'T_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', None
```

Note that `plot_genes_per_cell_type` function often need customization so it is not included into cell2location package - you need to copy it from https://github.com/BayraktarLab/cell2location/blob/master/docs/notebooks/tutorial_utils.p to use on your system.

In [129…
```
cell2location.__version__
```

Out[129…
```
'0.1.5'
```

In [ ]:
```
```